DSA5208: Scalable Distributed Computing for Data Science

Lecture 5: Distributed Algorithms in Apache Spark

Zhenning Cai

Department of Mathematics, National University of Singapore

AY2026/2027, Semester 1

1 Sorting algorithms

2 Joins

3 Adaptive Query Execution (AQE)

4 Summary

1 Sorting algorithms
- Sort in Apache Spark
- Sort in CUDA-X

2 Joins
- Types of joins
- Join algorithms in Spark

3 Adaptive Query Execution (AQE)

4 Summary

Distributed sorting problem

Problem: Assume that in a distributed system with n processes, each process p_k, $k = 1, \cdots, n$ stores a list of data:

$$\{x_{k,1}, x_{k,2}, \cdots, x_{k,m_k}\}.$$

Sort the data in an ascending order such that the output of the kth process

$$\{y_{k,1}, y_{k,2}, \cdots, y_{k,m'_k}\}$$

satisfies

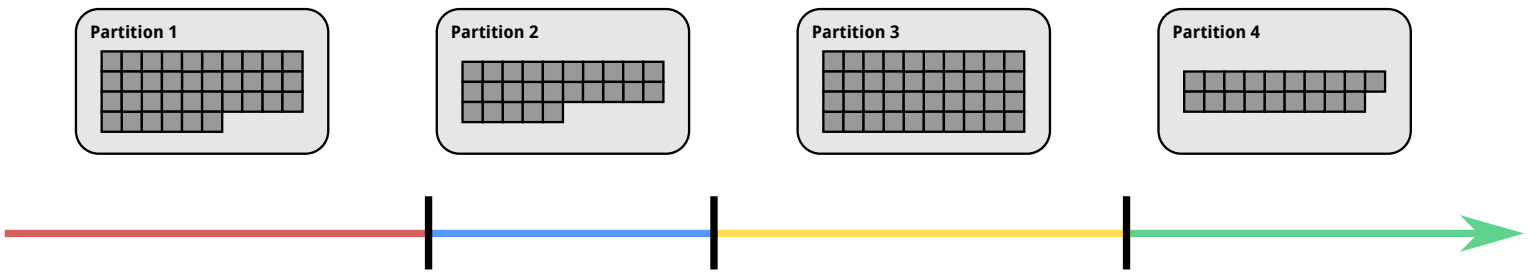
$$y_{k-1,m'_{k-1}} \leqslant y_{k,1} \leqslant y_{k,2} \leqslant \cdots \leqslant y_{k,m'_k} \leqslant y_{k+1,1},$$

where $y_{0,m'_0} = -\infty$ and $y_{n+1,1} = +\infty$.

How does Spark solve this problem?

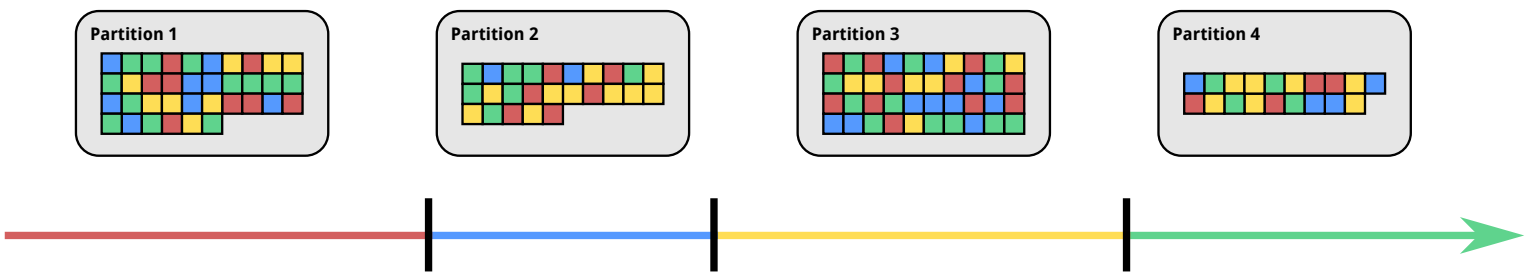
General idea of the sorting algorithm

1. Determine n ranges $R_k = [r_{k-1}, r_k)$, $k = 1, \cdots, n$.



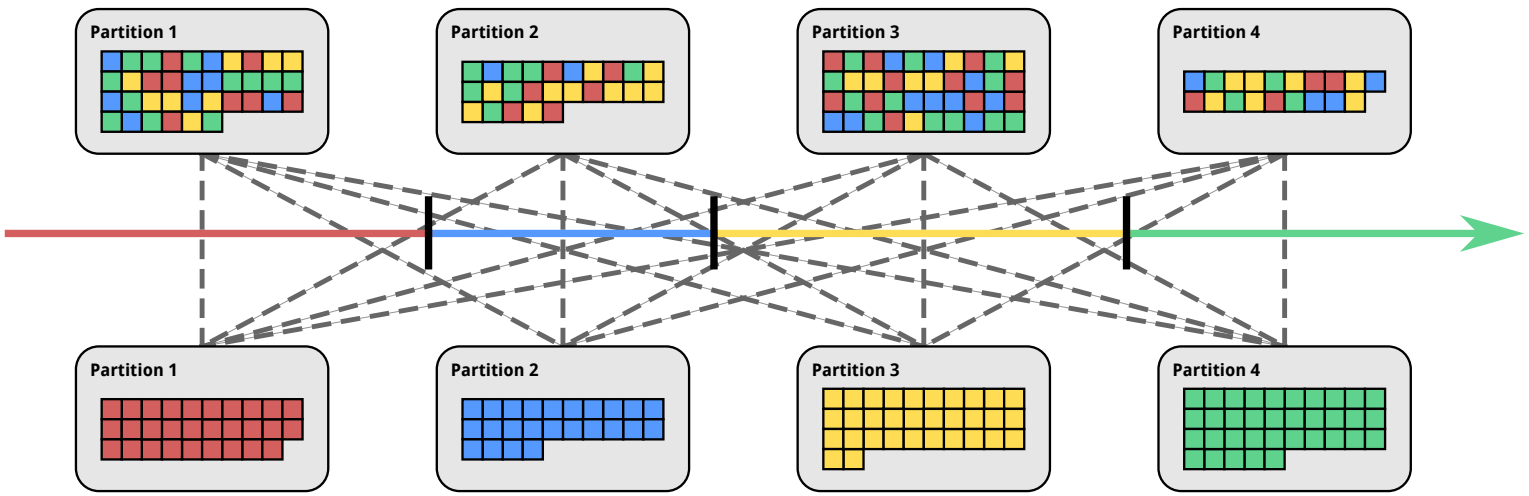
General idea of the sorting algorithm

1. Determine n ranges $R_k = [r_{k-1}, r_k)$, $k = 1, \cdots, n$.
2. For each process p_k, send all $x_{k,j}$ satisfying $x_{k,j} \in R_i$ to the ith process p_i.



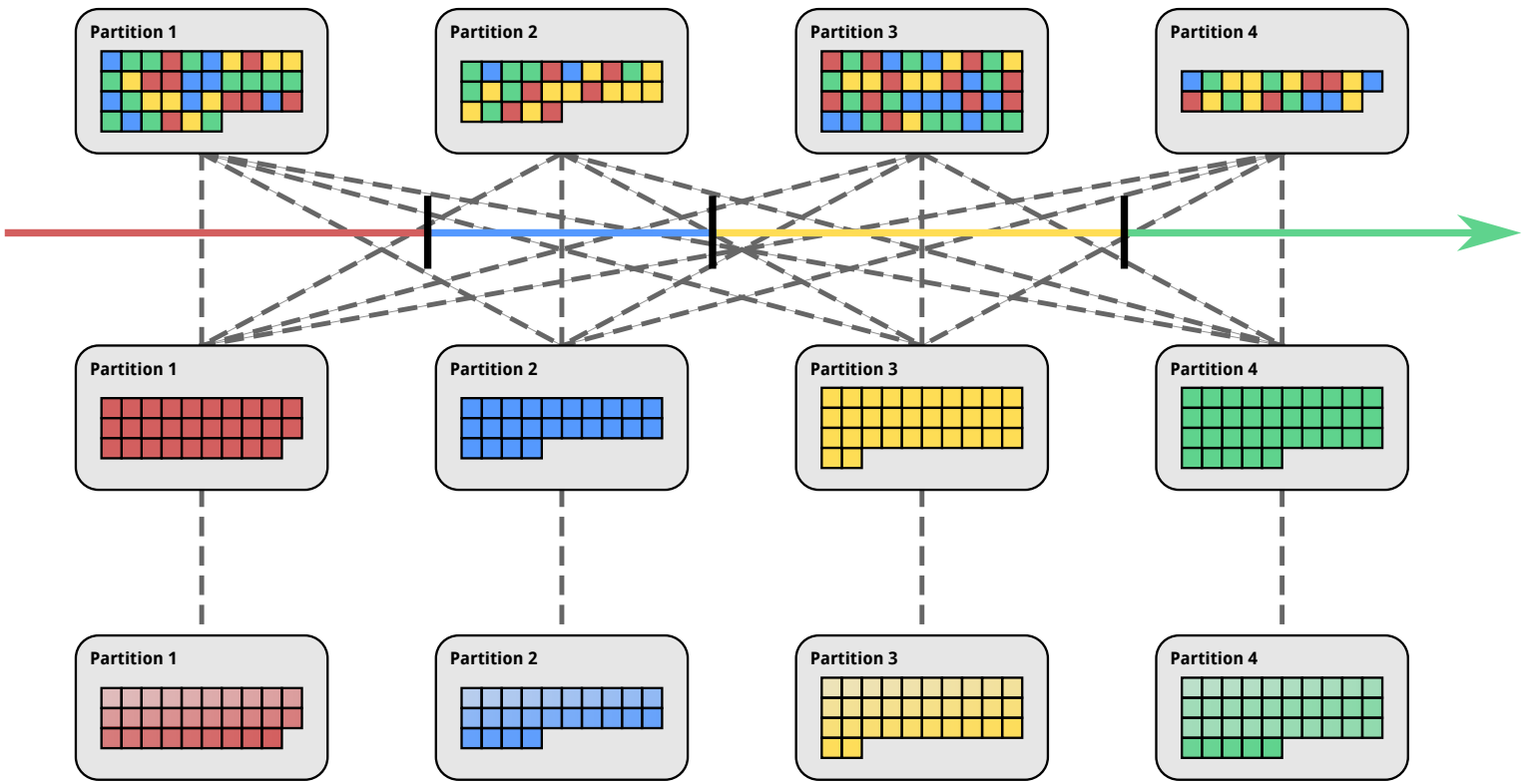
General idea of the sorting algorithm

1. Determine n ranges $R_k = [r_{k-1}, r_k)$, $k = 1, \cdots, n$.
2. For each process p_k, send all $x_{k,j}$ satisfying $x_{k,j} \in R_i$ to the ith process p_i.



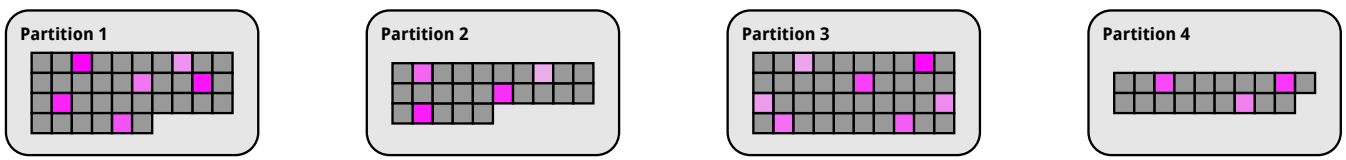
General idea of the sorting algorithm

1. Determine n ranges $R_k = [r_{k-1}, r_k)$, $k = 1, \cdots, n$.
2. For each process p_k, send all $x_{k,j}$ satisfying $x_{k,j} \in R_i$ to the ith process p_i.
3. Sort the data within each process in an ascending order.



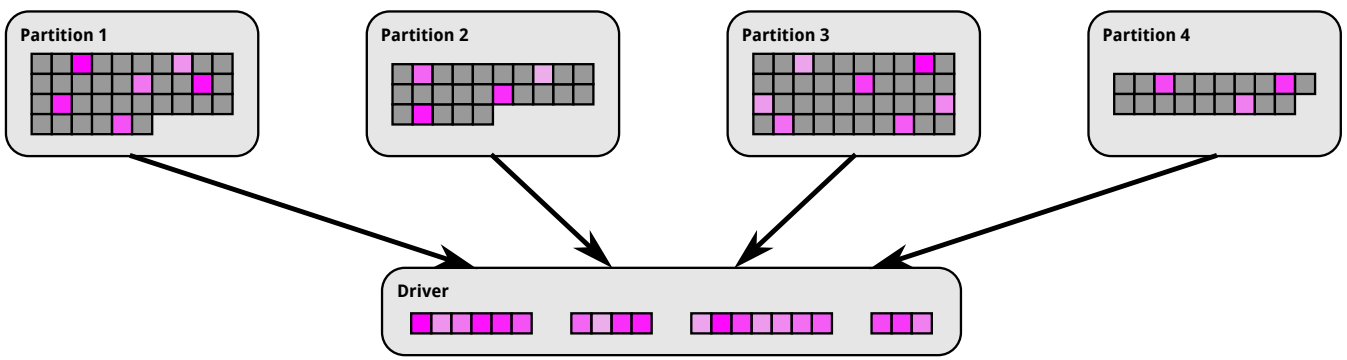
Determine the range boundaries

1. Randomly sample some values in each partition.



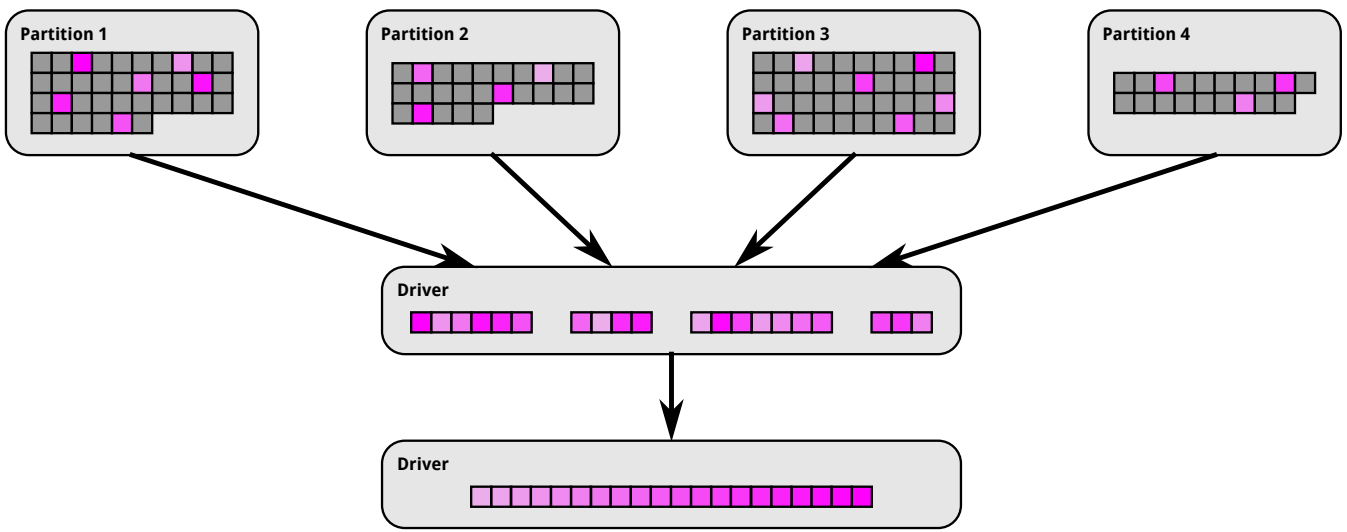
Determine the range boundaries

1. Randomly sample some values in each partition.
2. Collect the values to the driver node.



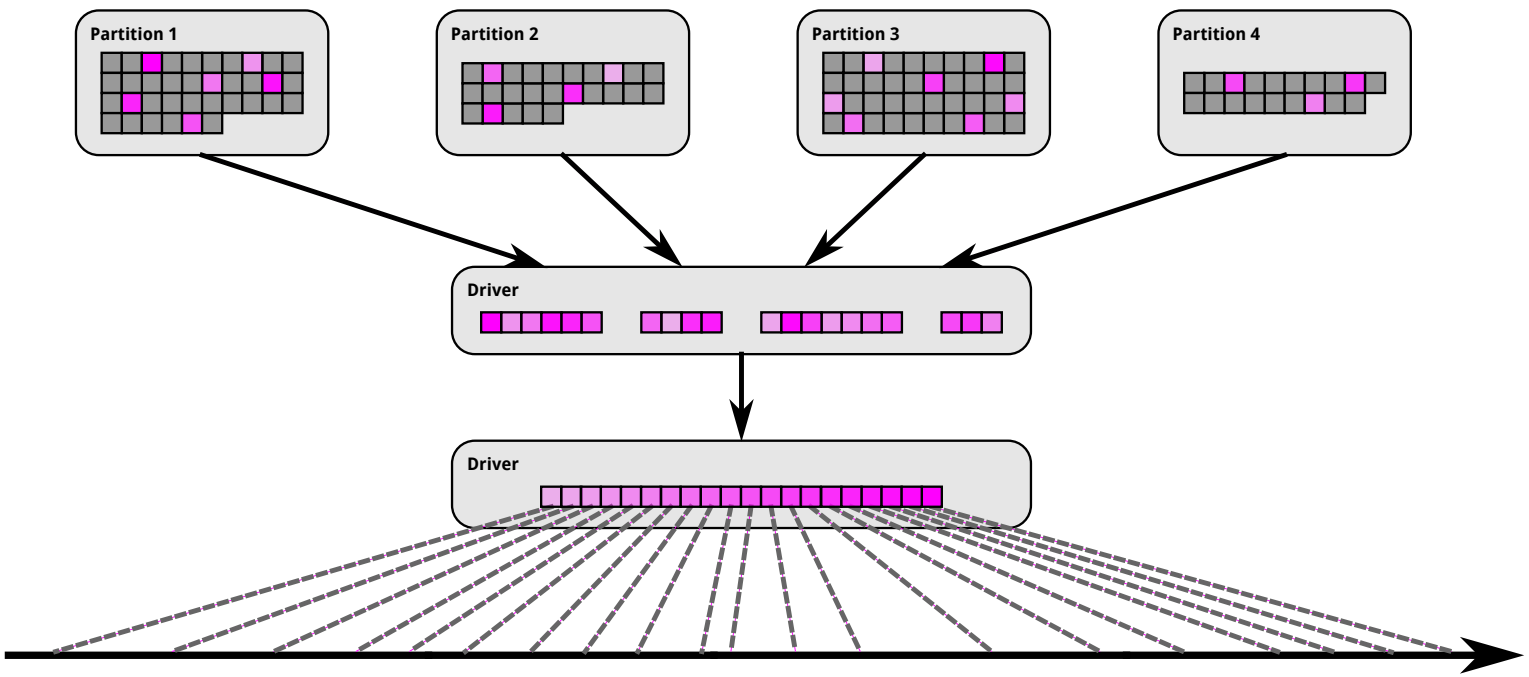
Determine the range boundaries

1. Randomly sample some values in each partition.
2. Collect the values to the driver node.
3. Sort the data on the driver node.



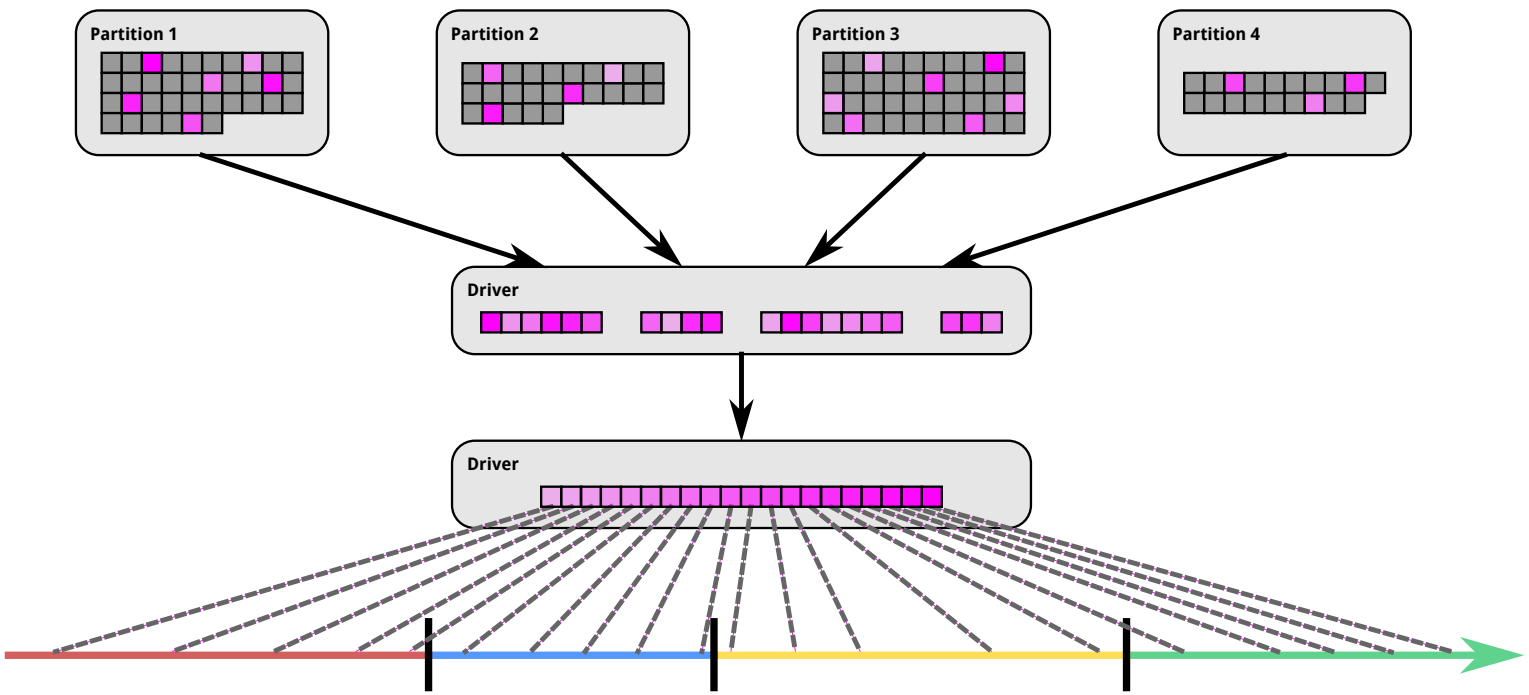
Determine the range boundaries

1. Randomly sample some values in each partition.
2. Collect the values to the driver node.
3. Sort the data on the driver node.
4. Determine the range boundaries such that each interval contains roughly the same number of samples.



Determine the range boundaries

1. Randomly sample some values in each partition.
2. Collect the values to the driver node.
3. Sort the data on the driver node.
4. Determine the range boundaries such that each interval contains roughly the same number of samples.



Sampling algorithm

Difficulty: It may be expensive to get the total number of values in each partition!

Sampling algorithm

Difficulty: It may be expensive to get the total number of values in each partition!

Question: Without knowing m_k, how to sample K values from the dataset $\{x_{k,1}, x_{k,2}, \cdots, x_{k,m_k}\}$ so that the probability that each value is chosen is the same?

Sampling algorithm

Difficulty: It may be expensive to get the total number of values in each partition!

Question: Without knowing m_k, how to sample K values from the dataset $\{x_{k,1}, x_{k,2}, \cdots, x_{k,m_k}\}$ so that the probability that each value is chosen is the same?

Sampling algorithm:

1. For a given number K (default value: 60), use reservior sampling to sample K values from each partition. Meanwhile, get the value of m_k for each partition.
2. If m_k is too large (the kth partition has too many data), resample from the kth partition using Bernoulli sampling to guarantee that enough items are collected.

Reservior sampling

Reservior sampling allows us to randomly take K samples from the set $\{x_1, x_2, \cdots\}$ without knowing the total number of elements.

1. Initialize the reservoir $R = \{r_1, r_2, \cdots, r_K\}$, where $r_k = x_k$ for $k = 1, \cdots, K$.
2. For each new input x_i, $i > K$, generate a random number j uniformly in $\{1, \cdots, i\}$. Set r_j to be x_i if $j \leqslant K$.
3. Return the reservoir R.

Reservior sampling

Reservior sampling allows us to randomly take K samples from the set $\{x_1, x_2, \cdots\}$ without knowing the total number of elements.

1. Initialize the reservoir $R = \{r_1, r_2, \cdots, r_K\}$, where $r_k = x_k$ for $k = 1, \cdots, K$.
2. For each new input x_i, $i > K$, generate a random number j uniformly in $\{1, \cdots, i\}$. Set r_j to be x_i if $j \leqslant K$.
3. Return the reservoir R.

Proof: Let m be the total number of elements.

- If $m = K$, then $P(x_i \text{ is selected}) = 1$ for all i.
- Assume that when $m = M$, we have $P(x_i \text{ is selected}) = K/M$ for all i. Then when $m = M + 1$,

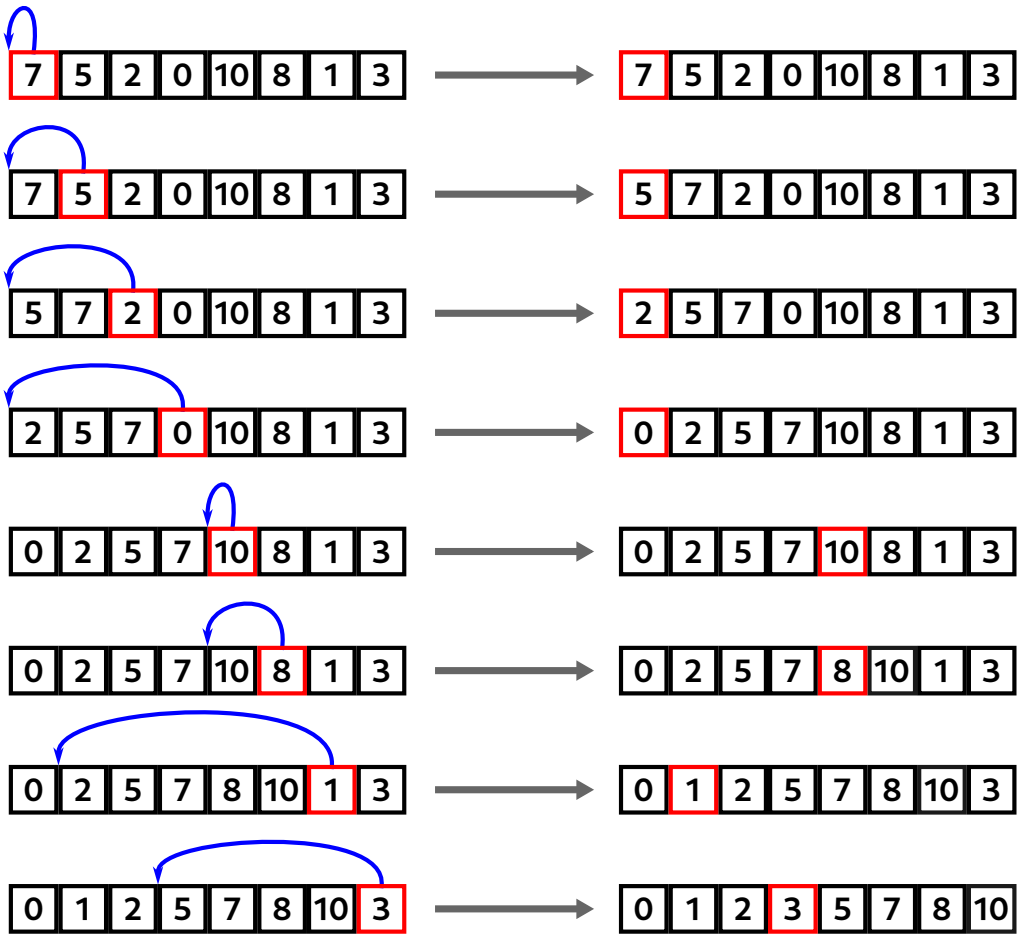
$$P(x_i \text{ is selected}) =$$

$$\begin{cases} P(x_i \text{ has been selected}) \cdot P(j \neq i) = \frac{K}{M} \cdot \frac{M}{M+1} = \frac{K}{M+1}, & \text{if } i \leqslant M, \\ P(j \leqslant K) = \frac{K}{M+1}, & \text{if } i = M+1. \end{cases}$$

Sorting within a partition

When the length of the array is small (less than 32 or 64), Spark uses binary insertion sort:

- Locations of elements are found by binary search.
- Worst-case performance: $O(n^2)$
- Extra space required: $O(1)$
- The method is efficient when n is small.



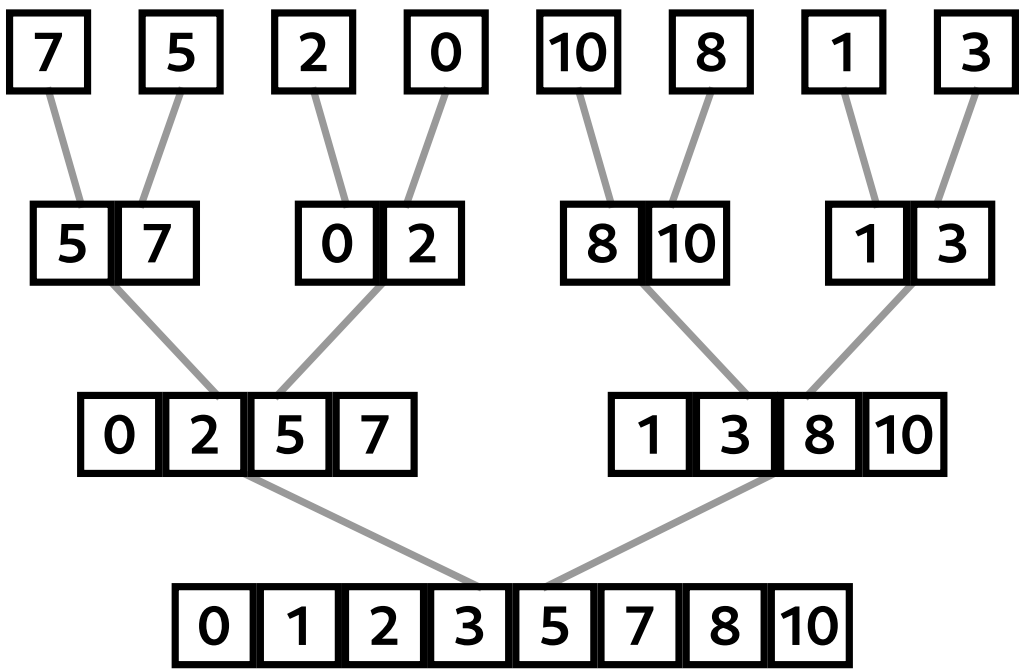
Sorting within a partition

For larger arrays, Spark uses Timsort to sort the data. Timsort is an efficient sorting algorithm derived from binary insertion sort and merge sort.

Sorting within a partition

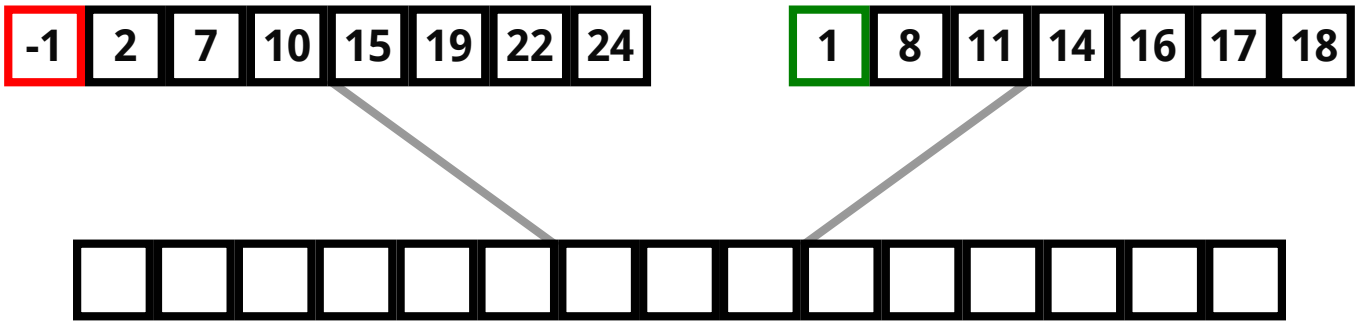
For larger arrays, Spark uses Timsort to sort the data. Timsort is an efficient sorting algorithm derived from binary insertion sort and merge sort.

Merge sort:



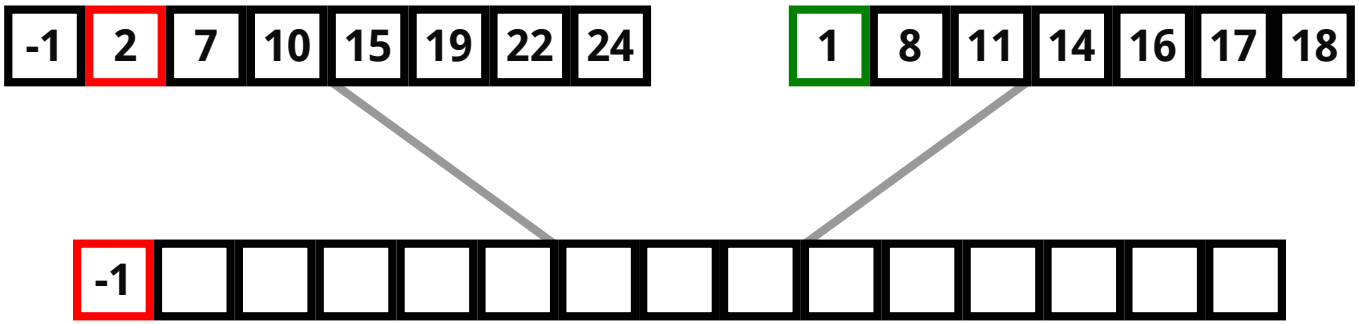
How to merge two sorted chunks?

Method 1:



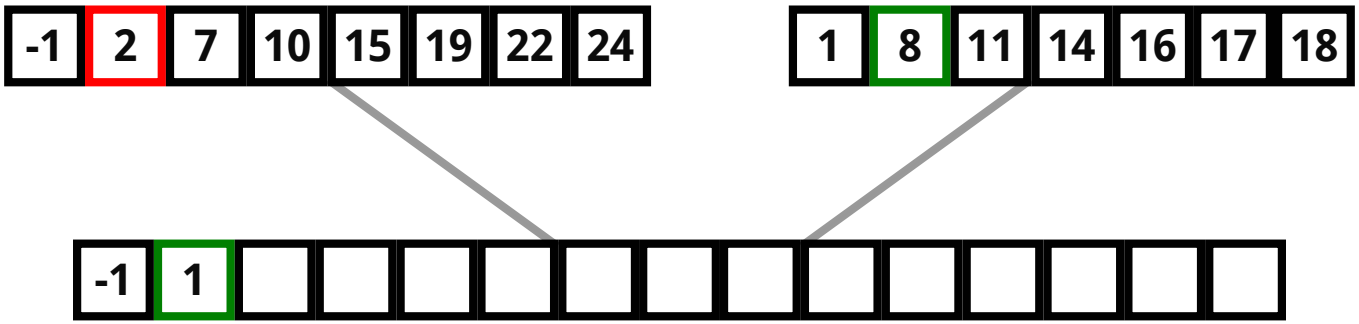
How to merge two sorted chunks?

Method 1:



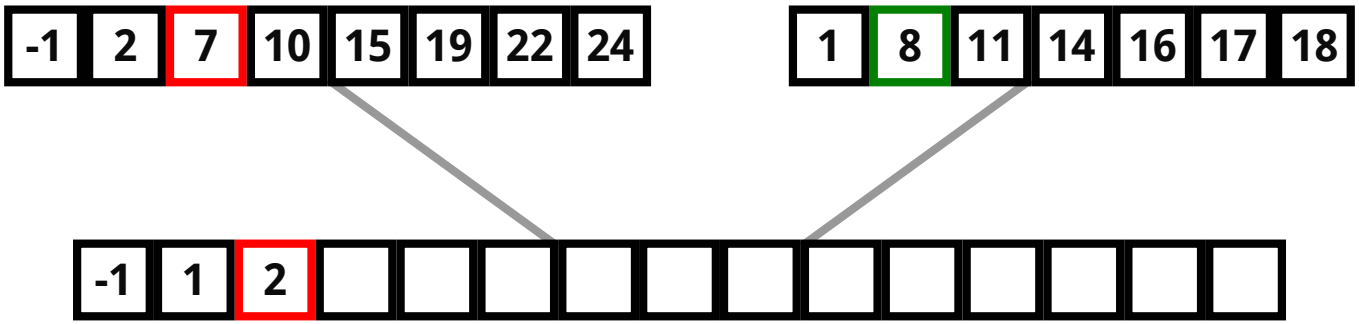
How to merge two sorted chunks?

Method 1:



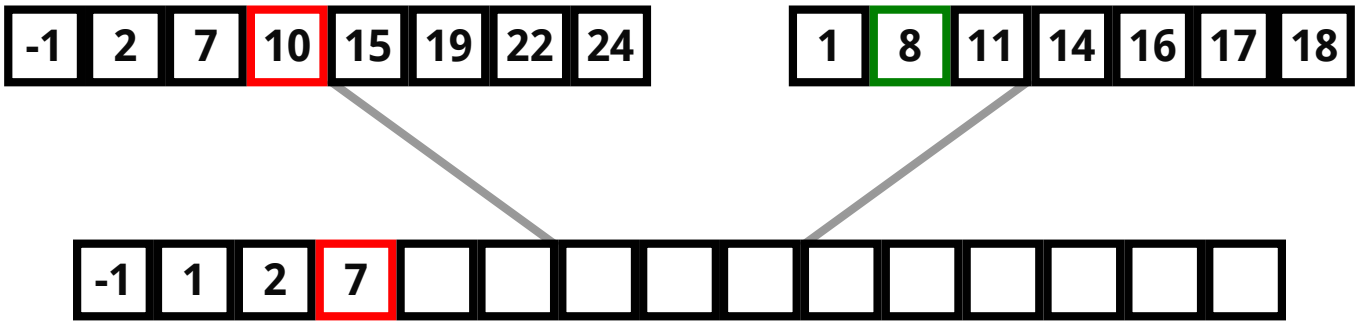
How to merge two sorted chunks?

Method 1:



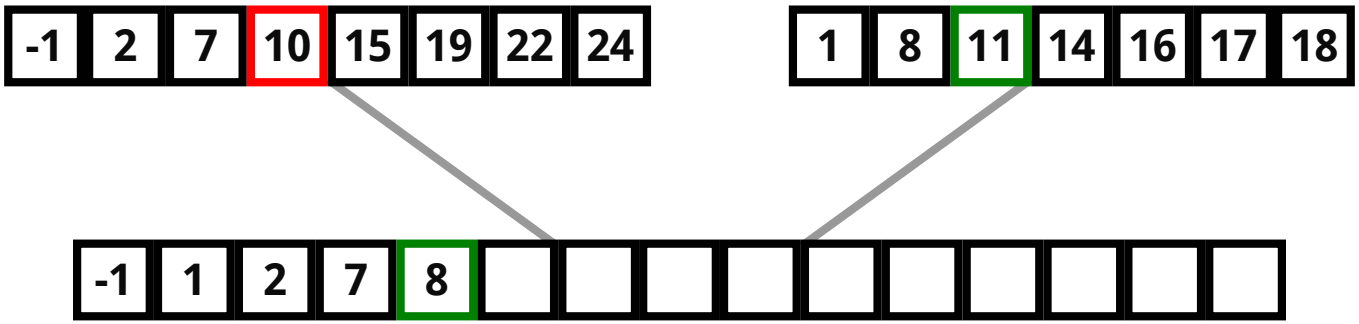
How to merge two sorted chunks?

Method 1:



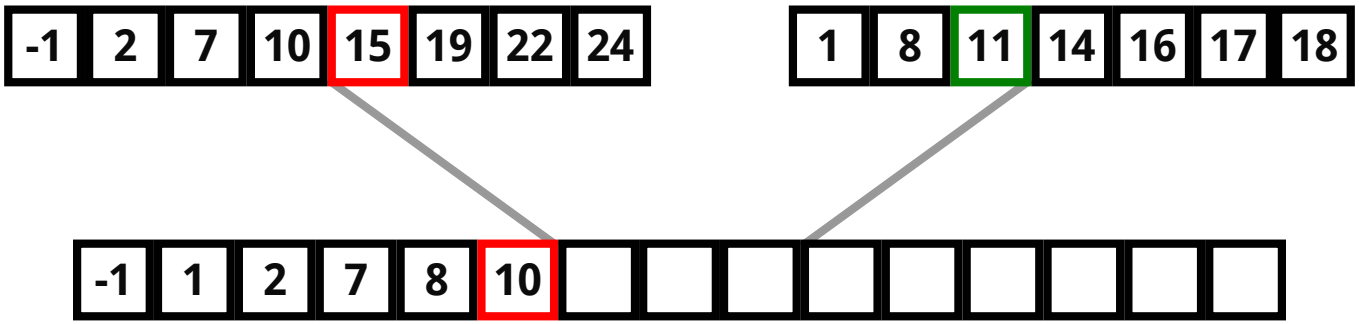
How to merge two sorted chunks?

Method 1:



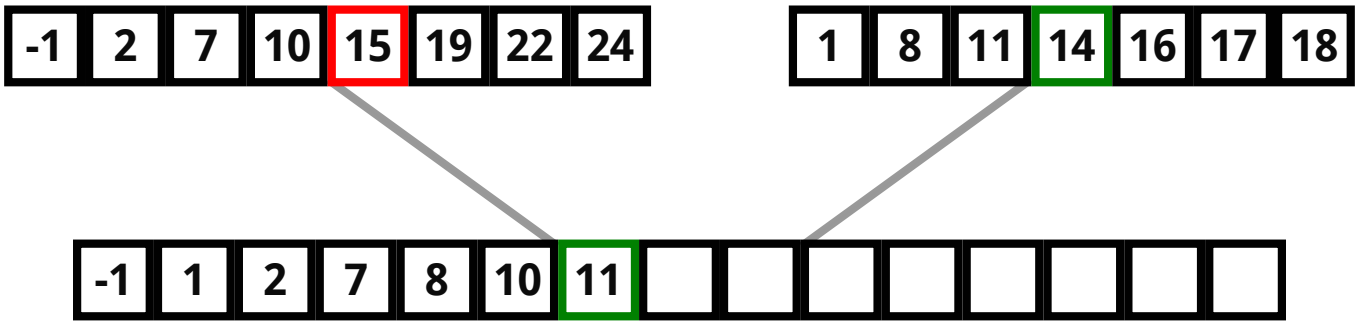
How to merge two sorted chunks?

Method 1:



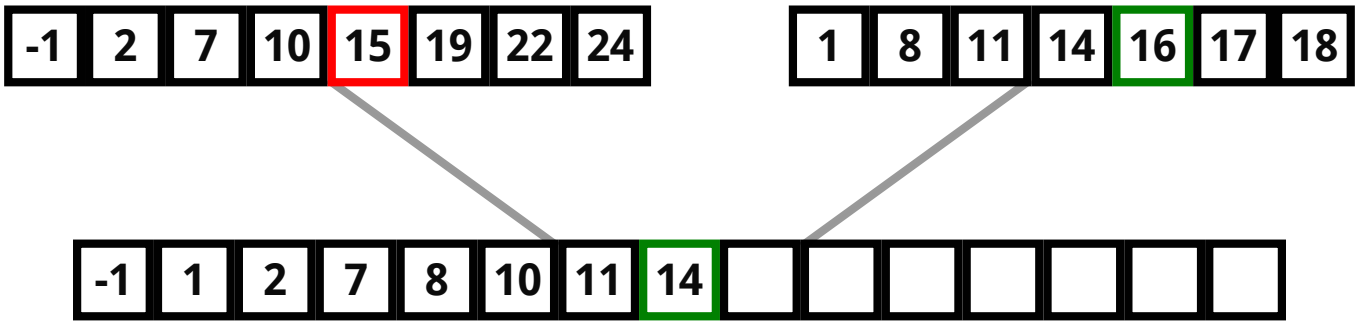
How to merge two sorted chunks?

Method 1:



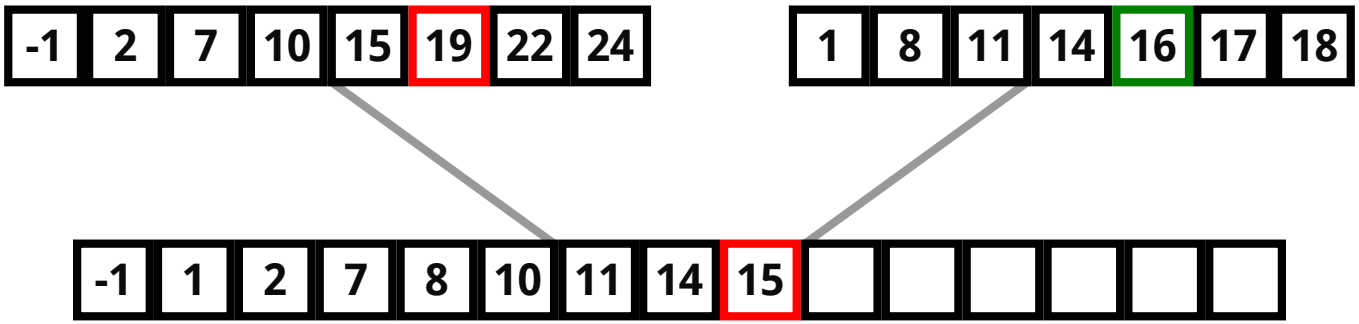
How to merge two sorted chunks?

Method 1:



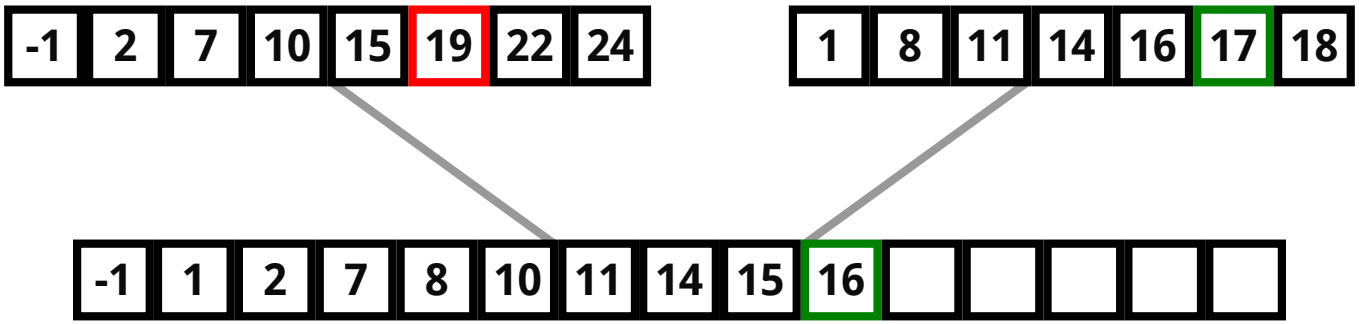
How to merge two sorted chunks?

Method 1:



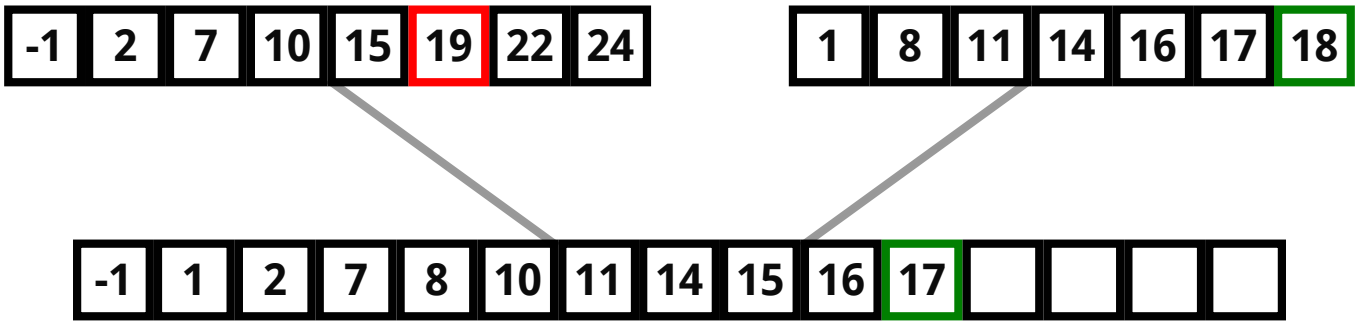
How to merge two sorted chunks?

Method 1:



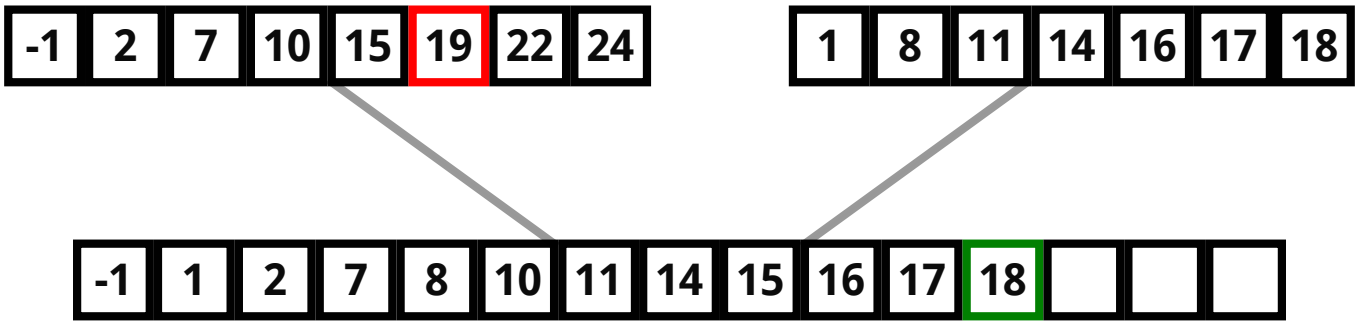
How to merge two sorted chunks?

Method 1:



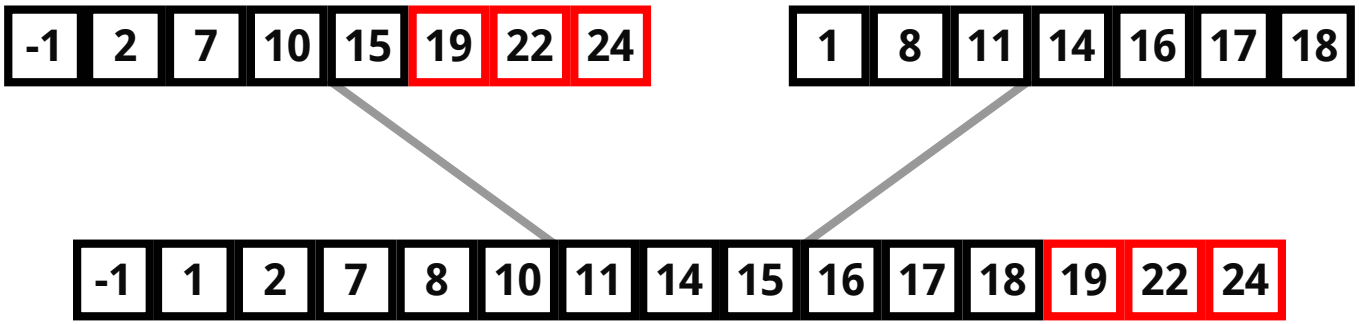
How to merge two sorted chunks?

Method 1:



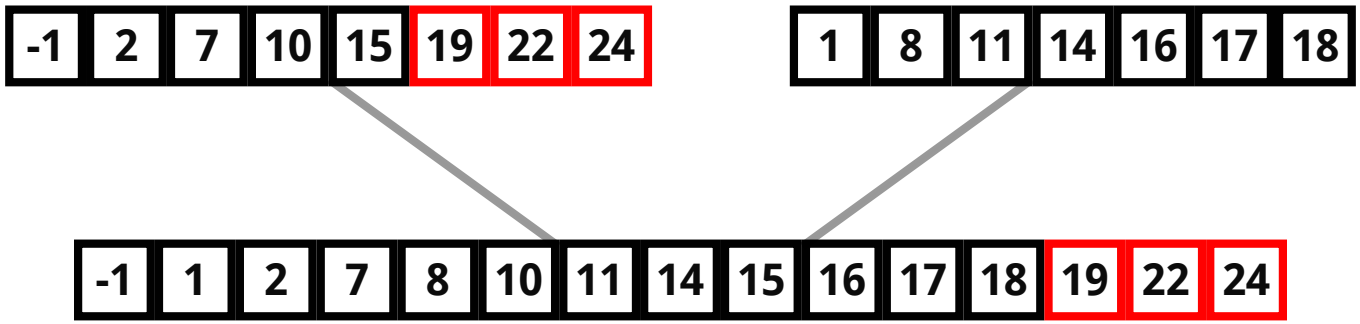
How to merge two sorted chunks?

Method 1:

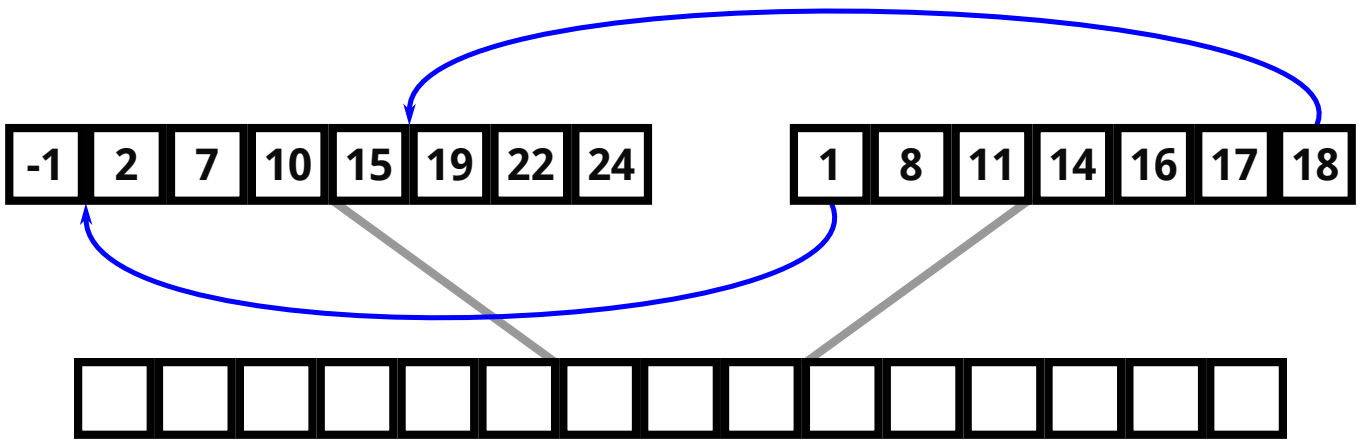


How to merge two sorted chunks?

Method 1:

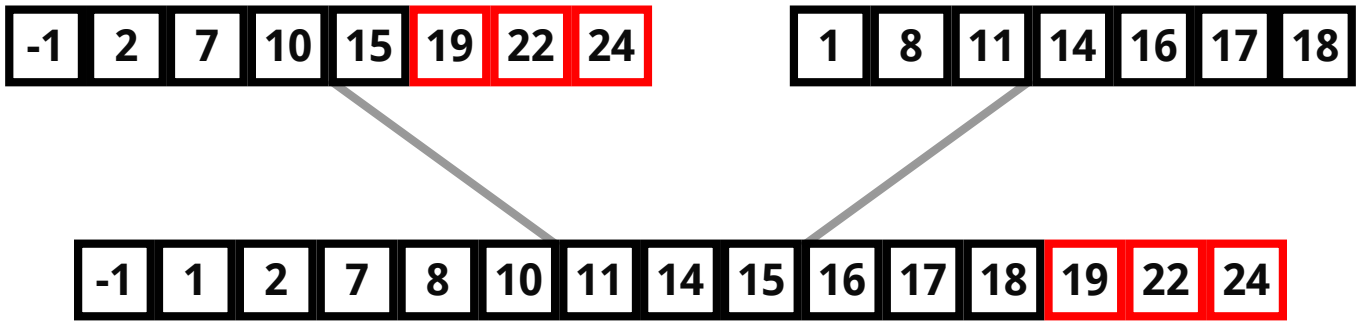


Method 2 (gallop mode):

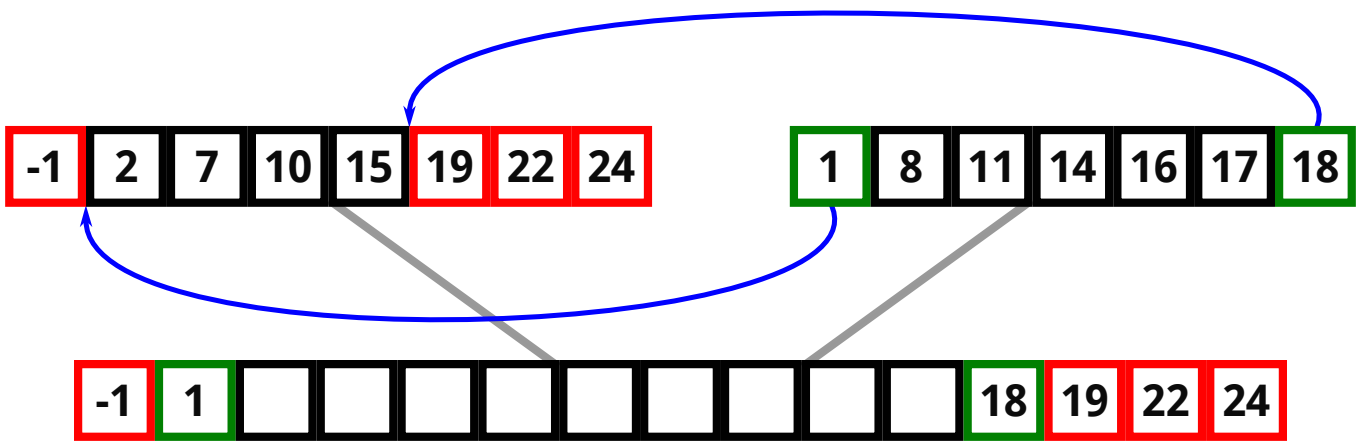


How to merge two sorted chunks?

Method 1:

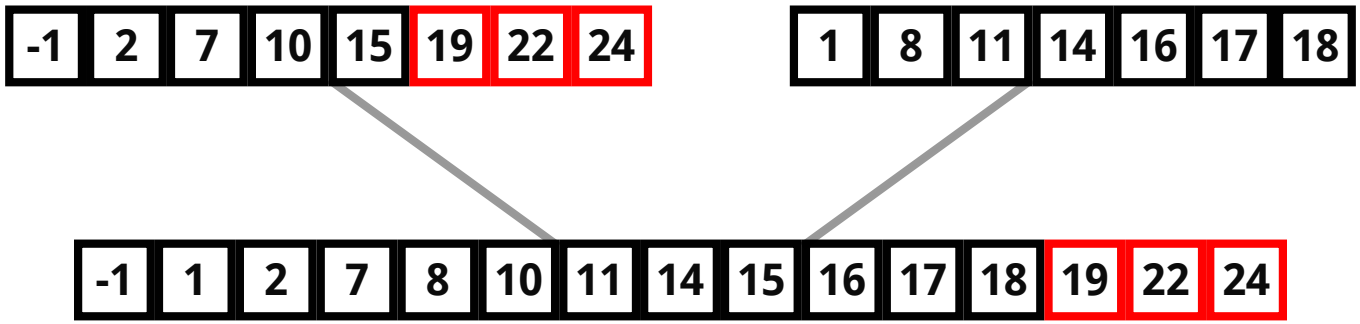


Method 2 (gallop mode):

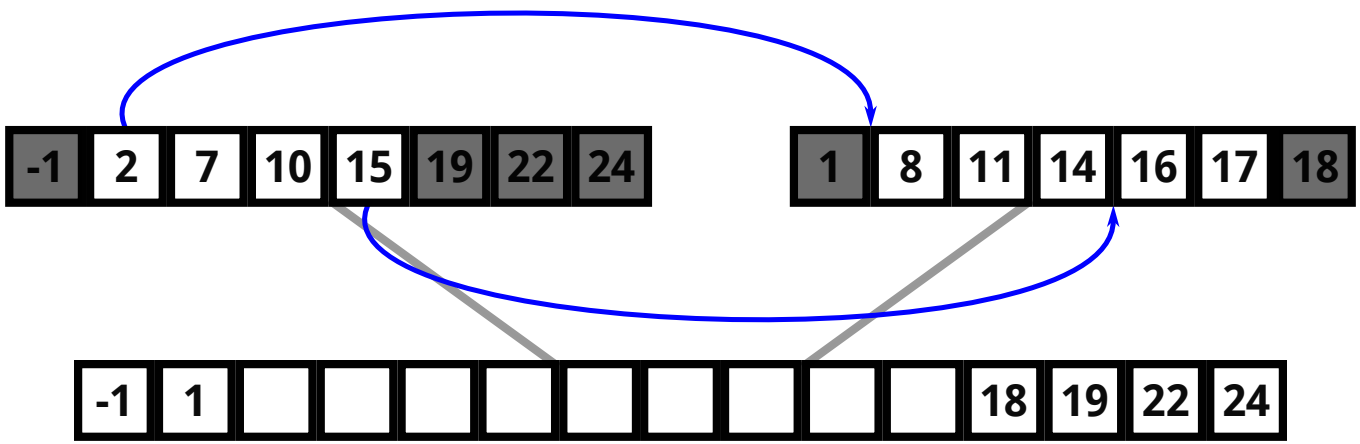


How to merge two sorted chunks?

Method 1:

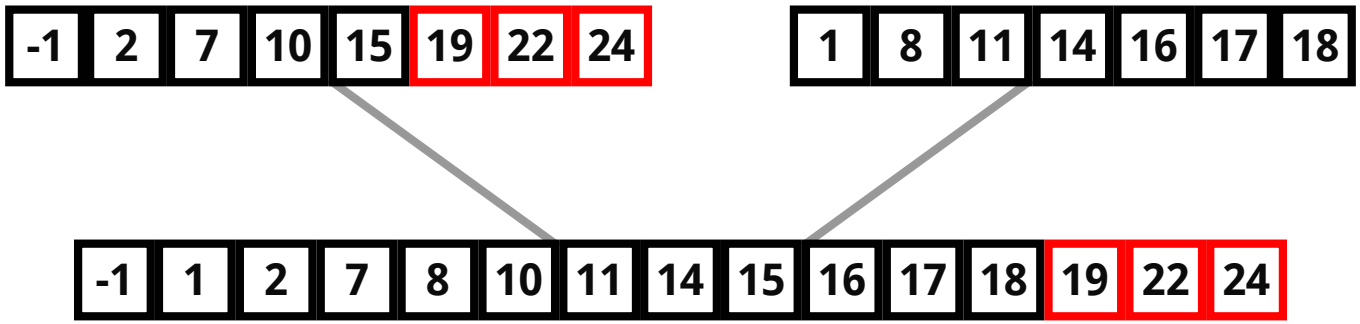


Method 2 (gallop mode):

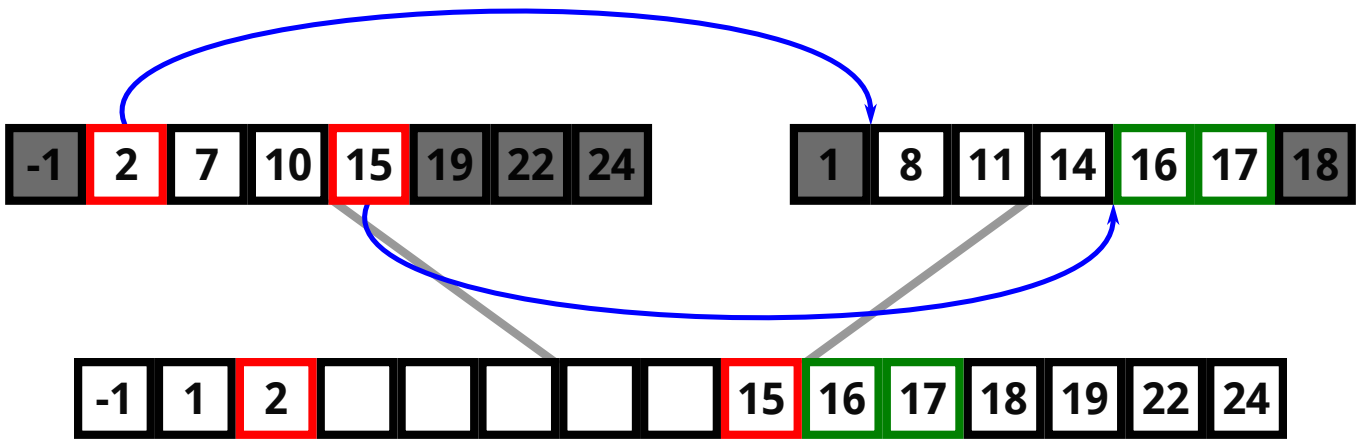


How to merge two sorted chunks?

Method 1:

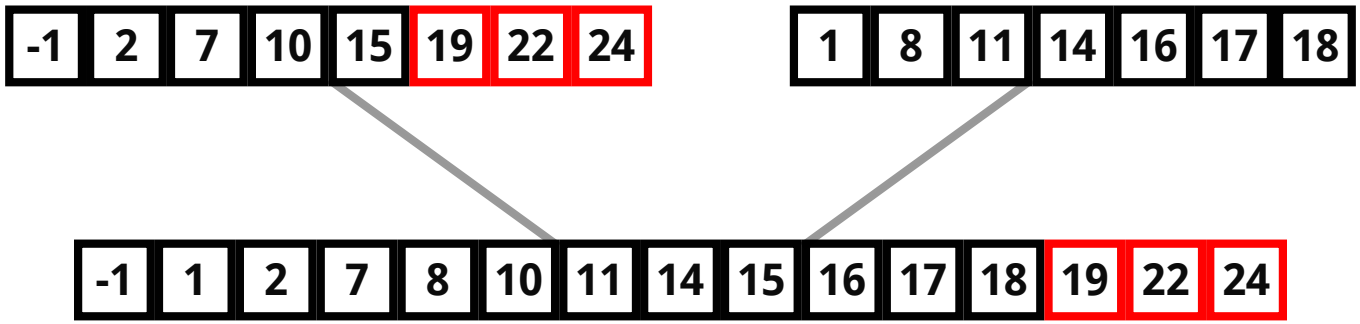


Method 2 (gallop mode):

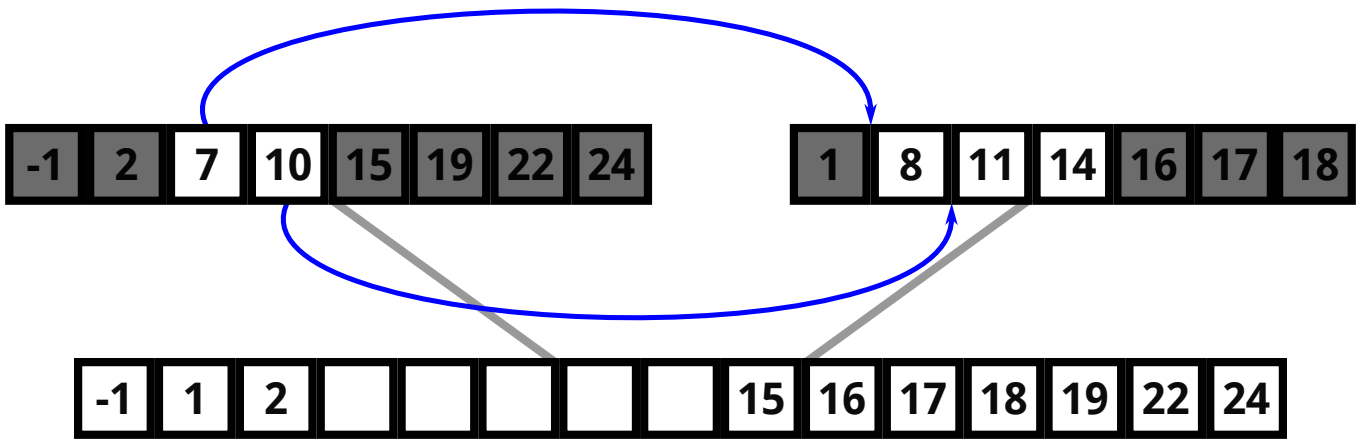


How to merge two sorted chunks?

Method 1:

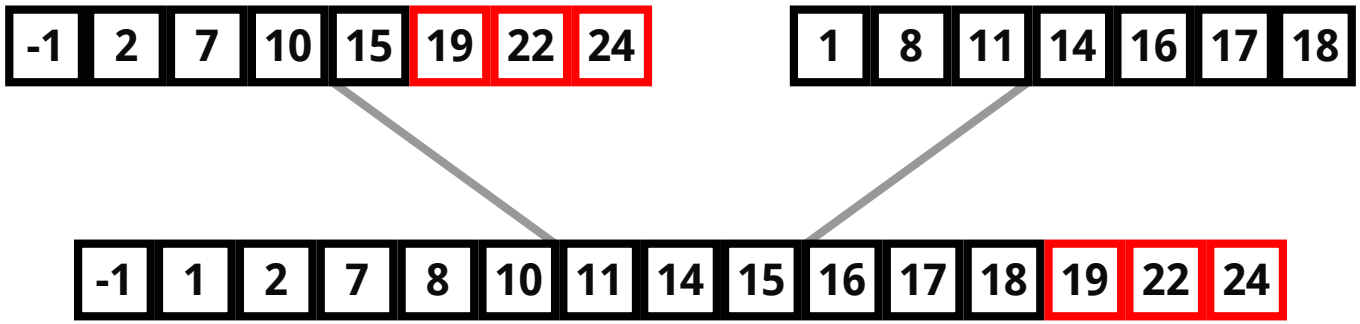


Method 2 (gallop mode):

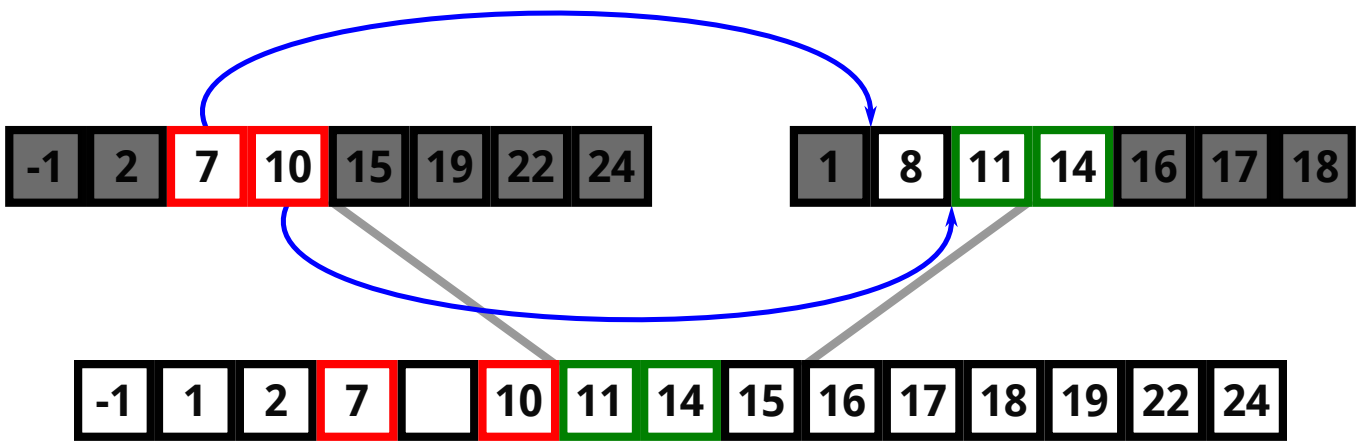


How to merge two sorted chunks?

Method 1:

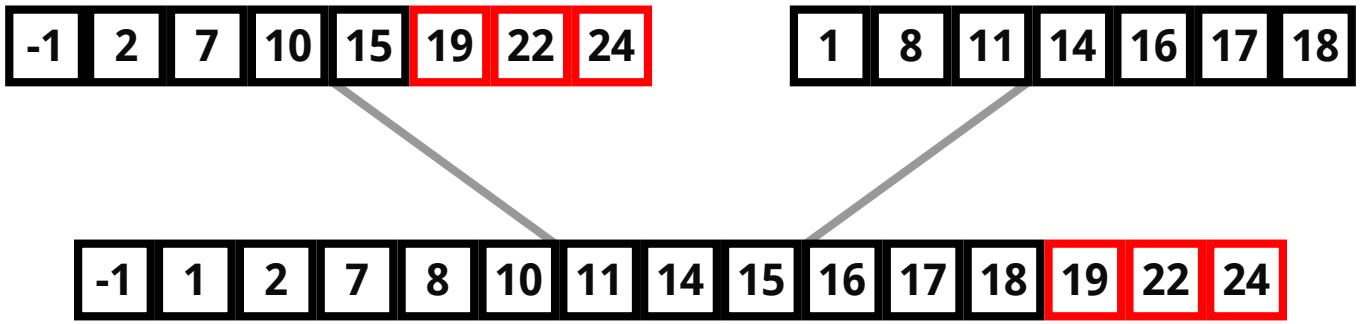


Method 2 (gallop mode):

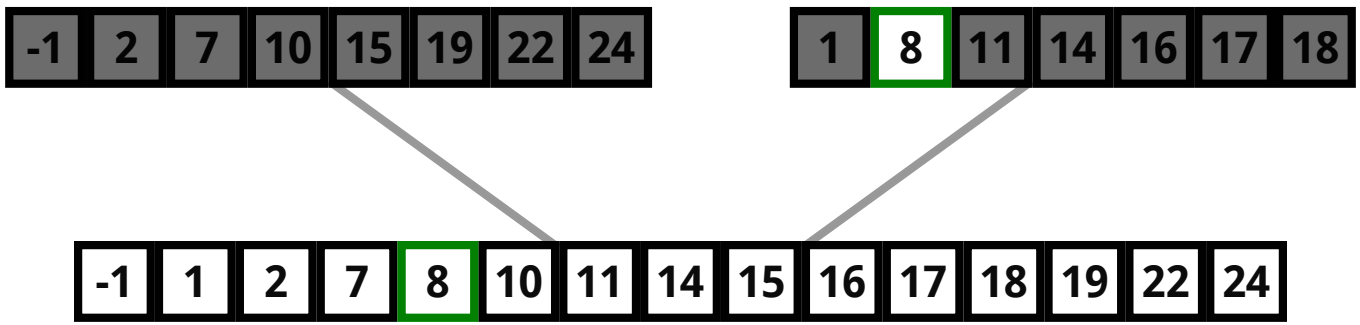


How to merge two sorted chunks?

Method 1:

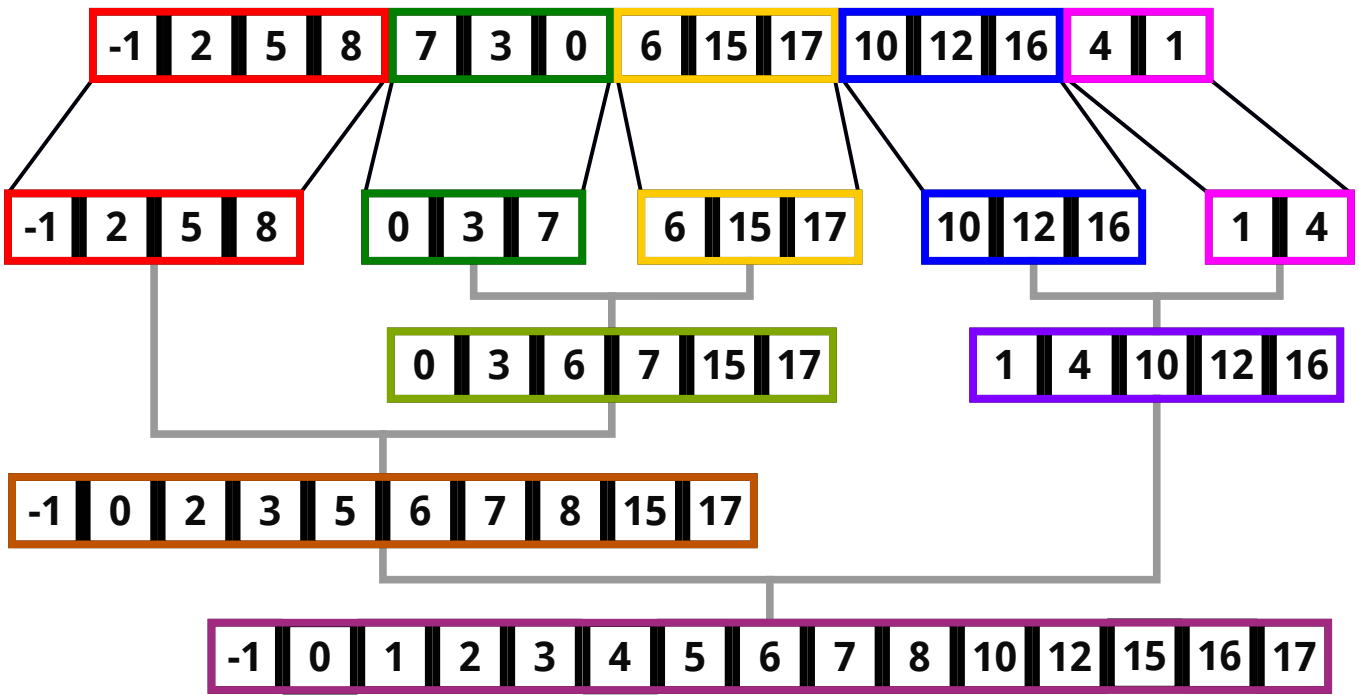


Method 2 (gallop mode):



General idea of Timsort

1. Divide the array into increasing or decreasing chunks (runs). Reverse decreasing chunks.
2. Merge the chunks using the methods in merge sort.



Optimizations in Timsort

- Timsort sets a minimum length of each chunk (usually 32 or 64). If a monotonic subsequence has elements less than the minimum length, extend it and sort the elements using the binary insertion sort.
- The monotonic chunks need to be merged in a specific order to guarantee that the merging can be completed within $O(\log n)$ levels.
- When merging chunks, Timsort adaptively turns the gallop mode on and off to achieve good efficiency.

1 Sorting algorithms
- Sort in Apache Spark
- Sort in CUDA-X

2 Joins
- Types of joins
- Join algorithms in Spark

3 Adaptive Query Execution (AQE)

4 Summary

General workflow

The general workflow of the sort algorithm used in CUDA-X is the same as that in Spark:

1. Use reservior sampling to select a few rows from each partition. Resampling is performed to select more rows if necessary.
   - Random numbers are generated in CPU.
   - Selected rows are concatenated in GPU.

General workflow

The general workflow of the sort algorithm used in CUDA-X is the same as that in Spark:

1. Use reservior sampling to select a few rows from each partition. Resampling is performed to select more rows if necessary.
   - Random numbers are generated in CPU.
   - Selected rows are concatenated in GPU.
2. Selected rows are moved to the host memory and collected by the driver.

General workflow

The general workflow of the sort algorithm used in CUDA-X is the same as that in Spark:

1. Use reservior sampling to select a few rows from each partition. Resampling is performed to select more rows if necessary.
   - Random numbers are generated in CPU.
   - Selected rows are concatenated in GPU.
2. Selected rows are moved to the host memory and collected by the driver.
3. The driver determines the range boundaries, which are then sent to the GPU memory of executors to determine the partitions of all the rows.

General workflow

The general workflow of the sort algorithm used in CUDA-X is the same as that in Spark:

1. Use reservior sampling to select a few rows from each partition. Resampling is performed to select more rows if necessary.
   - Random numbers are generated in CPU.
   - Selected rows are concatenated in GPU.
2. Selected rows are moved to the host memory and collected by the driver.
3. The driver determines the range boundaries, which are then sent to the GPU memory of executors to determine the partitions of all the rows.
4. Rows belonging to the same partition are moved to the same executor.

General workflow

The general workflow of the sort algorithm used in CUDA-X is the same as that in Spark:

1. Use reservior sampling to select a few rows from each partition. Resampling is performed to select more rows if necessary.
   - Random numbers are generated in CPU.
   - Selected rows are concatenated in GPU.
2. Selected rows are moved to the host memory and collected by the driver.
3. The driver determines the range boundaries, which are then sent to the GPU memory of executors to determine the partitions of all the rows.
4. Rows belonging to the same partition are moved to the same executor.
5. Each executor sorts the data locally.

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

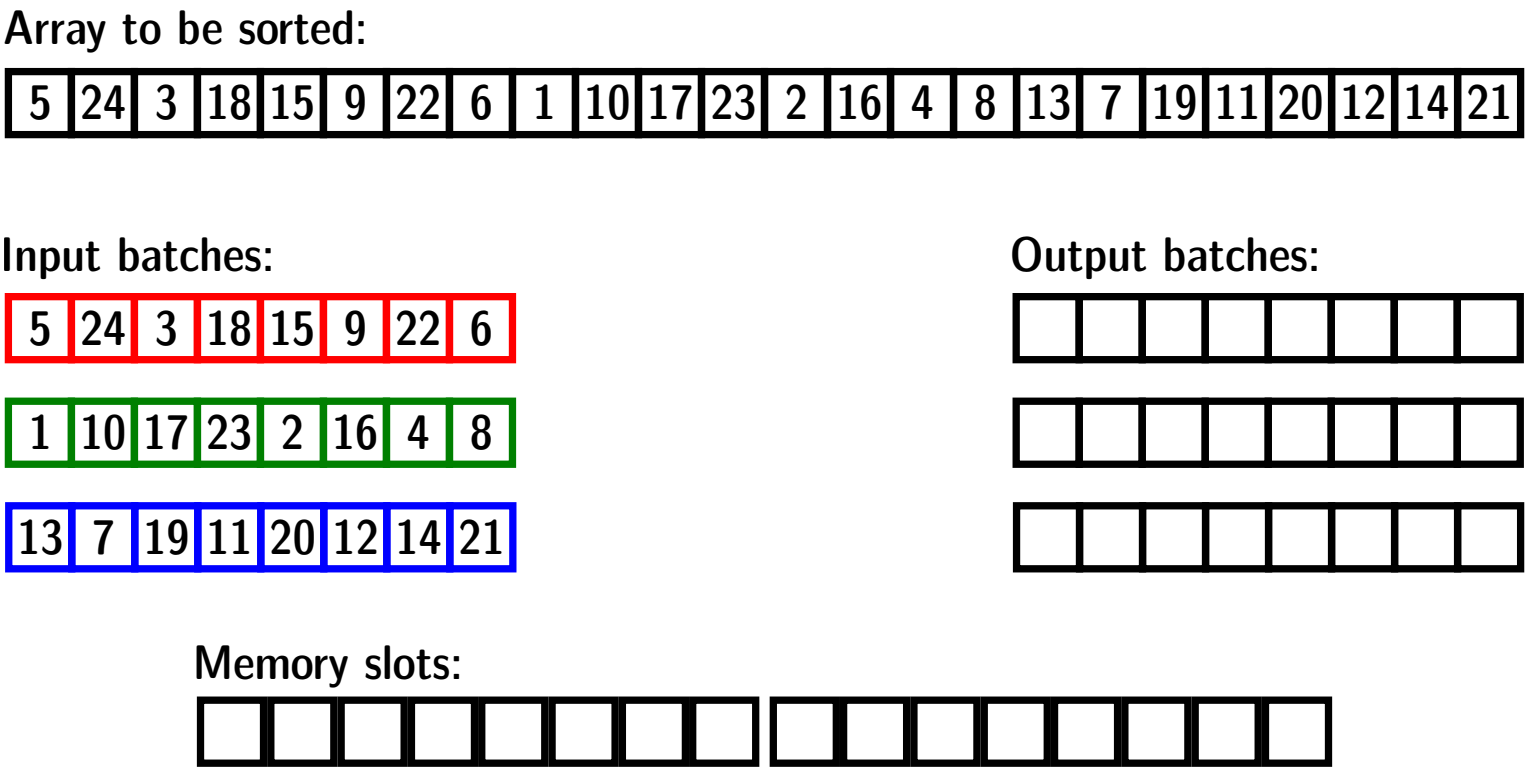
Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

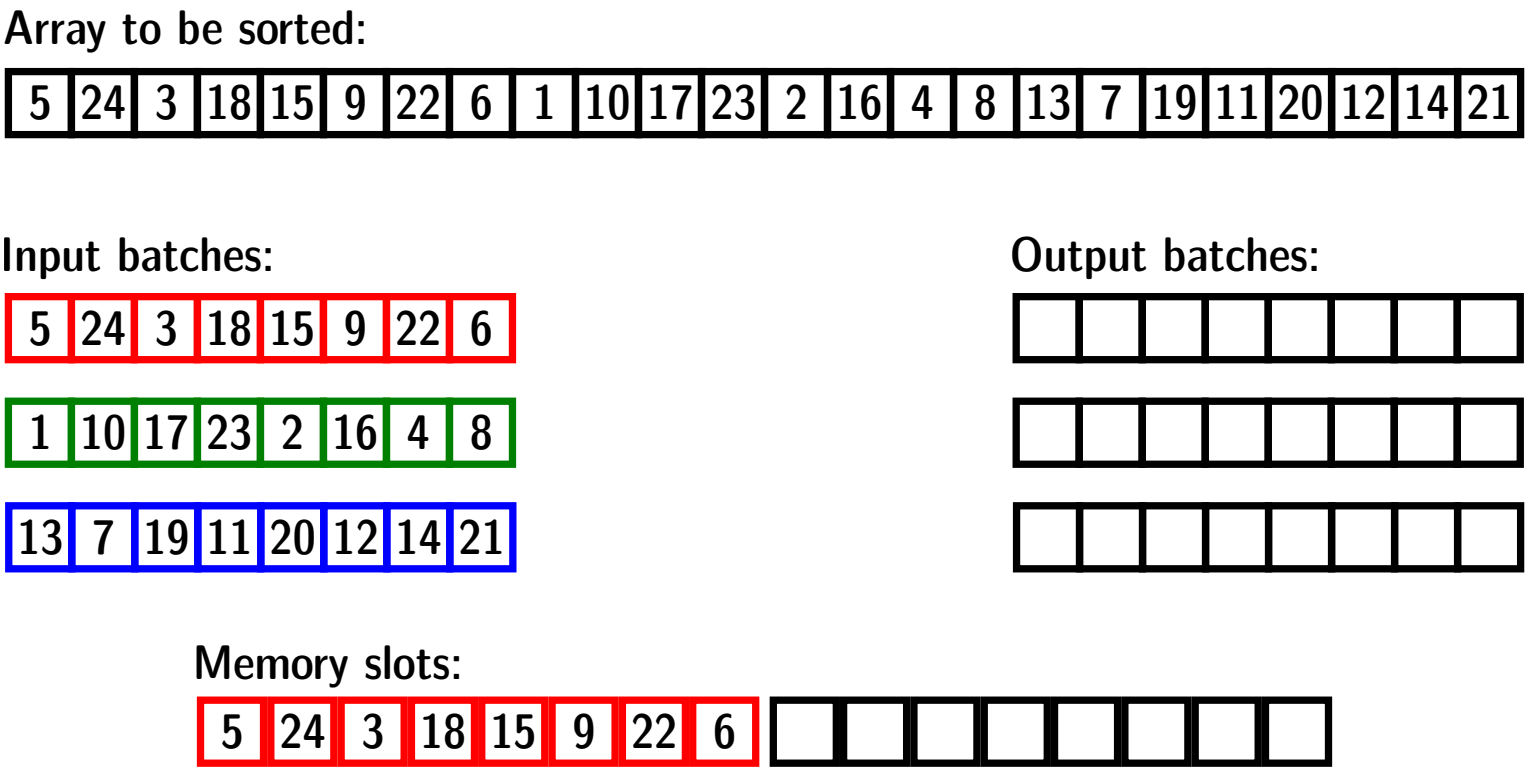
External merge sort:



Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

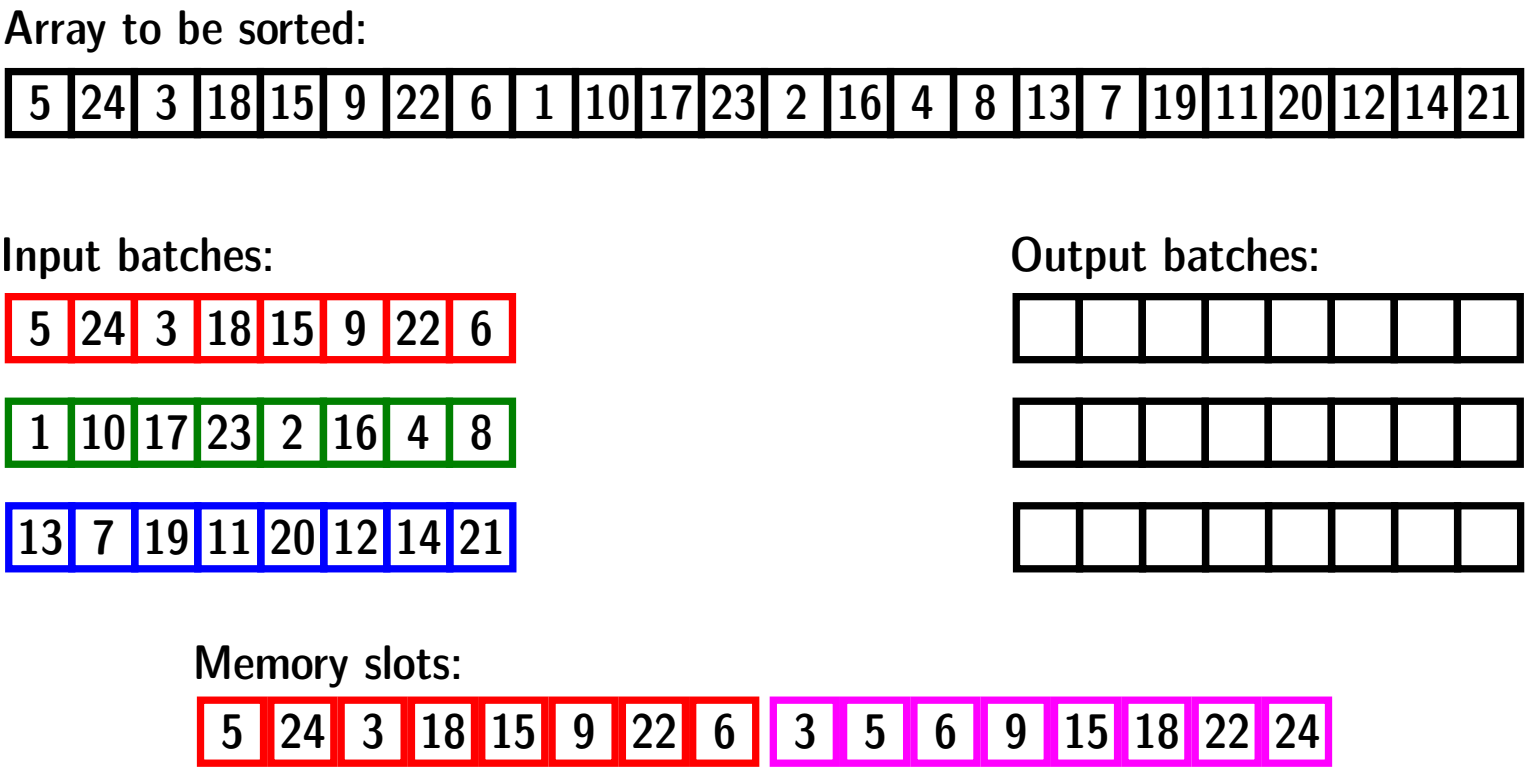
External merge sort:



Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:



Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	10	17	23	2	16	4	8

13	7	19	11	20	12	14	21

Output batches:

Memory slots:

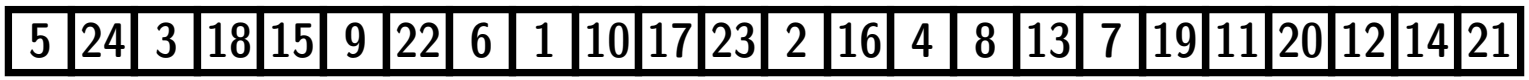
5	24	3	18	15	9	22	6	3	5	6	9	15	18	22	24

Out-of-core sort

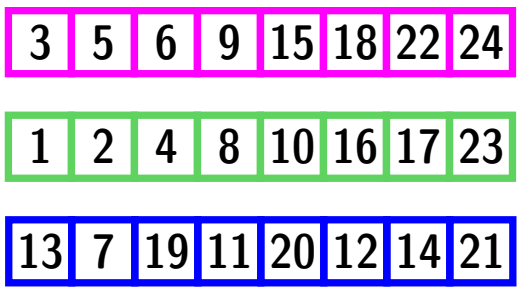
However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

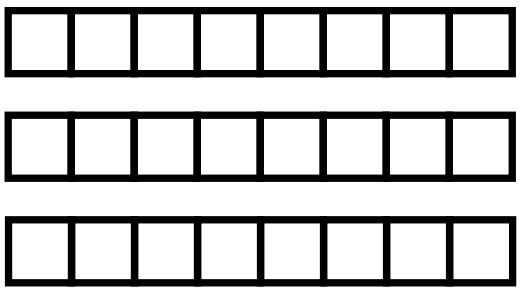
Array to be sorted:



Input batches:



Output batches:



Memory slots:



Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

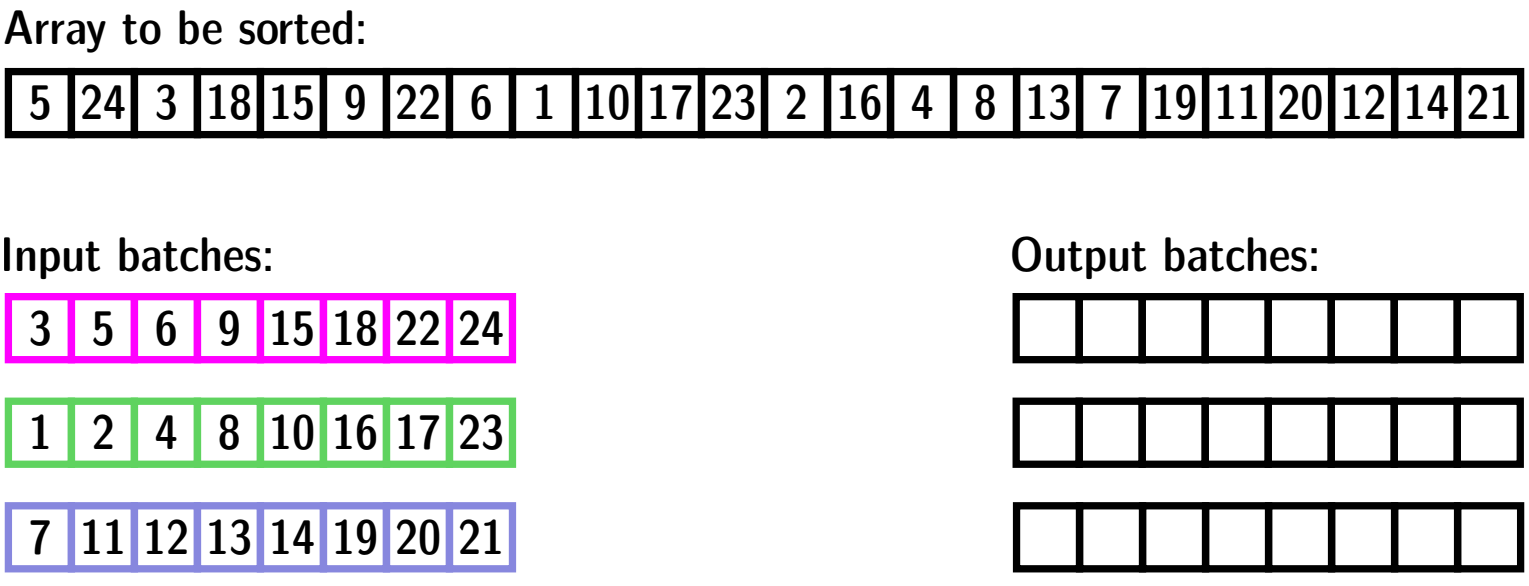
Memory slots:

13	7	19	11	20	12	14	21	7	11	12	13	14	19	20	21

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:



Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

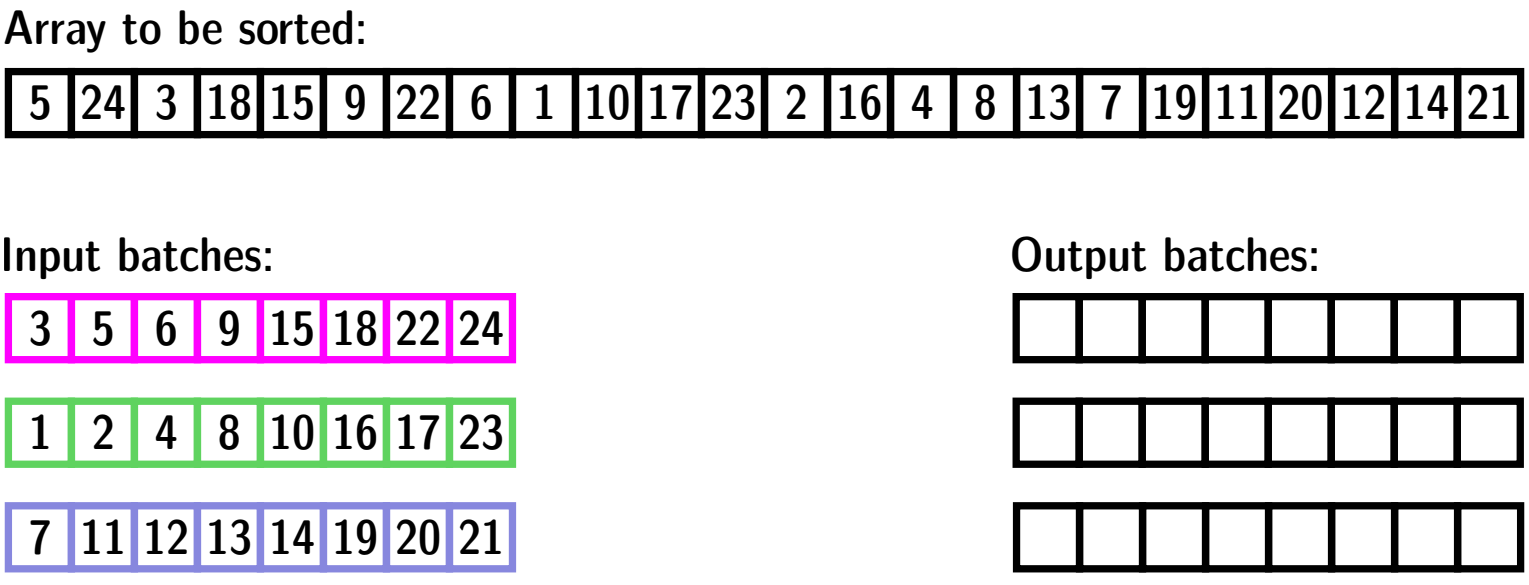
Memory slots:

3	5	6	1	2	4	7	11	1	2	3	4				

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:



Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

Memory slots:

3	5	6	8	10	16	7	11	1	2	3	4	5	6		

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

Memory slots:

9	15	18	8	10	16	7	11	1	2	3	4	5	6		

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

Memory slots:

9	15	18	8	10	16	7	11	1	2	3	4	5	6	7	8

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

1	2	3	4	5	6	7	8

Memory slots:

9	15	18	8	10	16	7	11								

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

1	2	3	4	5	6	7	8

Memory slots:

9	15	18	8	10	16	7	11	9	10	11					

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

1	2	3	4	5	6	7	8

Memory slots:

9	15	18	8	10	16	12	13	9	10	11					

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

1	2	3	4	5	6	7	8

Memory slots:

9	15	18	8	10	16	12	13	9	10	11	12	13			

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

1	2	3	4	5	6	7	8

Memory slots:

9	15	18	8	10	16	14	19	9	10	11	12	13			

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

1	2	3	4	5	6	7	8

Memory slots:

9	15	18	8	10	16	14	19	9	10	11	12	13	14	15	16

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

1	2	3	4	5	6	7	8

9	10	11	12	13	14	15	16

Memory slots:

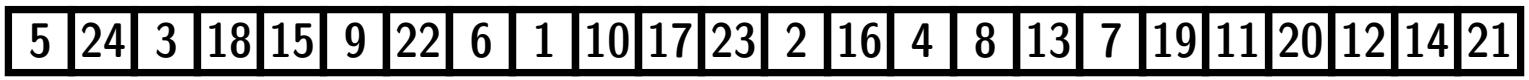
9	15	18	8	10	16	14	19								

Out-of-core sort

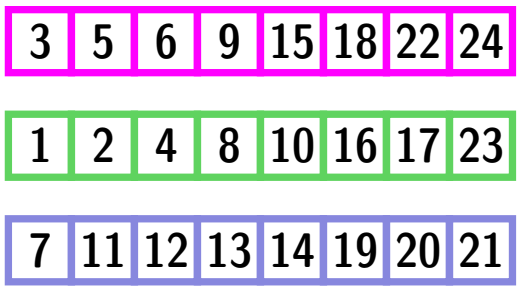
However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

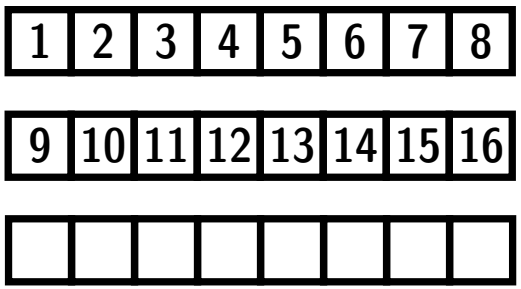
Array to be sorted:



Input batches:



Output batches:



Memory slots:



Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

1	2	3	4	5	6	7	8

9	10	11	12	13	14	15	16

Memory slots:

9	15	18	17	23		14	19	17	18						

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

1	2	3	4	5	6	7	8

9	10	11	12	13	14	15	16

Memory slots:

22	24		17	23		14	19	17	18						

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

1	2	3	4	5	6	7	8

9	10	11	12	13	14	15	16

Memory slots:

22	24		17	23		14	19	17	18	19					

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

1	2	3	4	5	6	7	8

9	10	11	12	13	14	15	16

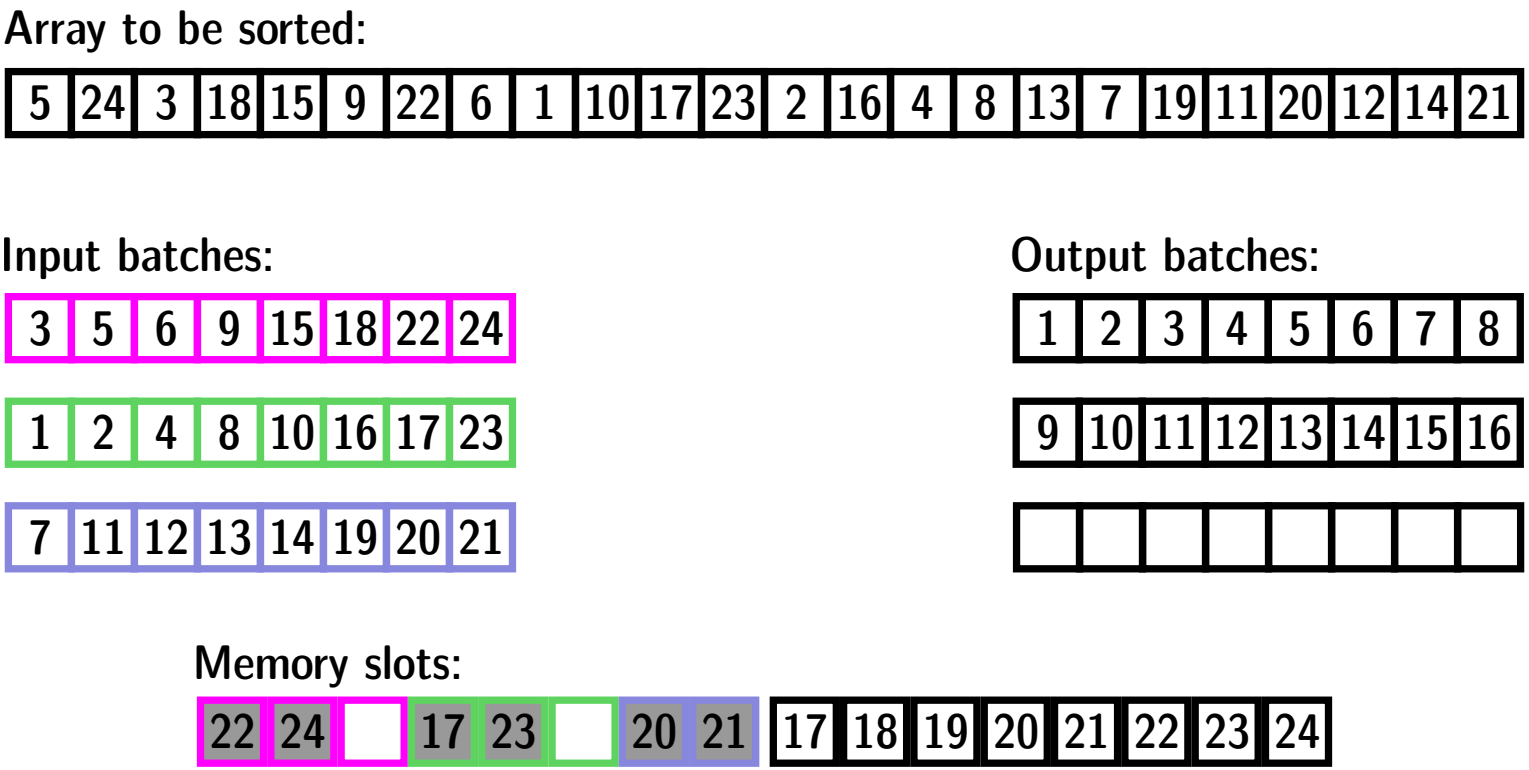
Memory slots:

22	24		17	23		20	21	17	18	19					

Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:



Out-of-core sort

However, each partition may include several batches, which means the GPU memory may not be enough to load all the data to perform the local sort.

External merge sort:

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

Input batches:

3	5	6	9	15	18	22	24

1	2	4	8	10	16	17	23

7	11	12	13	14	19	20	21

Output batches:

1	2	3	4	5	6	7	8

9	10	11	12	13	14	15	16

17	18	19	20	21	22	23	24

Memory slots:

22	24		17	23		20	21								

General procedure of external merge sort

1. Split the array into batches that fit into the memory.
2. Sort each batch in memory and spill the sorted result.
3. For each batch, load part of the data into memory.
4. Perform a multi-way merge until one of the loaded batches runs out of data or there is enough data to produce an output batch.
5. If one batch runs out of data, load the next part into memory. If the sorted data is large enough to produce an output batch, spill the merged result.
6. Return to 4 if more data are to be sorted.

General procedure of external merge sort

1. Split the array into batches that fit into the memory.
2. Sort each batch in memory and spill the sorted result.
3. For each batch, load part of the data into memory.
4. Perform a multi-way merge until one of the loaded batches runs out of data or there is enough data to produce an output batch.
5. If one batch runs out of data, load the next part into memory. If the sorted data is large enough to produce an output batch, spill the merged result.
6. Return to 4 if more data are to be sorted.

However, this method is not suitable for GPUs.

> *The main difference is that we cannot stream the data when doing a merge sort. — by developer of CUDA-X*

GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

GPU Sort

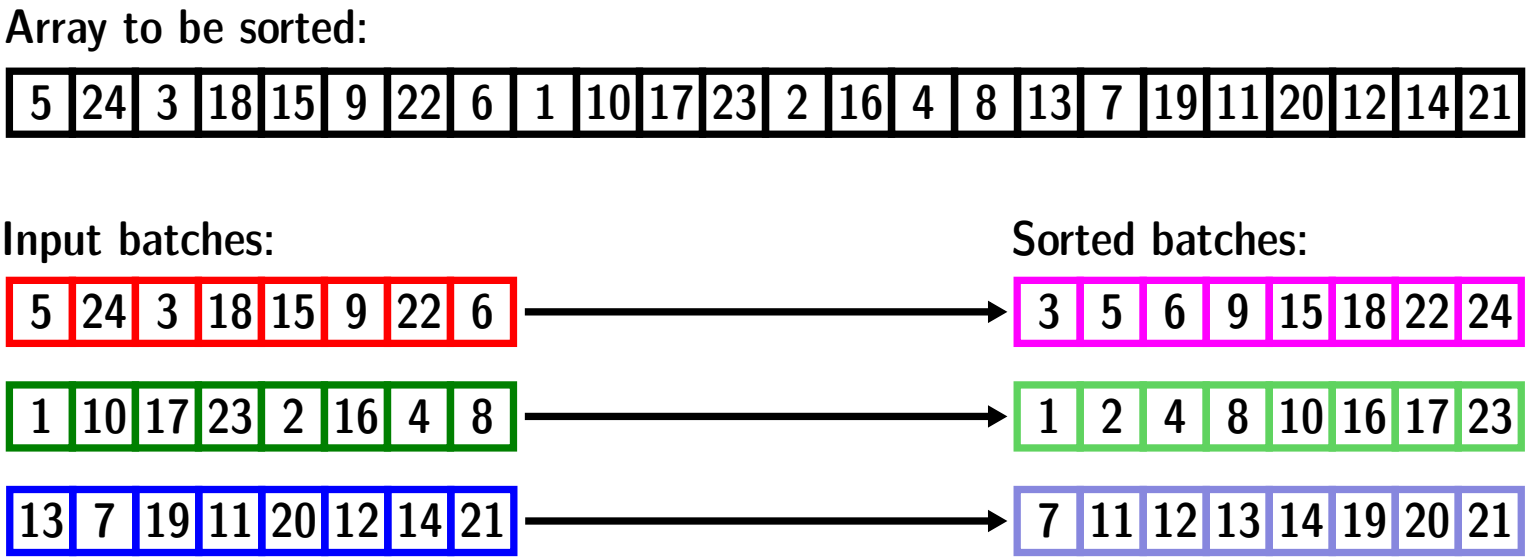
So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Array to be sorted:

5	24	3	18	15	9	22	6	1	10	17	23	2	16	4	8	13	7	19	11	20	12	14	21

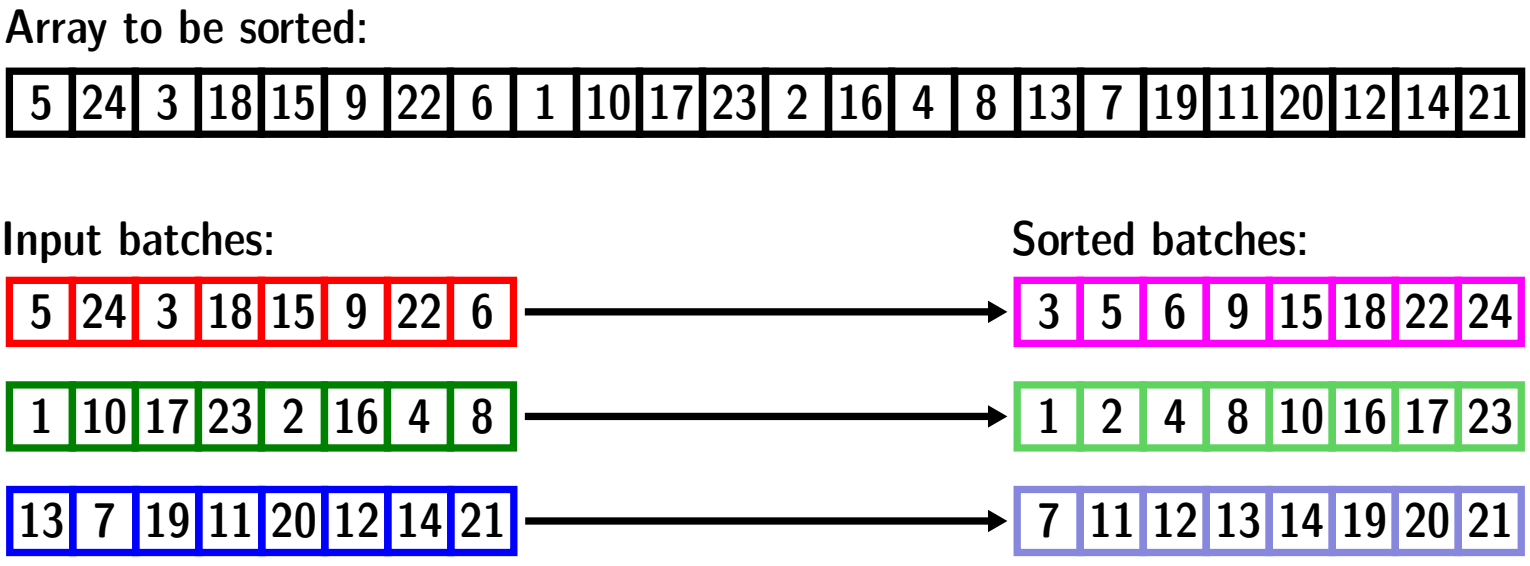
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X



GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X



GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:

1	2	4
3	5	6
7	11	12
8	10	16
9	15	18
13	14	19
17	23	
20	21	
22	24	

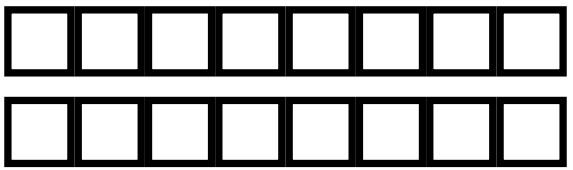
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:

1	2	4
3	5	6
7	11	12
8	10	16
9	15	18
13	14	19
17	23	
20	21	
22	24	

Memory slots:



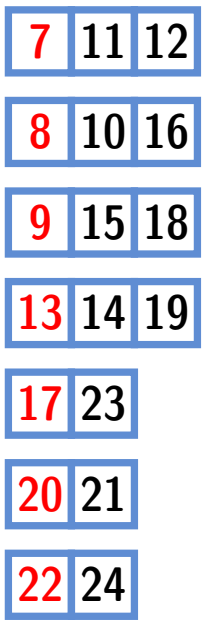
Generate an output batch if there are at least 6 globally sorted elements.

Output batches:

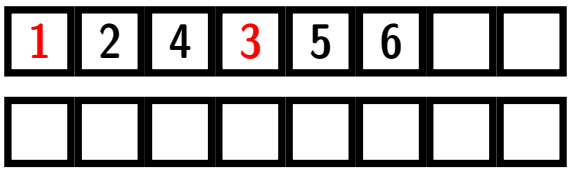
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:



Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

Output batches:

GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:

7	11	12
8	10	16
9	15	18
13	14	19
17	23	
20	21	
22	24	

Memory slots:

1	2	4	3	5	6		
1	2	3	4	5	6		

Generate an output batch if there are at least 6 globally sorted elements.

Output batches:

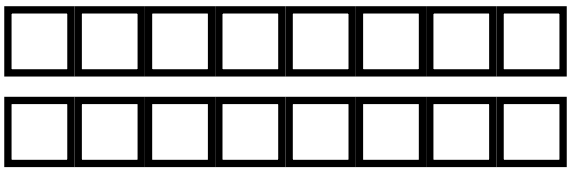
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:

7	11	12
8	10	16
9	15	18
13	14	19
17	23	
20	21	
22	24	

Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

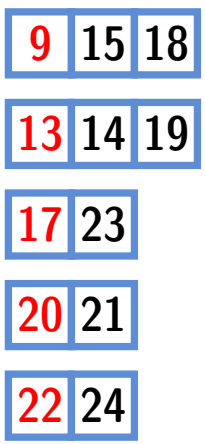
Output batches:

1	2	3	4	5	6		

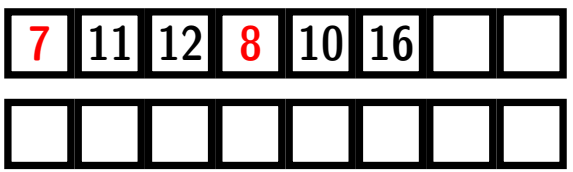
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:



Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

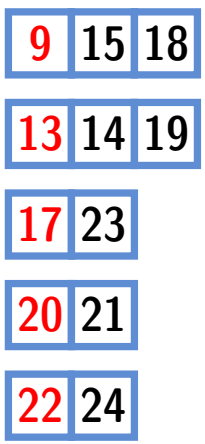
Output batches:



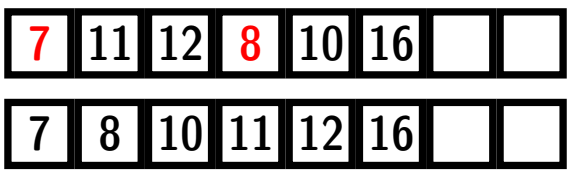
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:



Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

Output batches:



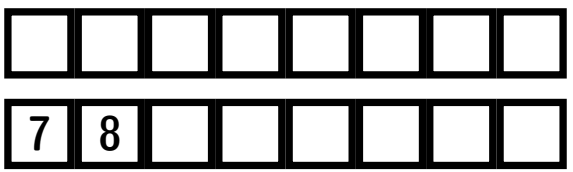
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:



Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

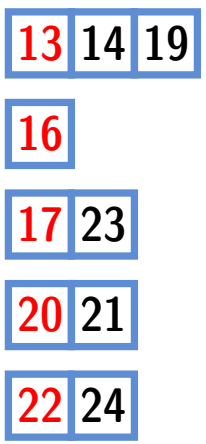
Output batches:



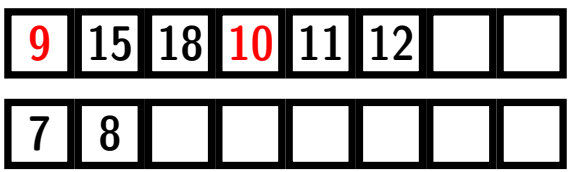
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:



Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

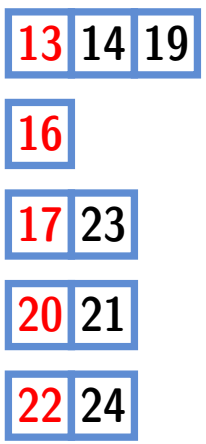
Output batches:



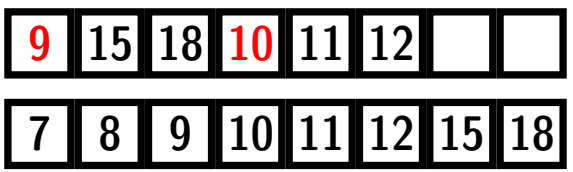
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:



Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

Output batches:



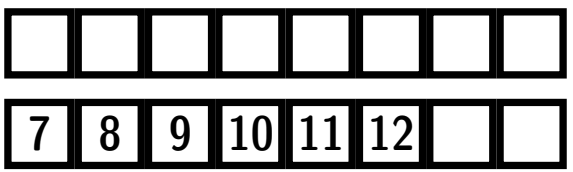
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:



Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

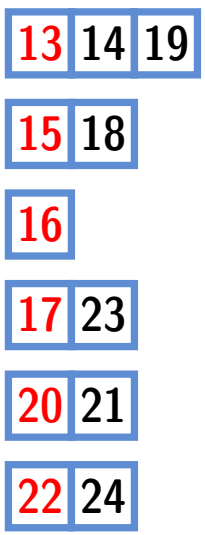
Output batches:



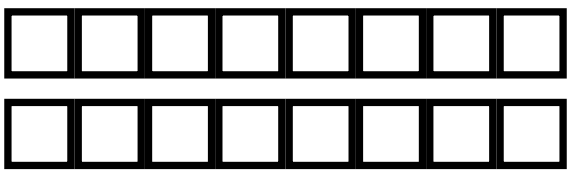
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:

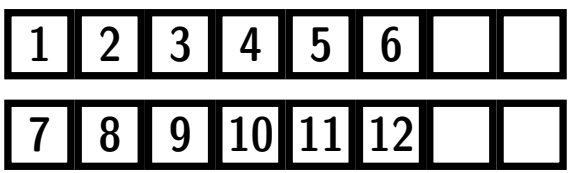


Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

Output batches:



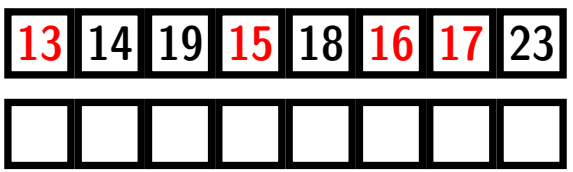
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:

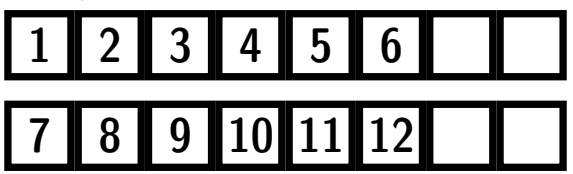


Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

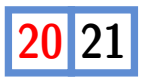
Output batches:



GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

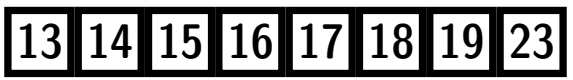
Place all the batches into a PriorityQueue:





Memory slots:





Generate an output batch if there are at least 6 globally sorted elements.

Output batches:





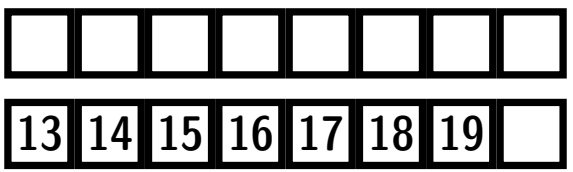
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:

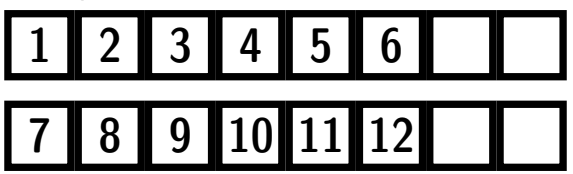


Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

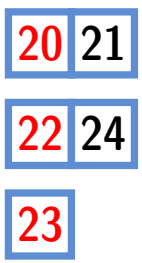
Output batches:



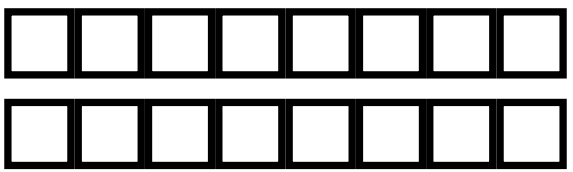
GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

Place all the batches into a PriorityQueue:

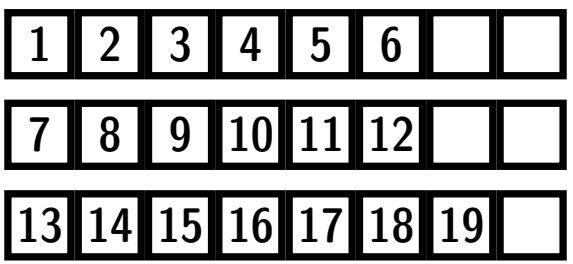


Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

Output batches:

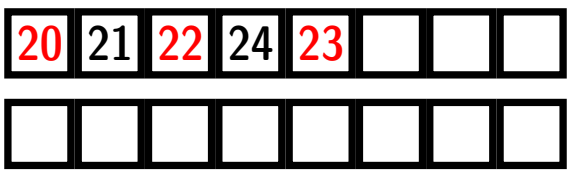


GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

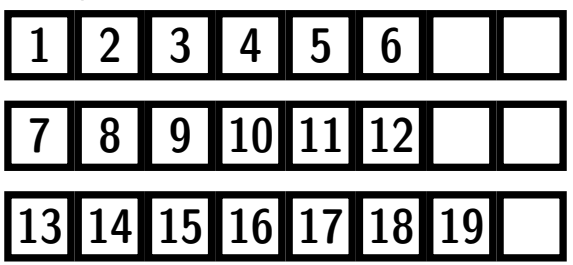
Place all the batches into a PriorityQueue:

Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

Output batches:

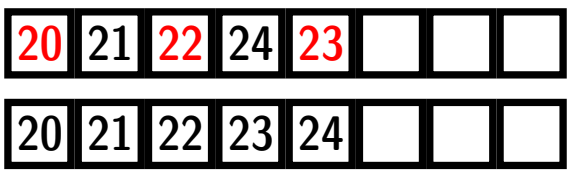


GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

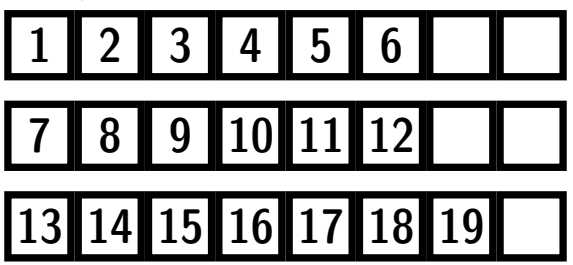
Place all the batches into a PriorityQueue:

Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

Output batches:

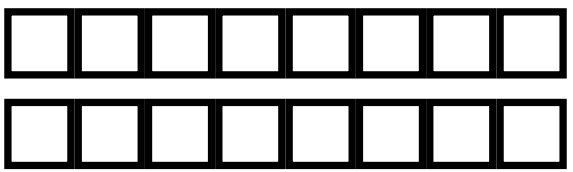


GPU Sort

So, we instead divide the data into batches that are small enough that we can do a merge sort on N batches and still fit the output within the target batch size. — by developer of CUDA-X

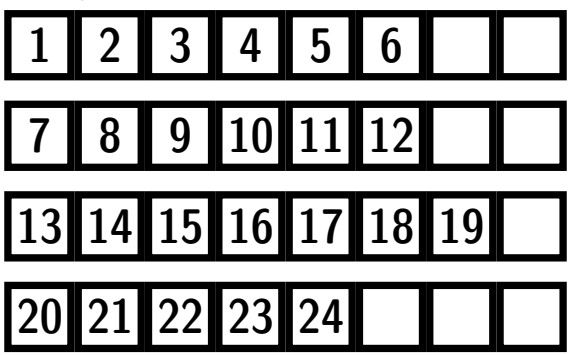
Place all the batches into a PriorityQueue:

Memory slots:



Generate an output batch if there are at least 6 globally sorted elements.

Output batches:



Summary of the out-of-core sort in CUDA-X

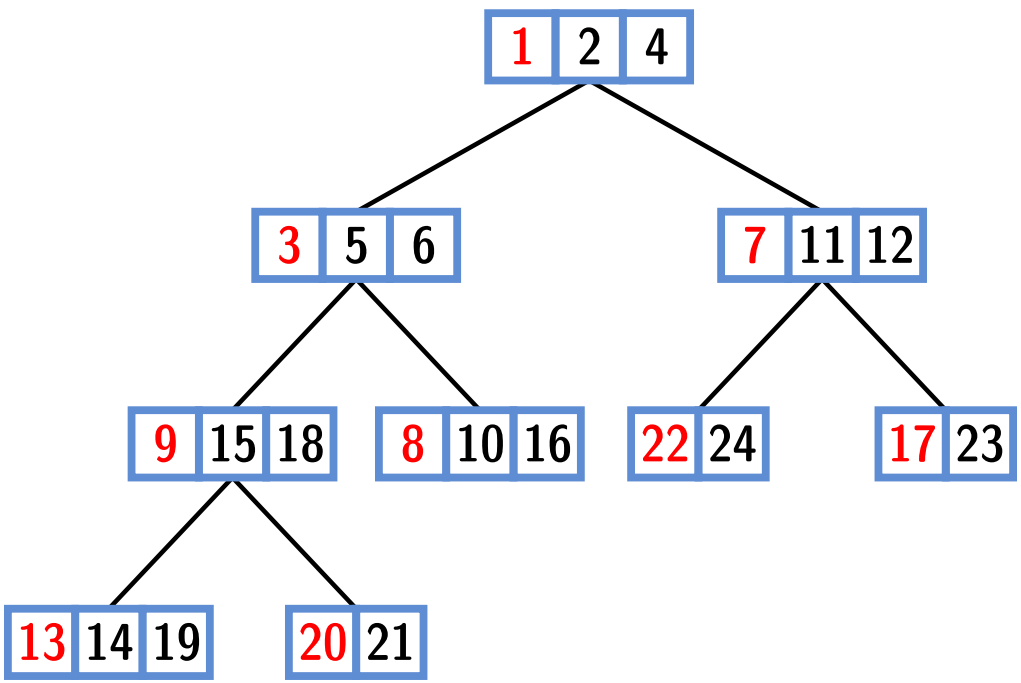
1. Sort each batch in GPU memory and spill the sorted result.
2. Split the sorted batches into smaller batches and put them into a priority queue. The priority is determined by the smallest key in each batch.
3. Load the first few batches from the priority queue into GPU memory, and merge them.
4. Determine the global sorted portion of the merged result. Then split the rest part into small batches and place them back to the priority queue.
5. Spill the the global sorted portion if it is large enough to produce an output batch.
6. Return to 3 if more data are to be sorted.

Priority queue

A priority queue is usually implemented as a binary heap, a complete binary tree where every node has higher priority than its children.

Priority queue

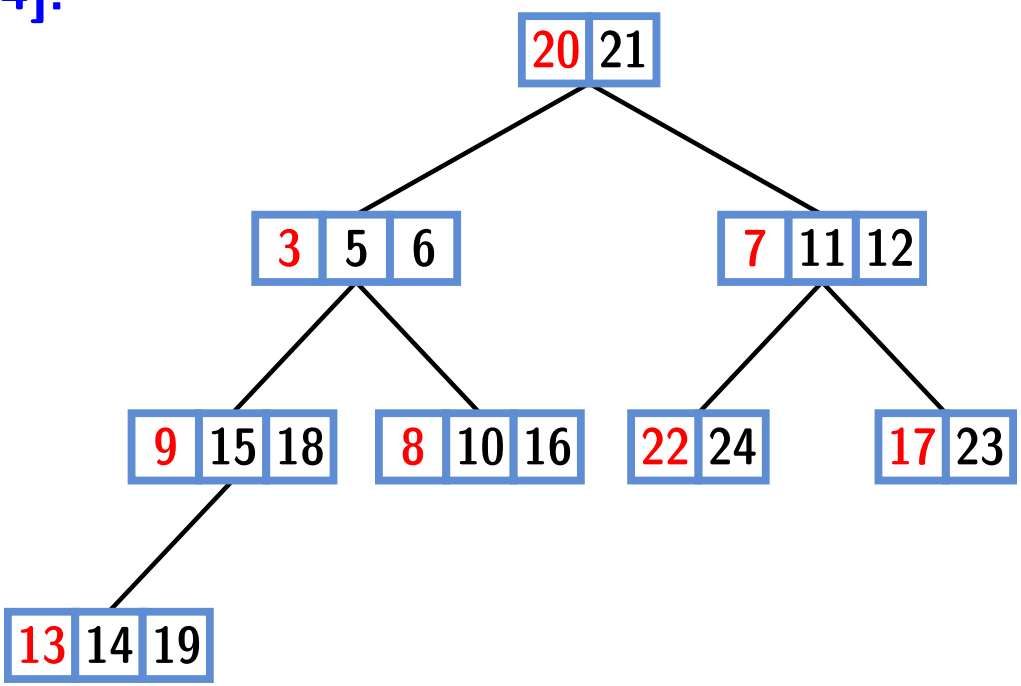
A priority queue is usually implemented as a binary heap, a complete binary tree where every node has higher priority than its children.



Priority queue

A priority queue is usually implemented as a binary heap, a complete binary tree where every node has higher priority than its children.

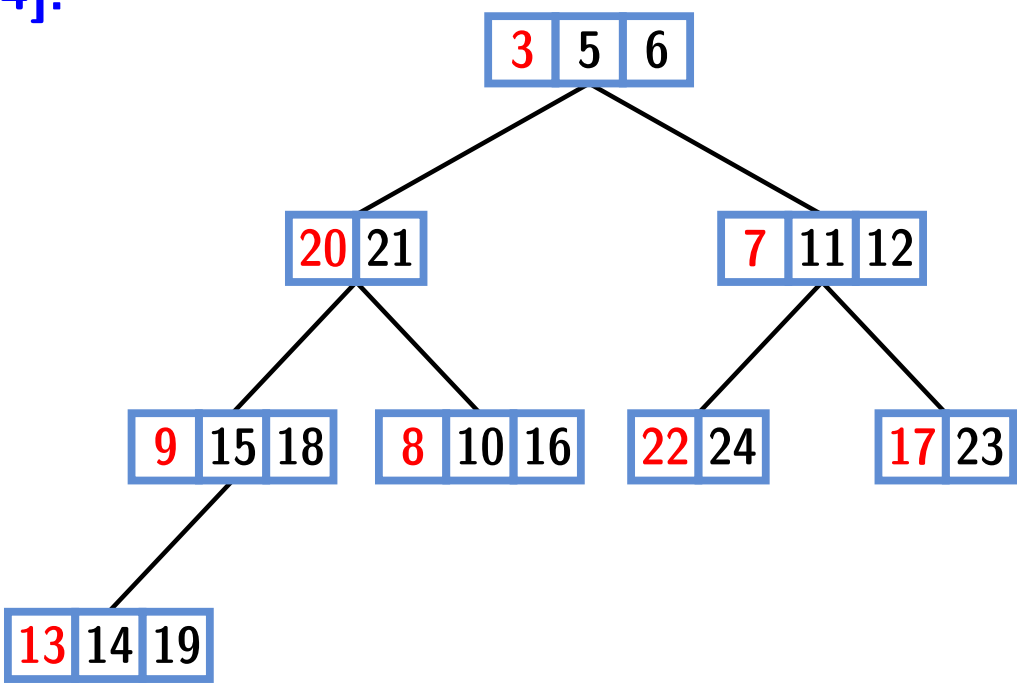
Remove [1,2,4]:



Priority queue

A priority queue is usually implemented as a binary heap, a complete binary tree where every node has higher priority than its children.

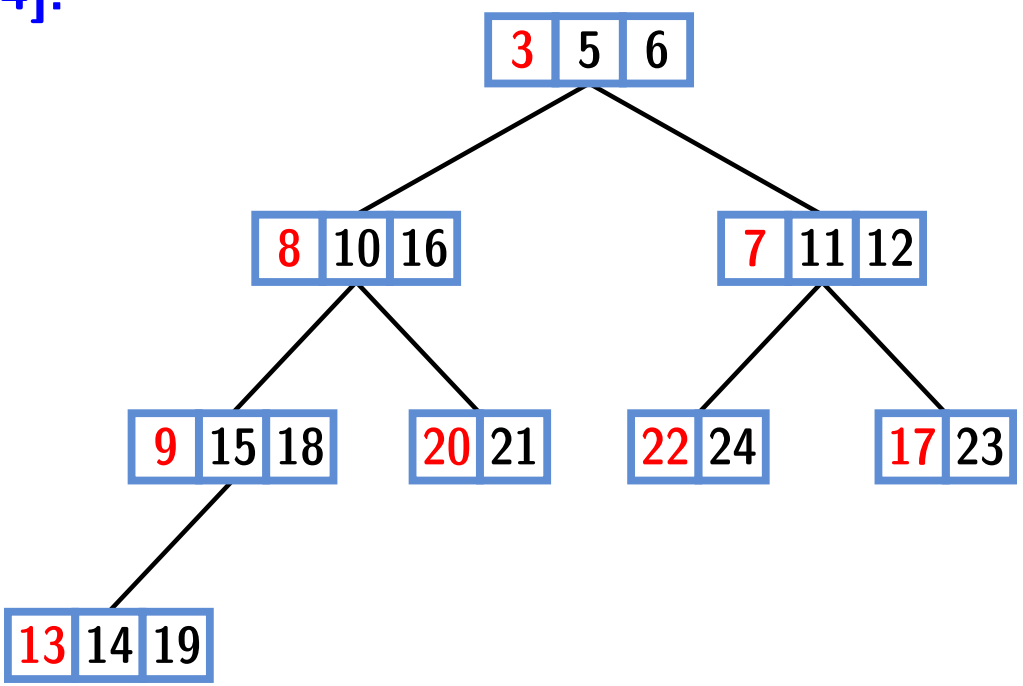
Remove [1,2,4]:



Priority queue

A priority queue is usually implemented as a binary heap, a complete binary tree where every node has higher priority than its children.

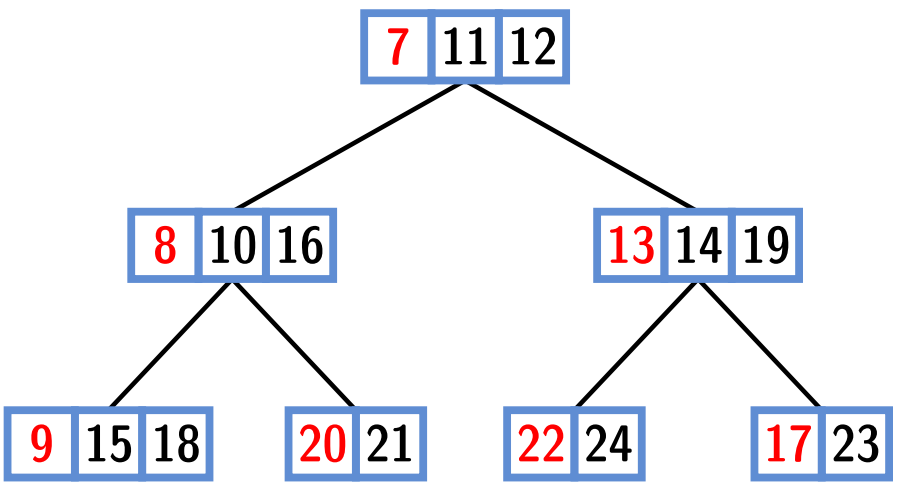
Remove [1,2,4]:



Priority queue

A priority queue is usually implemented as a binary heap, a complete binary tree where every node has higher priority than its children.

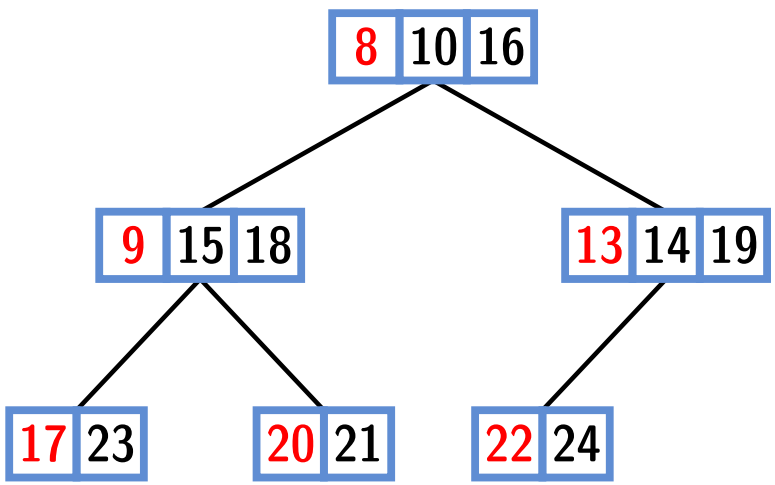
Remove [3,5,6]:



Priority queue

A priority queue is usually implemented as a binary heap, a complete binary tree where every node has higher priority than its children.

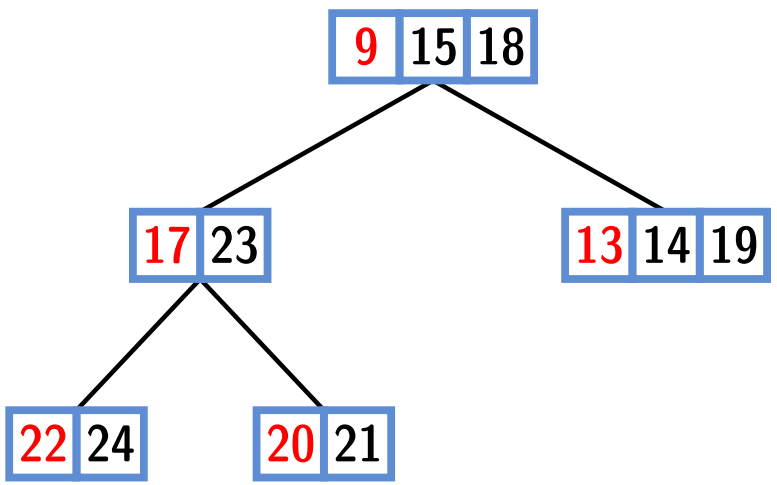
Remove [7,11,12]:



Priority queue

A priority queue is usually implemented as a binary heap, a complete binary tree where every node has higher priority than its children.

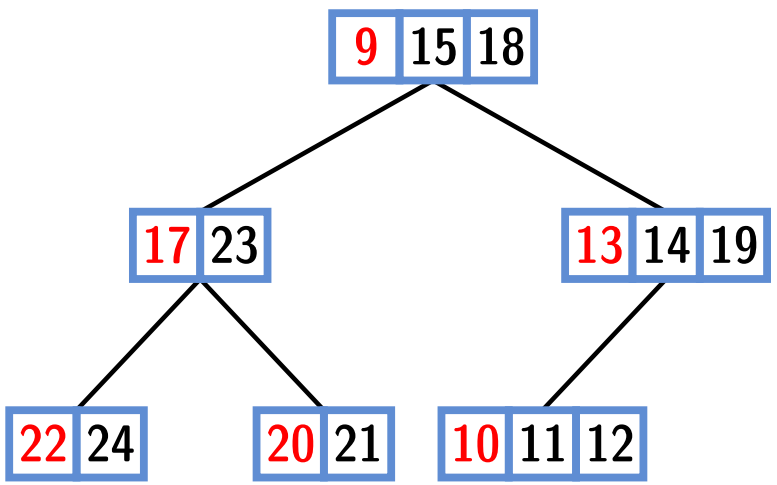
Remove [8,10,16]:



Priority queue

A priority queue is usually implemented as a binary heap, a complete binary tree where every node has higher priority than its children.

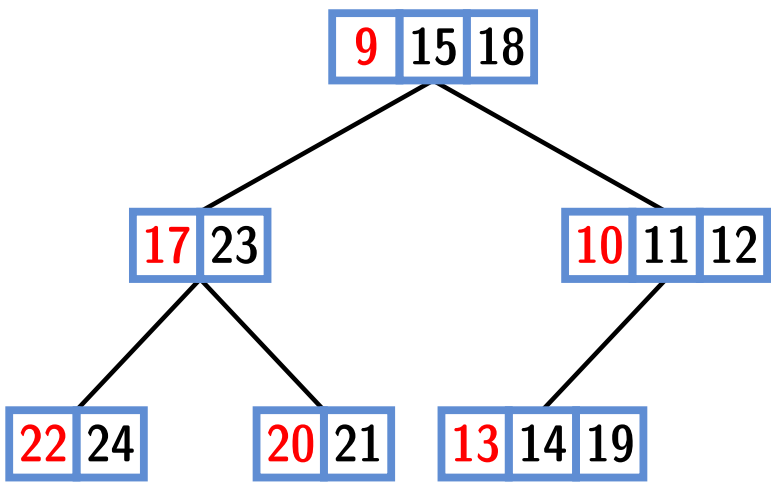
Insert [10,11,12]:



Priority queue

A priority queue is usually implemented as a binary heap, a complete binary tree where every node has higher priority than its children.

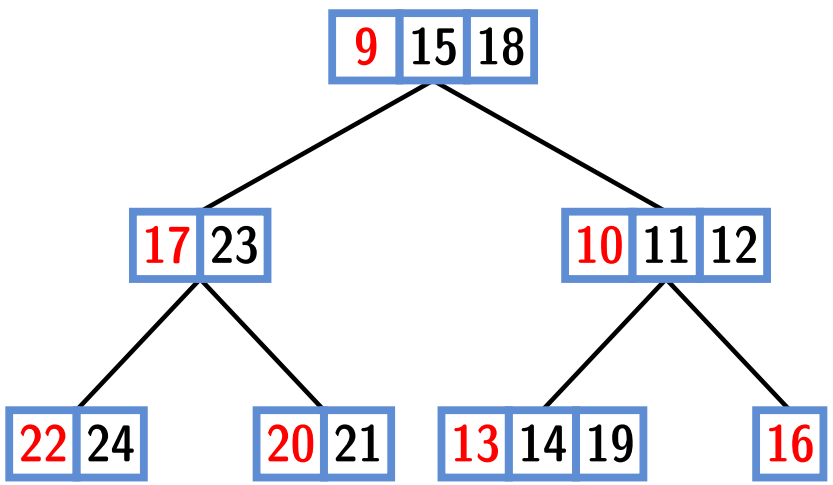
Insert [10,11,12]:



Priority queue

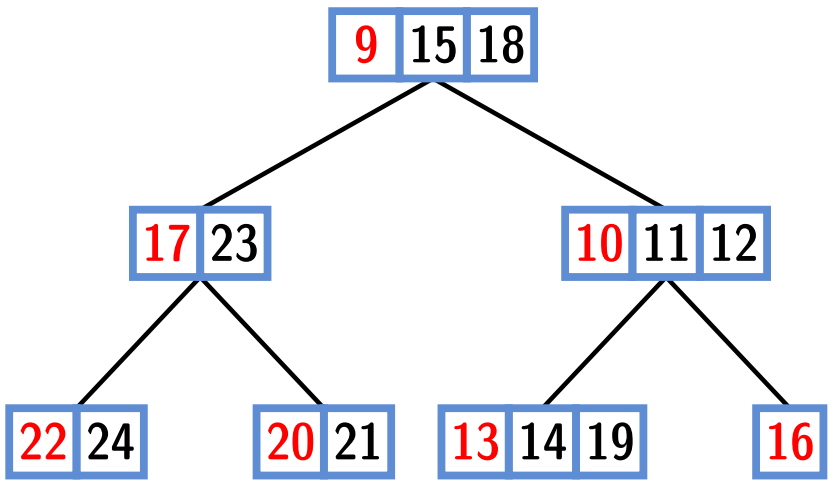
A priority queue is usually implemented as a binary heap, a complete binary tree where every node has higher priority than its children.

Insert [16]:



Priority queue

A priority queue is usually implemented as a binary heap, a complete binary tree where every node has higher priority than its children.



- Insert: add the new element at the end, and "bubble up" to restore the heap property.
- Remove: replace the root with the last element, and "bubble down" to restore the heap property.

1 Sorting algorithms

2 Joins

3 Adaptive Query Execution (AQE)

4 Summary

1 Sorting algorithms
- Sort in Apache Spark
- Sort in CUDA-X

2 Joins
- Types of joins
- Join algorithms in Spark

3 Adaptive Query Execution (AQE)

4 Summary

Example

Consider the dataframes `arrInfo` and `carrierName` created as follows:

```
>>> df = spark.read.csv("flights.csv", inferSchema=True, header=True)
>>> arrInfo = df.select("YEAR", "MONTH", "DAY",
...                     "AIRLINE", df.colRegex("`.*ARRIVAL.*`"))
>>> carrier_name = [
...     ("UA", "United Air Lines Inc."),
...     ("AA", "American Airlines Inc."),
...     ("US", "US Airways Inc."),
...     ("F9", "Frontier Airlines Inc."),
...     ("B6", "JetBlue Airways"),
...     ("OO", "Skywest Airlines Inc."),
...     ("AS", "Alaska Airlines Inc."),
...     ("NK", "Spirit Air Lines"),
...     ("WN", "Southwest Airlines Co."),
...     ("DL", "Delta Air Lines Inc."),
...     ("EV", "Atlantic Southeast Airlines"),
...     ("HA", "Hawaiian Airlines Inc."),
...     ("MQ", "American Eagle Airlines Inc."),
...     ("VX", "Virgin America")
... ]
>>> carrierName = spark.createDataFrame(carrier_name, ["CODE", "AIRLINE"])
```

How can we join the two dataframes and create a new dataframe that shows the name of the carrier for each flight?

Example

- **Join the two dataframes:**

```
>>> newDf = arrInfo.join(carrierName, arrInfo.AIRLINE == carrierName.CODE)
>>> newDf.show()
```

We can remove duplicated columns and rename the newly added column as follows:

```
>>> newDf = newDf.select("*", carrierName.AIRLINE.alias("COMPANY")).\
...     drop(col("CODE"), carrierName.AIRLINE)
```

Example

- **Join the two dataframes:**

```
>>> newDf = arrInfo.join(carrierName, arrInfo.AIRLINE == carrierName.CODE)
>>> newDf.show()
```

We can remove duplicated columns and rename the newly added column as follows:

```
>>> newDf = newDf.select("*", carrierName.AIRLINE.alias("COMPANY")).\
...     drop(col("CODE"), carrierName.AIRLINE)
```

- What happens if a code does not exist in carrierName?

```
>>> newArrInfo = spark.createDataFrame([(2015, 1, 1, "SQ", 1255, 1249, -6)],
...                                    arrInfo.schema).union(arrInfo)
>>> newArrInfo.join(carrierName, newArrInfo.AIRLINE == carrierName.CODE).\
...     filter(newArrInfo.AIRLINE == "SQ").show()
```

The result is empty!

Example

- **Join the two dataframes:**

```
>>> newDf = arrInfo.join(carrierName, arrInfo.AIRLINE == carrierName.CODE)
>>> newDf.show()
```

We can remove duplicated columns and rename the newly added column as follows:

```
>>> newDf = newDf.select("*", carrierName.AIRLINE.alias("COMPANY")).\
...     drop(col("CODE"), carrierName.AIRLINE)
```

- What happens if a code does not exist in `carrierName`?

```
>>> newArrInfo = spark.createDataFrame([(2015, 1, 1, "SQ", 1255, 1249, -6)],
...                                    arrInfo.schema).union(arrInfo)
>>> newArrInfo.join(carrierName, newArrInfo.AIRLINE == carrierName.CODE).\
...     filter(newArrInfo.AIRLINE == "SQ").show()
```

The result is empty!

- To get a `NULL` for flights whose carriers cannot be found in `carrierName`, we have to specify the join type:

```
>>> newArrInfo.join(carrierName, newArrInfo.AIRLINE == carrierName.CODE,
...     "left_outer").filter(newArrInfo.AIRLINE == "SQ").show()
```

Four major types of joins

X1	Y1
1	A1
2	B1
3	C1
1	D1

X2	Y2
1	A2
2	B2
4	C2

```
>>> df1 = spark.createDataFrame([(1,"A1"),(1,"B1"),(2,"C1"),(3,"D1")], ["X1","Y1"])
>>> df2 = spark.createDataFrame([(1,"A2"),(2,"B2"),(4,"C2")], ["X2","Y2"])
```

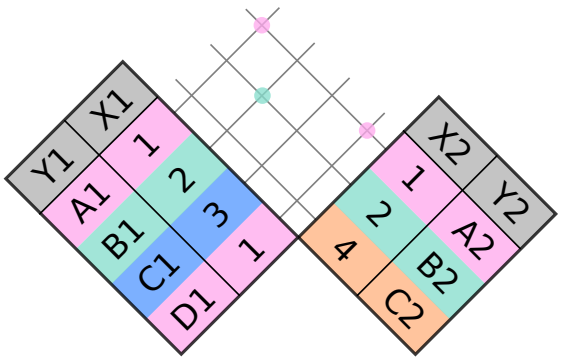
Four major types of joins

X1	Y1
1	A1
2	B1
3	C1
1	D1

X2	Y2
1	A2
2	B2
4	C2

```
>>> df1 = spark.createDataFrame([(1,"A1"),(1,"B1"),(2,"C1"),(3,"D1")], ["X1","Y1"])
>>> df2 = spark.createDataFrame([(1,"A2"),(2,"B2"),(4,"C2")], ["X2","Y2"])
```

Inner join:



X1	Y1	X2	Y2
1	A1	1	A2
1	D1	1	A2
2	B1	2	B2

```
>>> df1.join(df2, df1.X1 == df2.X2, "inner").show()
```

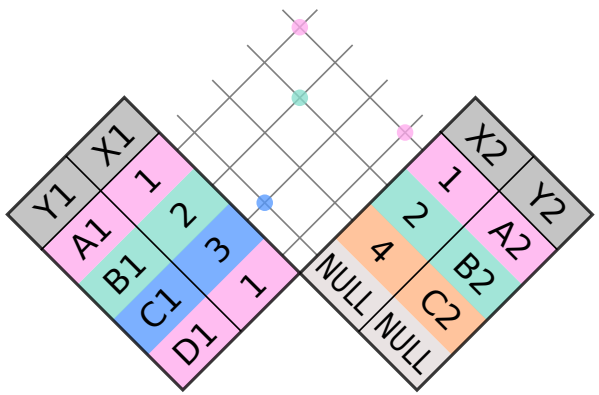
Four major types of joins

X1	Y1
1	A1
2	B1
3	C1
1	D1

X2	Y2
1	A2
2	B2
4	C2

```
>>> df1 = spark.createDataFrame([(1,"A1"),(1,"B1"),(2,"C1"),(3,"D1")], ["X1","Y1"])
>>> df2 = spark.createDataFrame([(1,"A2"),(2,"B2"),(4,"C2")], ["X2","Y2"])
```

Left join:



X1	Y1	X2	Y2
1	A1	1	A2
2	B1	2	B2
3	C1	NULL	NULL
1	D1	1	A2

```
>>> df1.join(df2, df1.X1 == df2.X2, "left").show()
```

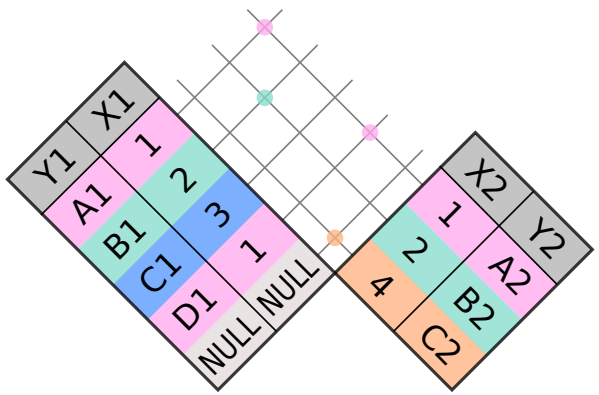
Four major types of joins

X1	Y1
1	A1
2	B1
3	C1
1	D1

X2	Y2
1	A2
2	B2
4	C2

```
>>> df1 = spark.createDataFrame([(1,"A1"),(1,"B1"),(2,"C1"),(3,"D1")], ["X1","Y1"])
>>> df2 = spark.createDataFrame([(1,"A2"),(2,"B2"),(4,"C2")], ["X2","Y2"])
```

Right join:



X1	Y1	X2	Y2
1	A1	1	A2
1	D1	1	A2
2	B1	2	B2
NULL	NULL	4	C2

```
>>> df1.join(df2, df1.X1 == df2.X2, "right").show()
```

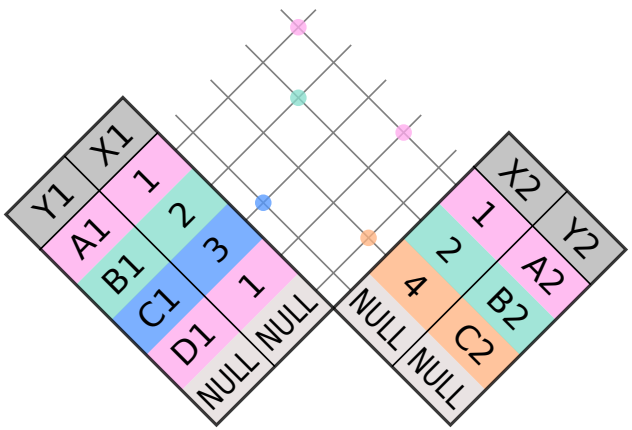
Four major types of joins

X1	Y1
1	A1
2	B1
3	C1
1	D1

X2	Y2
1	A2
2	B2
4	C2

```
>>> df1 = spark.createDataFrame([(1,"A1"),(1,"B1"),(2,"C1"),(3,"D1")], ["X1","Y1"])
>>> df2 = spark.createDataFrame([(1,"A2"),(2,"B2"),(4,"C2")], ["X2","Y2"])
```

Full join:



X1	Y1	X2	Y2
1	A1	1	A2
1	D1	1	A2
2	B1	2	B2
3	C1	NULL	NULL
NULL	NULL	4	C2

```
>>> df1.join(df2, df1.X1 == df2.X2, "full").show()
```

1 Sorting algorithms
- Sort in Apache Spark
- Sort in CUDA-X

2 Joins
- Types of joins
- Join algorithms in Spark

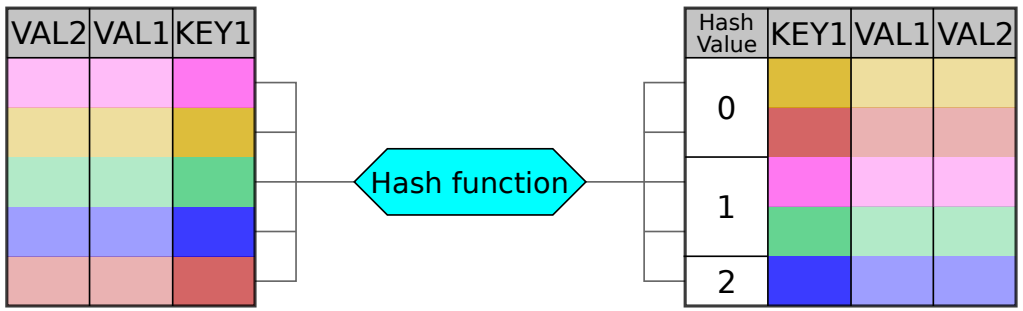
3 Adaptive Query Execution (AQE)

4 Summary

Hash join

Hash join is an efficient algorithm when one of the dataframes is small. A hash join contain two phases:

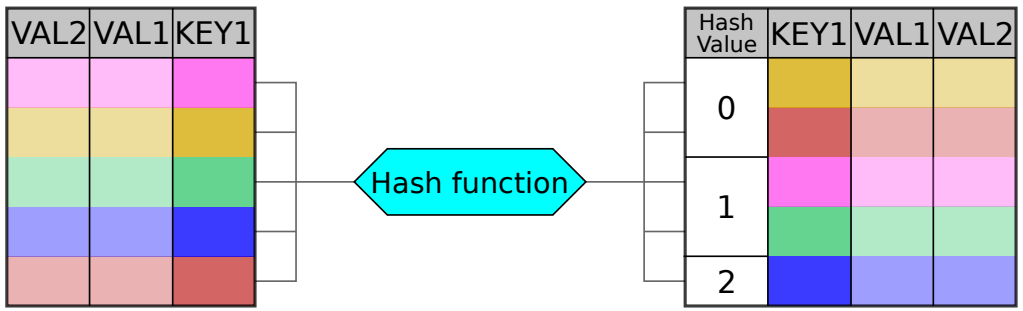
- **Build phase:** Build a hash table using the smaller dataframe:



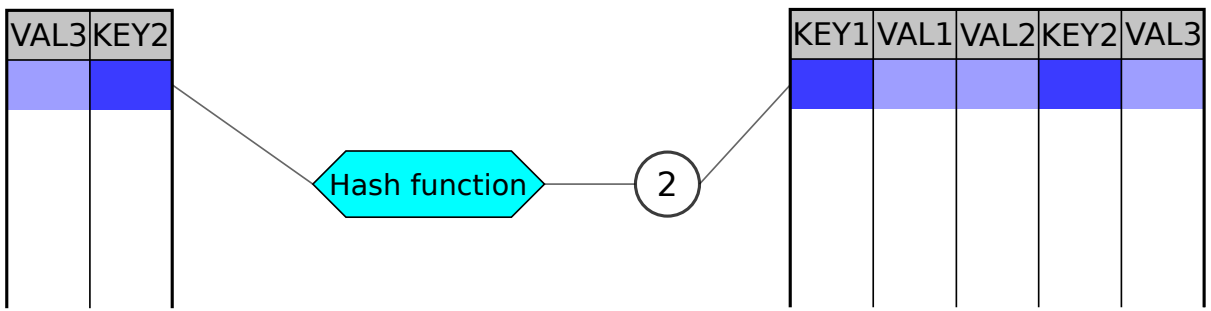
Hash join

Hash join is an efficient algorithm when one of the dataframes is small. A hash join contain two phases:

- **Build phase:** Build a hash table using the smaller dataframe:



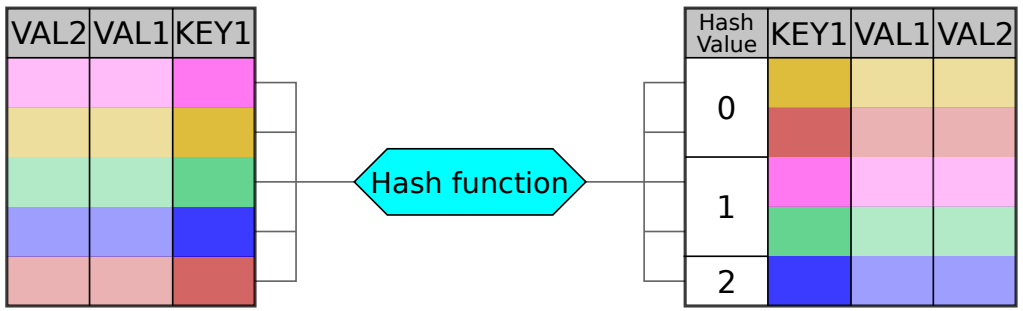
- **Probe phase:** Scan each row of the other dataframe. Find the same key from the hash table and perform the join:



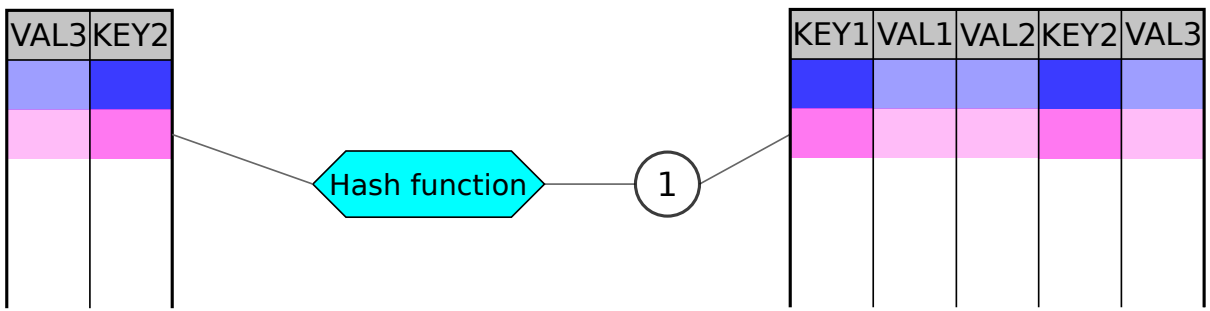
Hash join

Hash join is an efficient algorithm when one of the dataframes is small. A hash join contain two phases:

- **Build phase:** Build a hash table using the smaller dataframe:



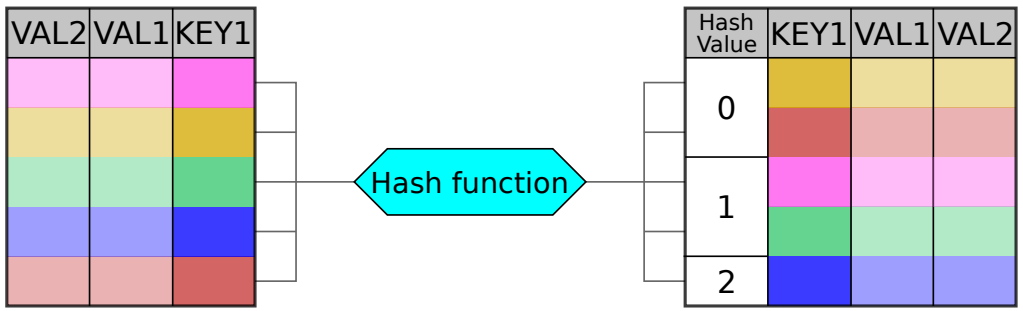
- **Probe phase:** Scan each row of the other dataframe. Find the same key from the hash table and perform the join:



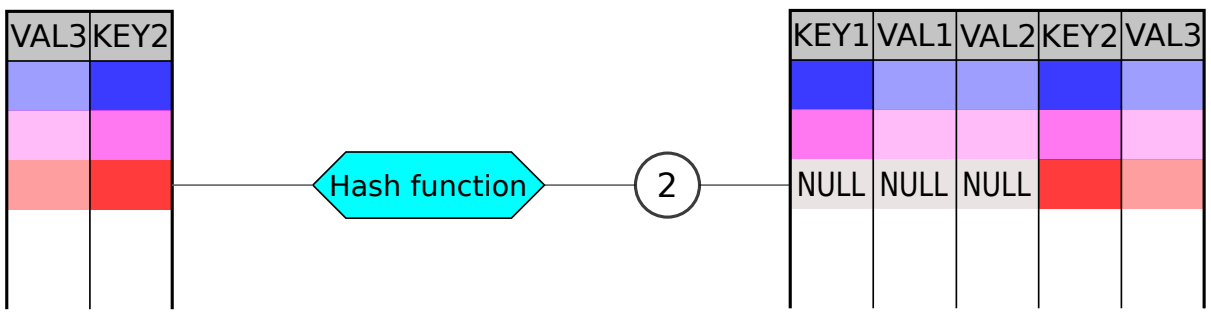
Hash join

Hash join is an efficient algorithm when one of the dataframes is small. A hash join contain two phases:

- **Build phase:** Build a hash table using the smaller dataframe:



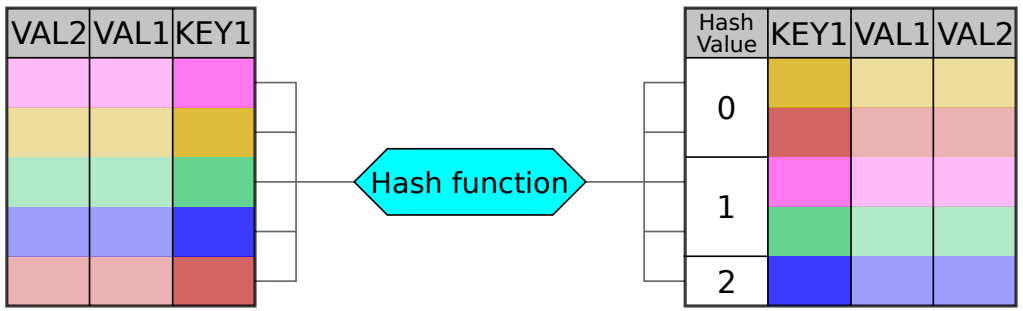
- **Probe phase:** Scan each row of the other dataframe. Find the same key from the hash table and perform the join:



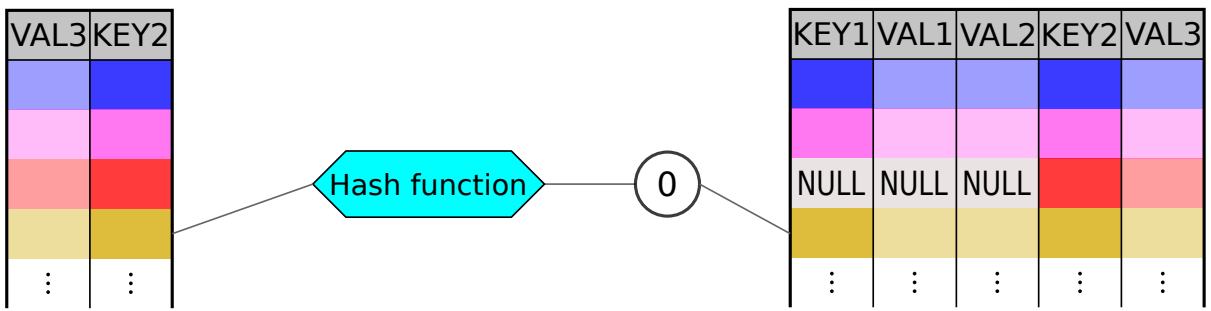
Hash join

Hash join is an efficient algorithm when one of the dataframes is small. A hash join contain two phases:

- **Build phase:** Build a hash table using the smaller dataframe:



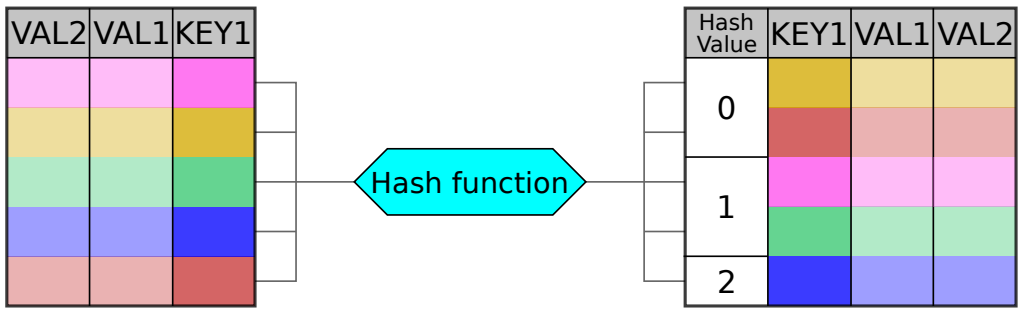
- **Probe phase:** Scan each row of the other dataframe. Find the same key from the hash table and perform the join:



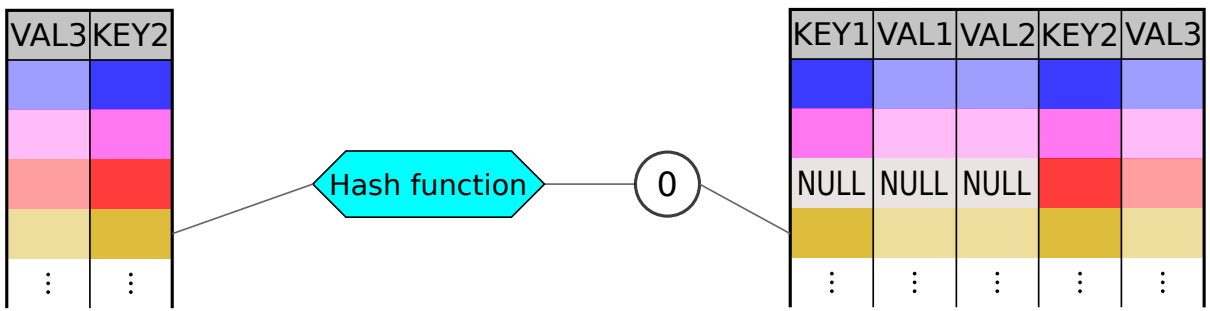
Hash join

Hash join is an efficient algorithm when one of the dataframes is small. A hash join contain two phases:

- **Build phase:** Build a hash table using the smaller dataframe:



- **Probe phase:** Scan each row of the other dataframe. Find the same key from the hash table and perform the join:



Keys that cannot be found in the hash table should be dealt with according to the type of the join.

Distributed hash join

- The hash join requires storing the hash table in memory, and is therefore applicable only when one of the dataframes is small.

Distributed hash join

- The hash join requires storing the hash table in memory, and is therefore applicable only when one of the dataframes is small.
- When dataframes are distributed, Spark scales up hash join using two approaches:

Distributed hash join

- The hash join requires storing the hash table in memory, and is therefore applicable only when one of the dataframes is small.
- When dataframes are distributed, Spark scales up hash join using two approaches:

1. **Broadcast hash join:** The hash table is collected to the driver and then broadcast to all executors, so that the hash join can be performed locally on each executor.

   ☞ This method requires memory to store the entire hash table on all exectutors.

Distributed hash join

- The hash join requires storing the hash table in memory, and is therefore applicable only when one of the dataframes is small.
- When dataframes are distributed, Spark scales up hash join using two approaches:

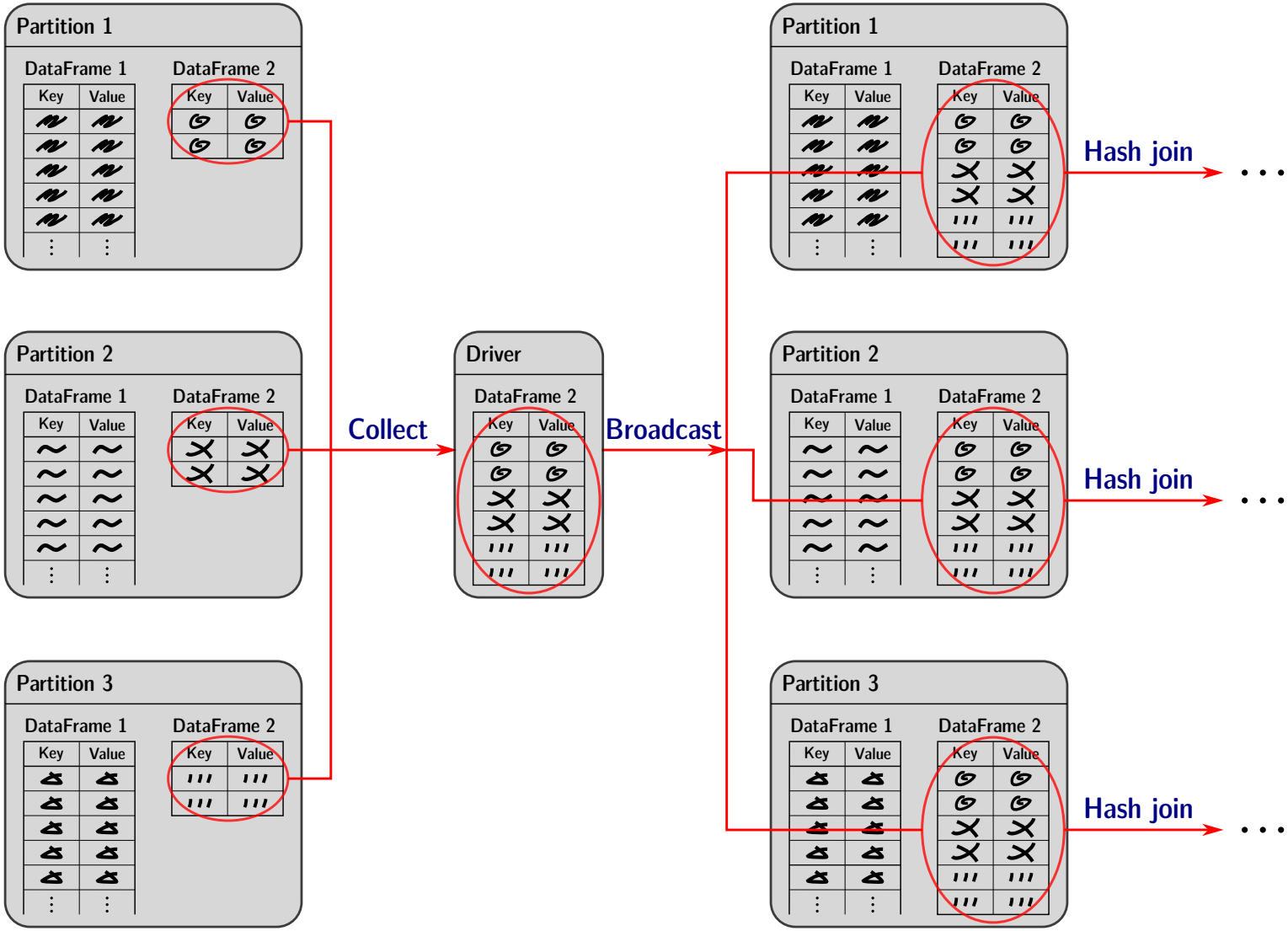
1. **Broadcast hash join:** The hash table is collected to the driver and then broadcast to all executors, so that the hash join can be performed locally on each executor.

   ☞ This method requires memory to store the entire hash table on all exectutors.

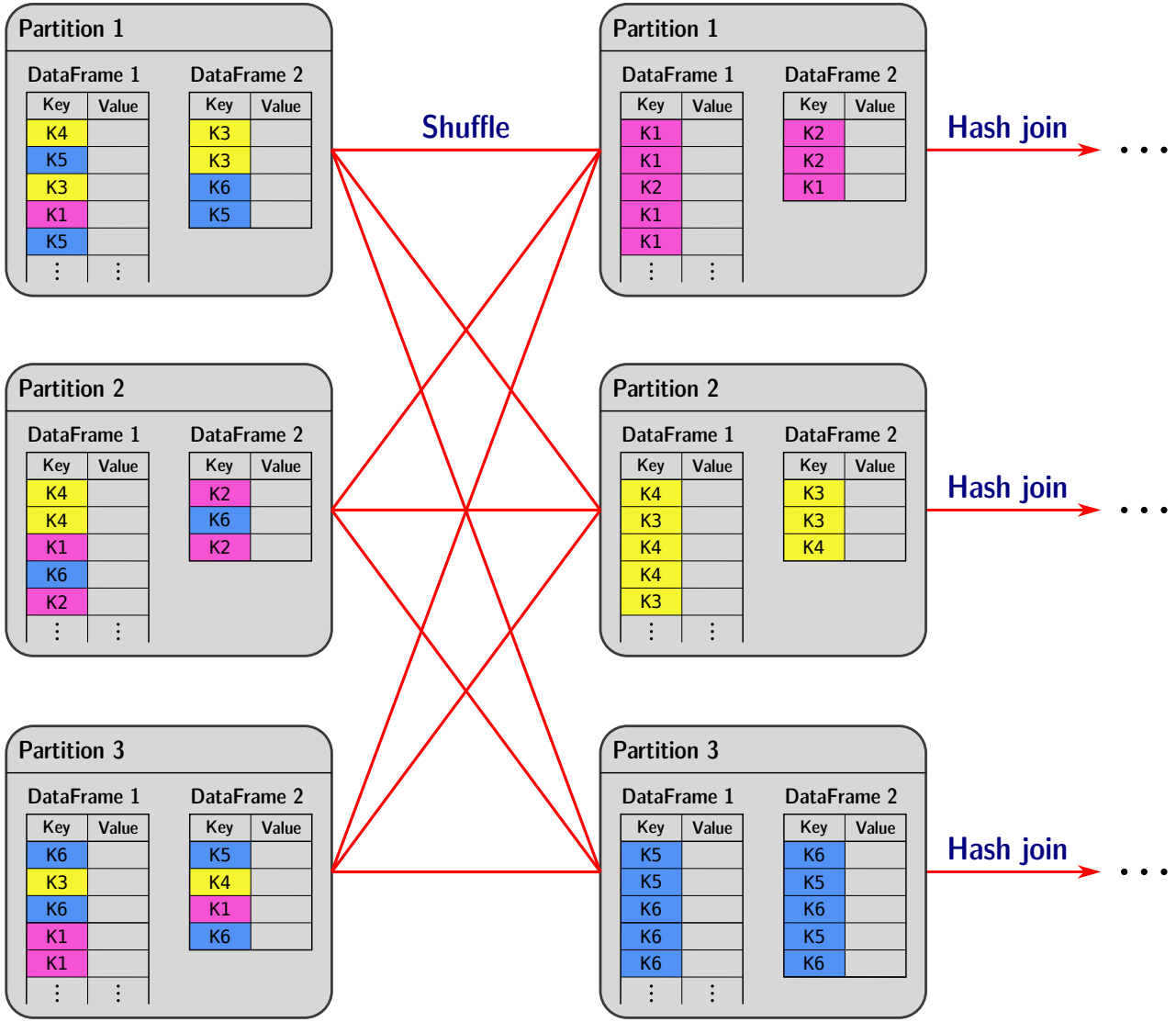
2. **Shuffle hash join:** Rows with the same join key are shuffle to the same partition, so that rows on different partitions are never joined.

   ☞ This method requires memory to store a local hash table on each exectutor.

Broadcast hash join



Shuffle hash join



Sort merge join

Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.

X1	Y1
1	A1
2	B1
3	C1
1	D1
2	E1

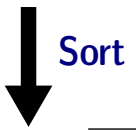
X2	Y2
1	A2
2	B2
4	C2
2	D2

Sort merge join

Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.

X1	Y1
1	A1
2	B1
3	C1
1	D1
2	E1

X2	Y2
1	A2
2	B2
4	C2
2	D2

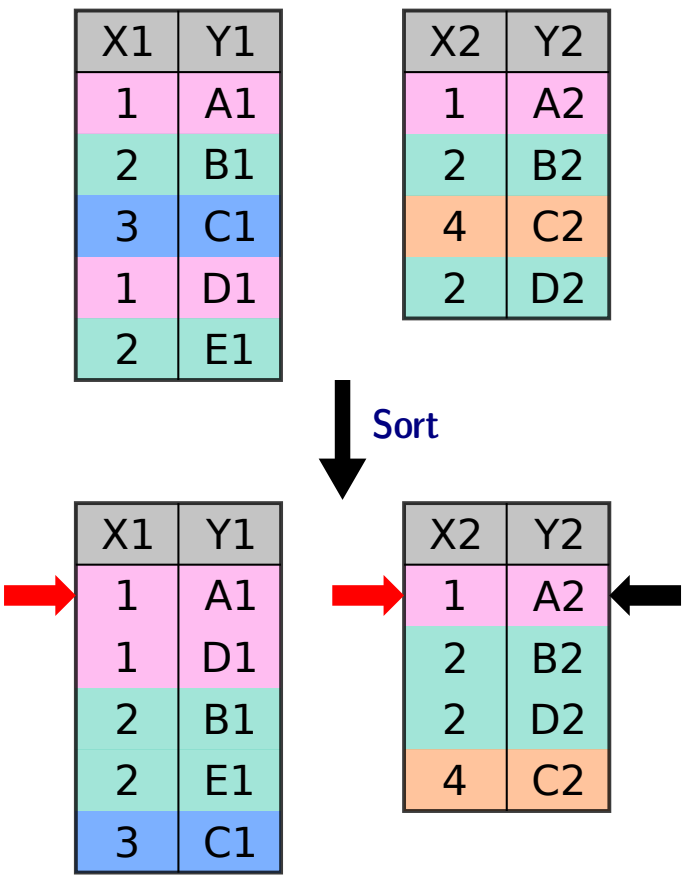


X1	Y1
1	A1
1	D1
2	B1
2	E1
3	C1

X2	Y2
1	A2
2	B2
2	D2
4	C2

Sort merge join

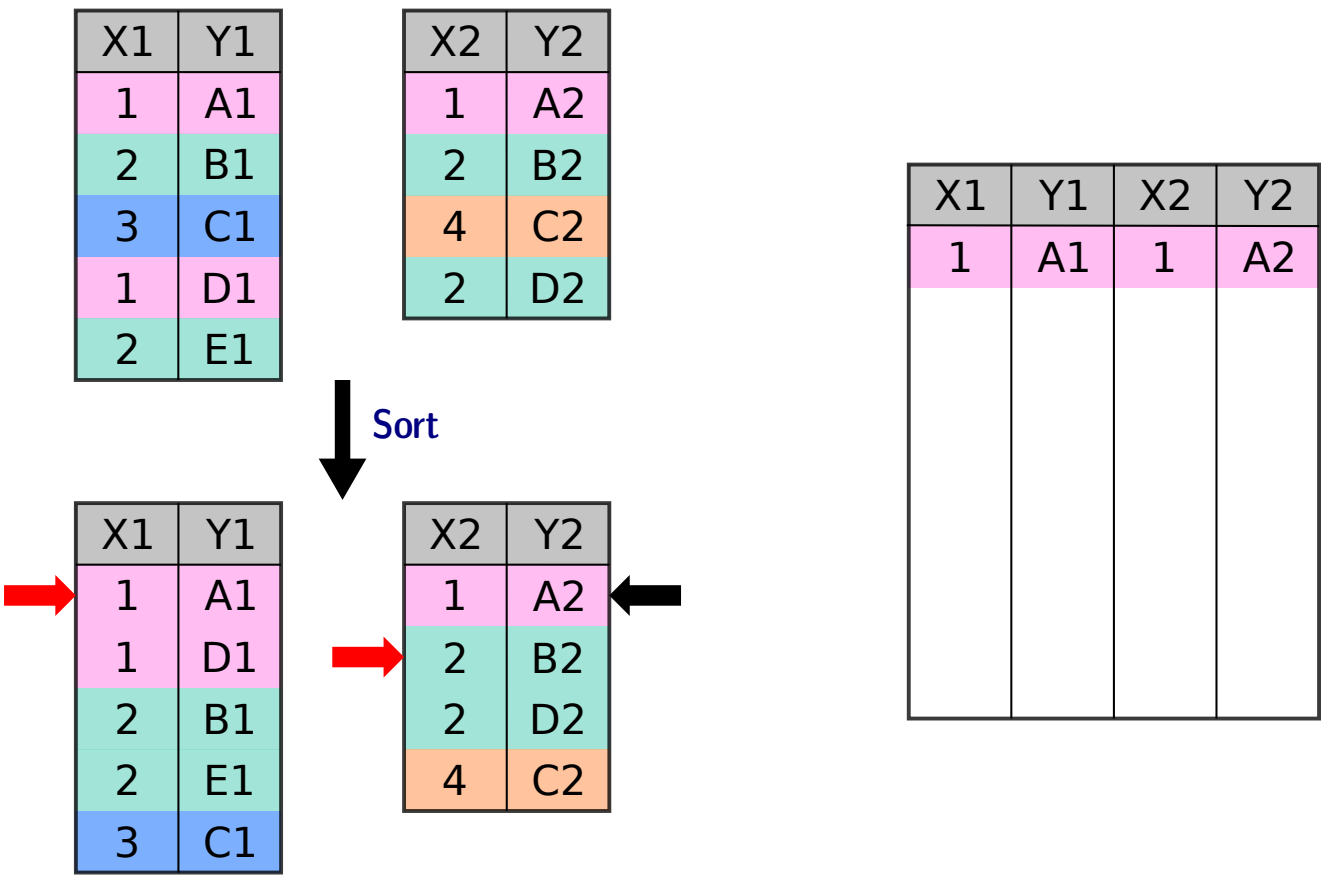
Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



X1	Y1	X2	Y2
1	A1	1	A2

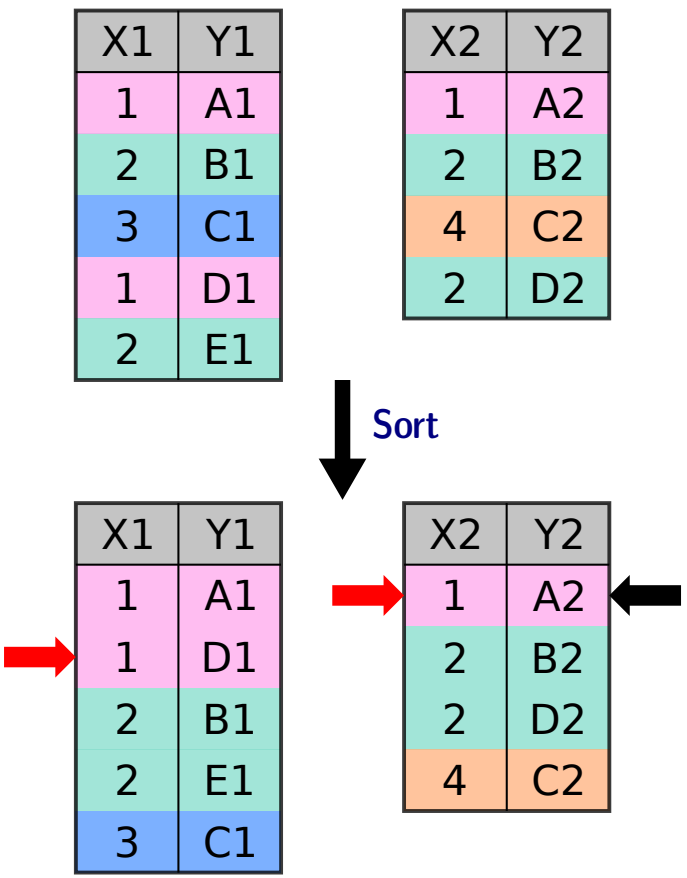
Sort merge join

Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



Sort merge join

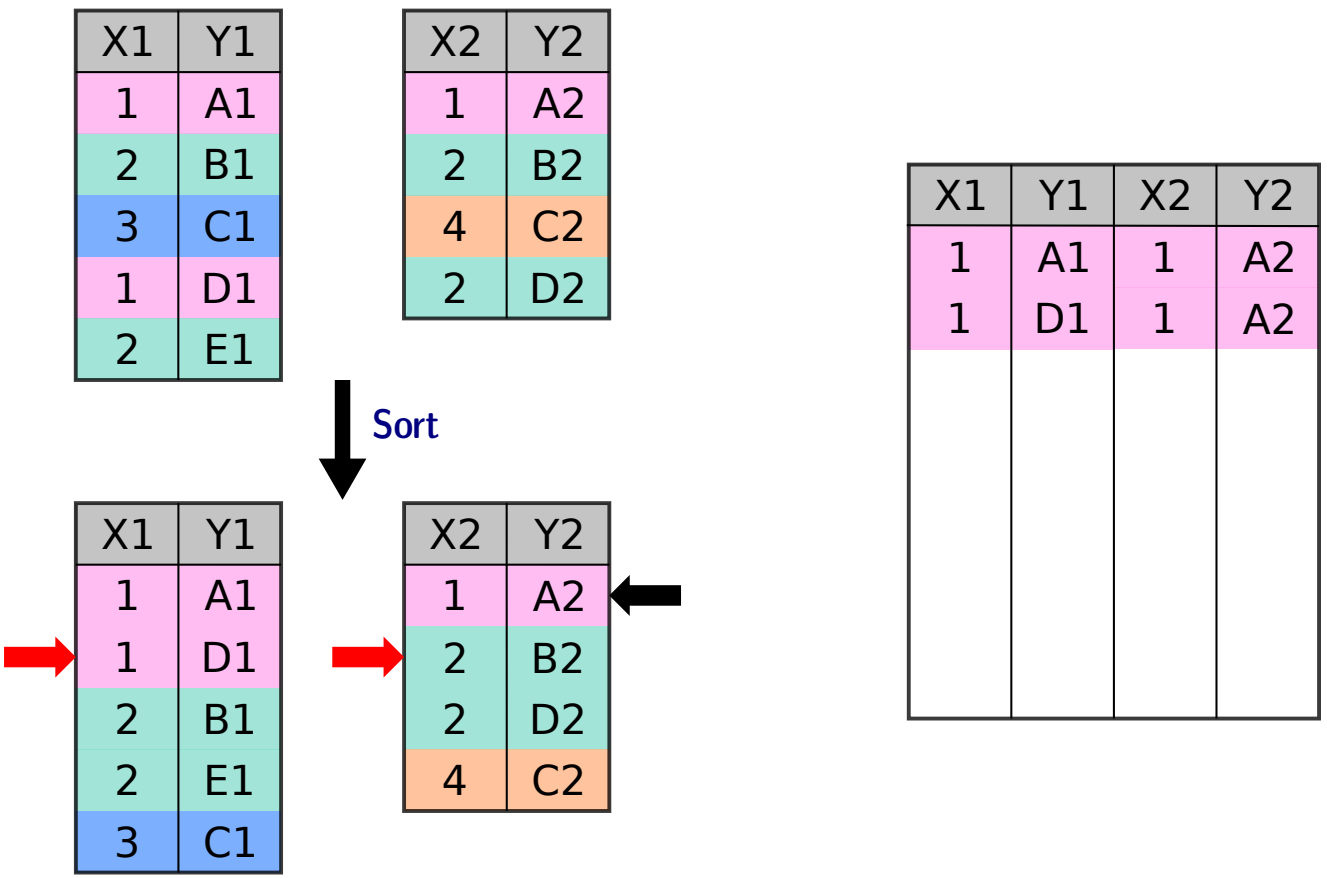
Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



X1	Y1	X2	Y2
1	A1	1	A2
1	D1	1	A2

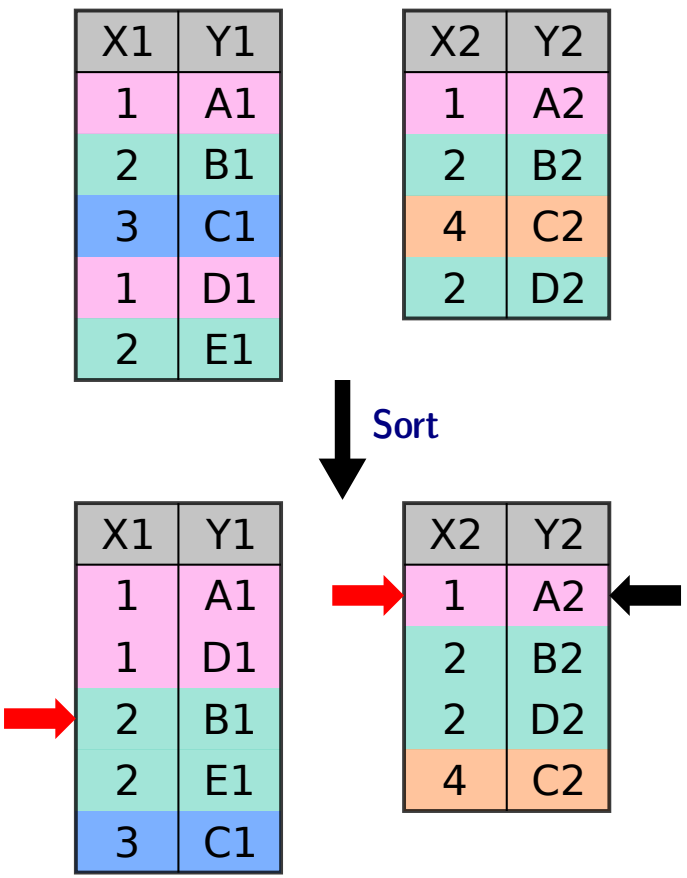
Sort merge join

Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



Sort merge join

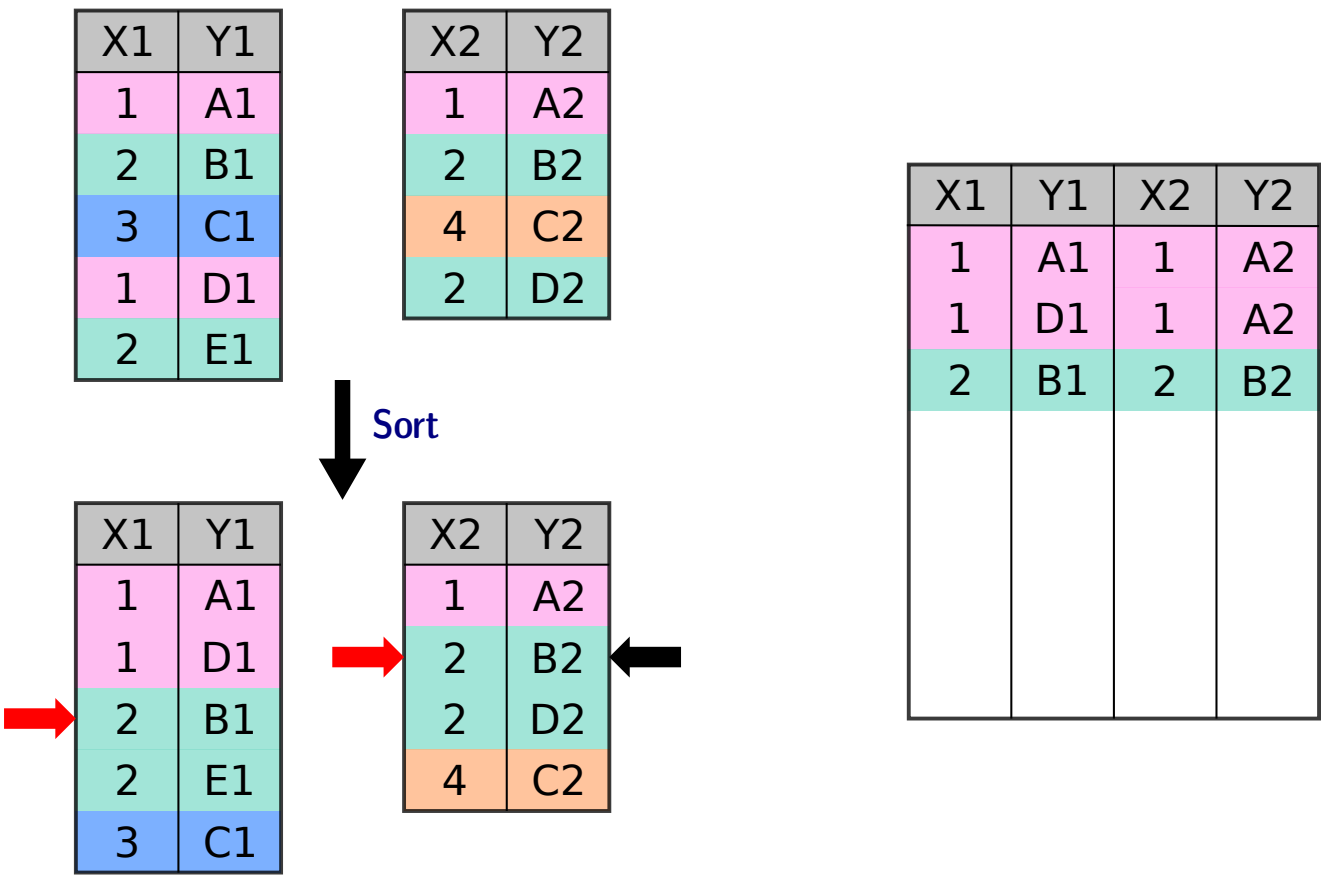
Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



X1	Y1	X2	Y2
1	A1	1	A2
1	D1	1	A2

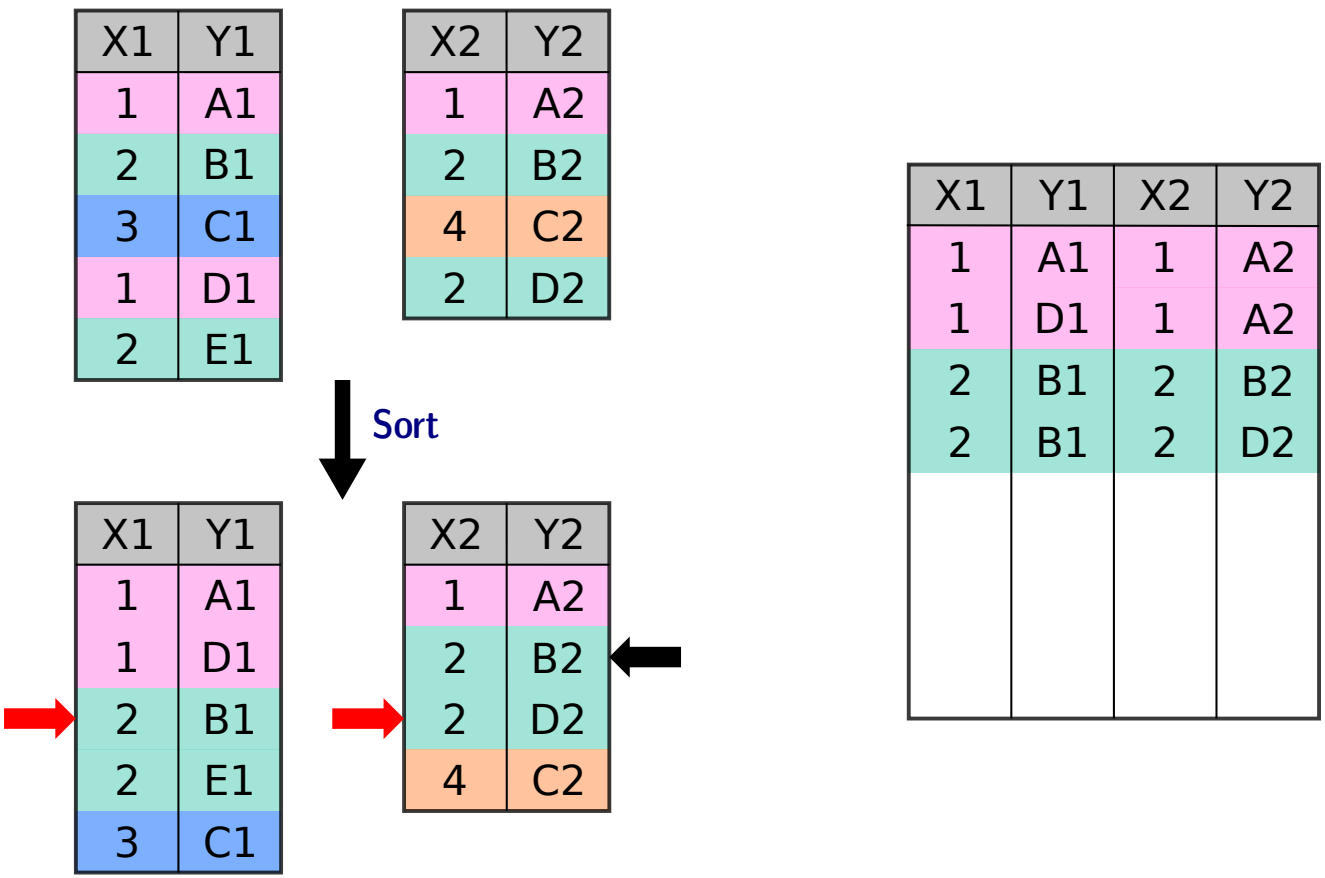
Sort merge join

Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



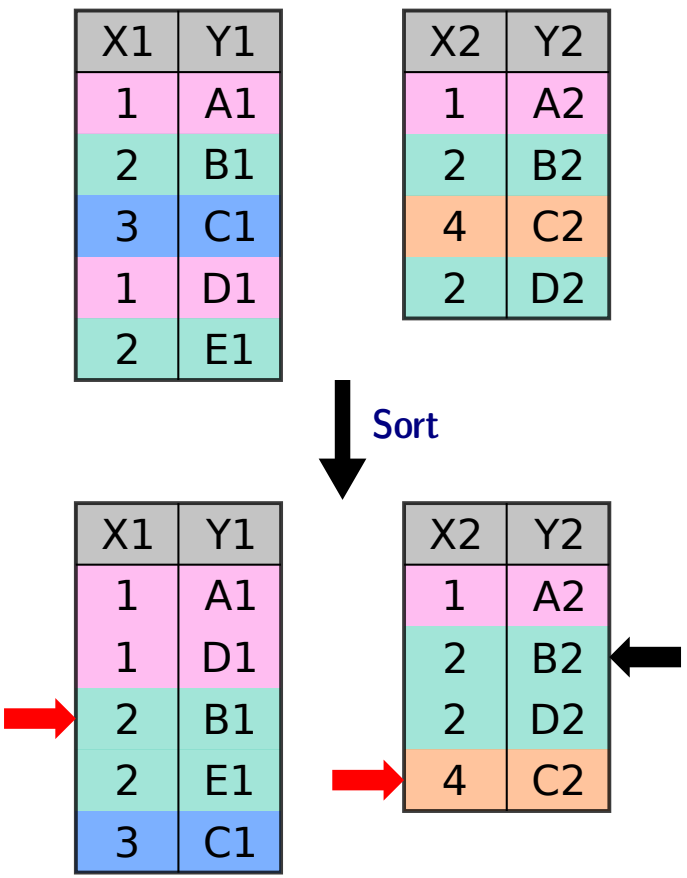
Sort merge join

Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



Sort merge join

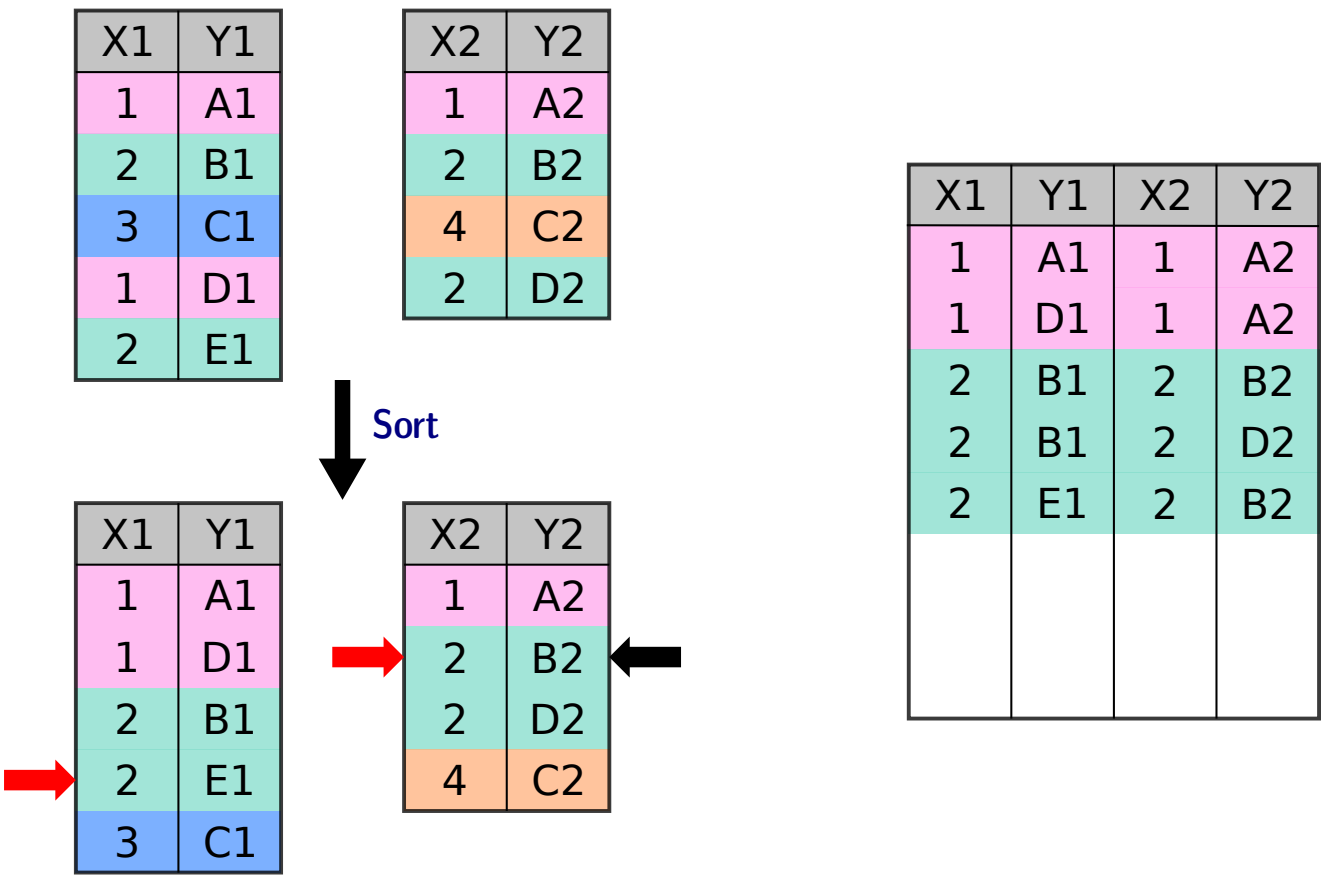
Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



X1	Y1	X2	Y2
1	A1	1	A2
1	D1	1	A2
2	B1	2	B2
2	B1	2	D2

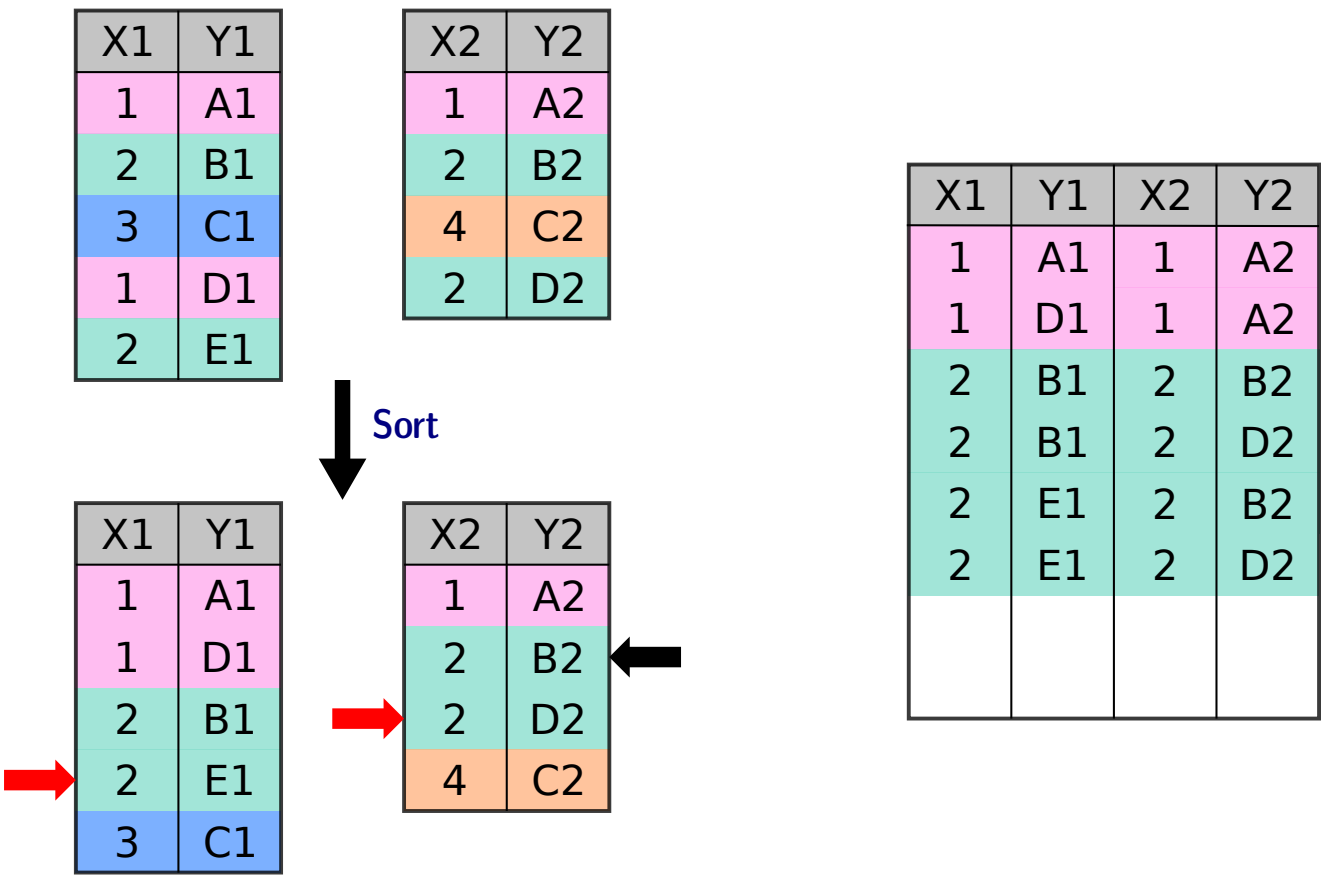
Sort merge join

Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



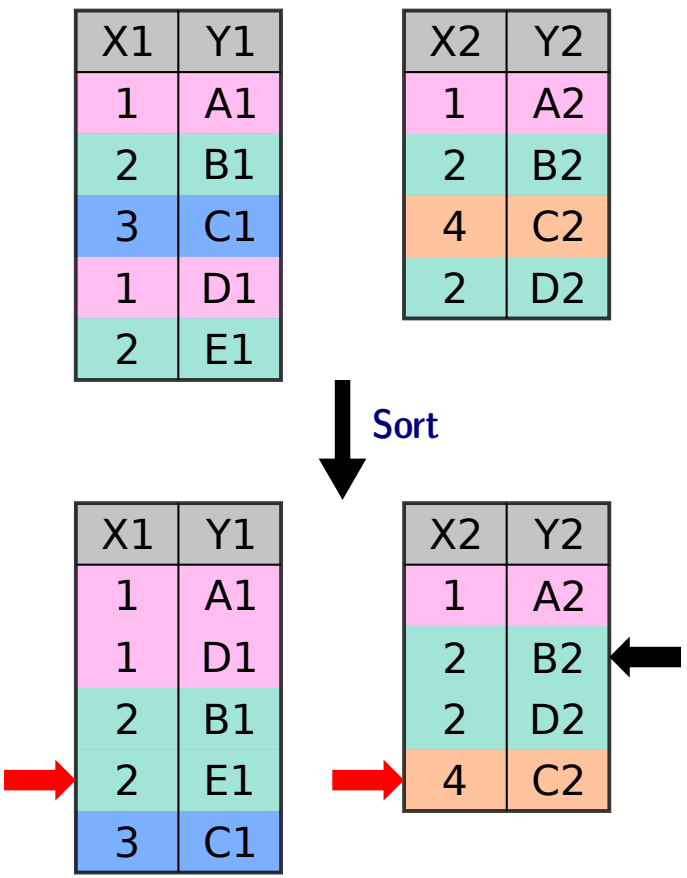
Sort merge join

Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



Sort merge join

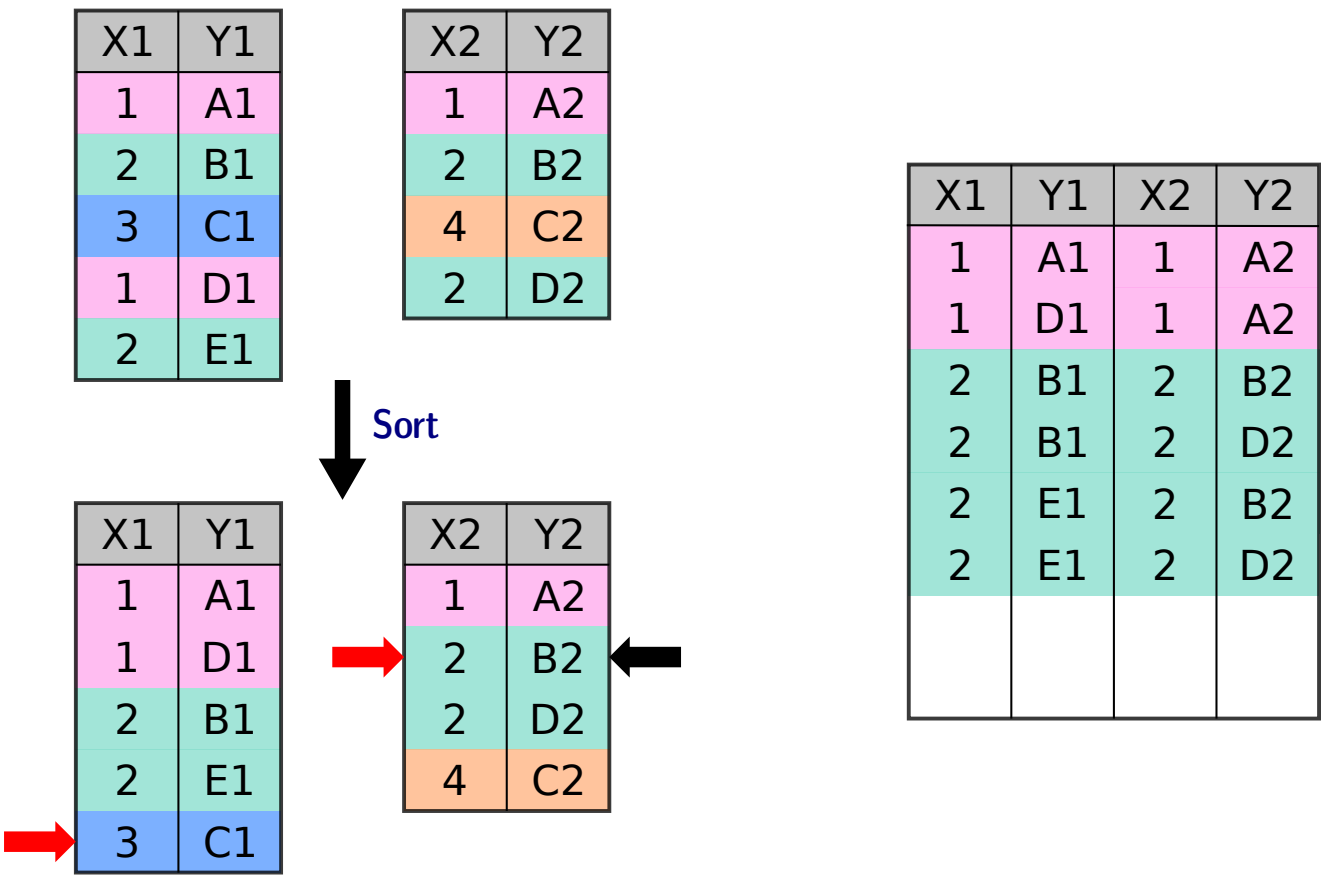
Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



X1	Y1	X2	Y2
1	A1	1	A2
1	D1	1	A2
2	B1	2	B2
2	B1	2	D2
2	E1	2	B2
2	E1	2	D2

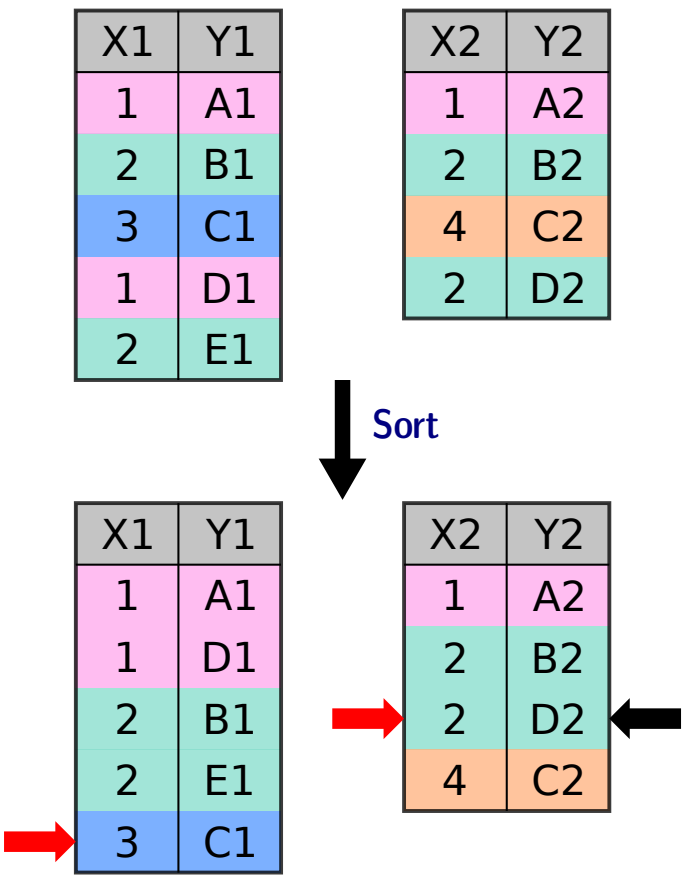
Sort merge join

Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



Sort merge join

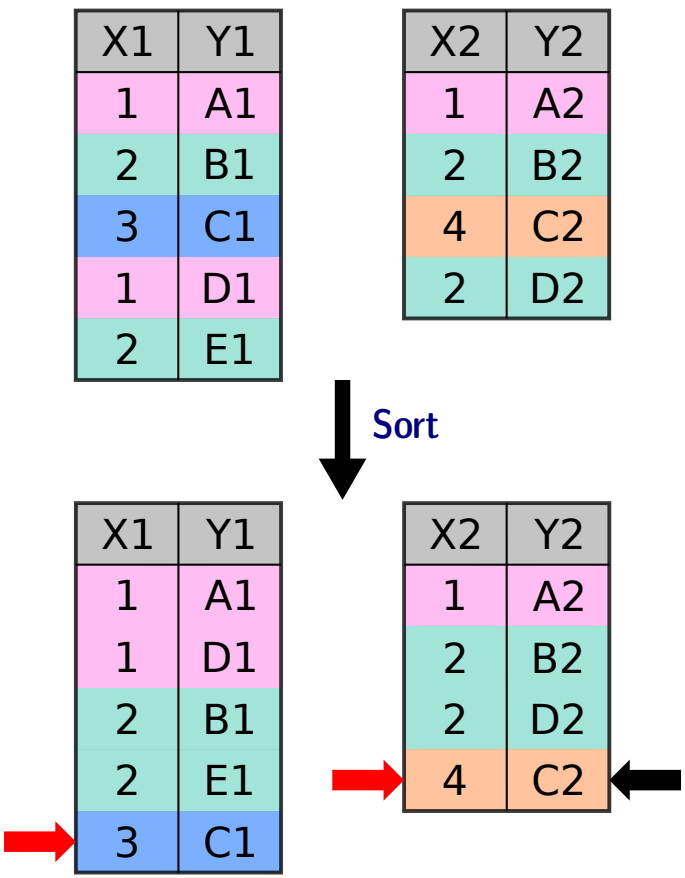
Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



X1	Y1	X2	Y2
1	A1	1	A2
1	D1	1	A2
2	B1	2	B2
2	B1	2	D2
2	E1	2	B2
2	E1	2	D2

Sort merge join

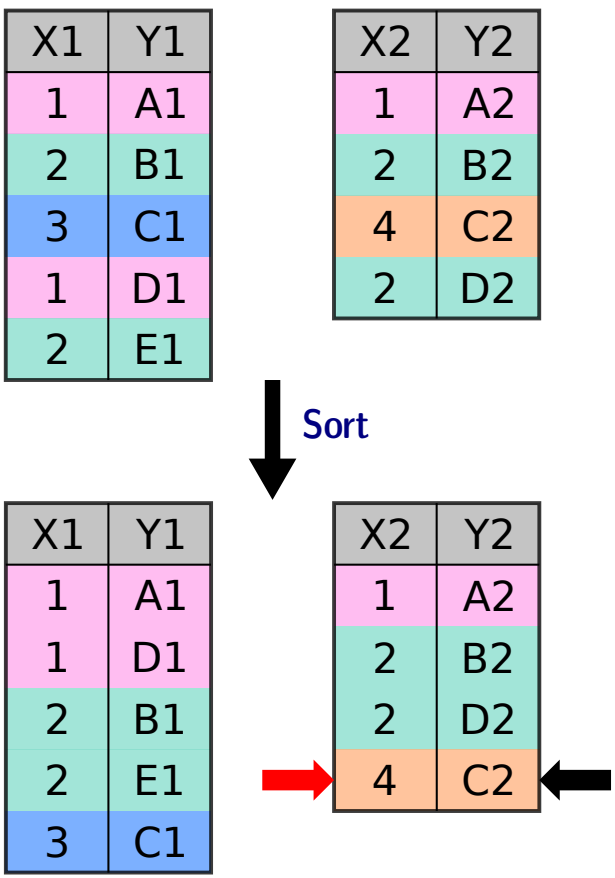
Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



X1	Y1	X2	Y2
1	A1	1	A2
1	D1	1	A2
2	B1	2	B2
2	B1	2	D2
2	E1	2	B2
2	E1	2	D2
3	C1	NULL	NULL

Sort merge join

Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.



X1	Y1	X2	Y2
1	A1	1	A2
1	D1	1	A2
2	B1	2	B2
2	B1	2	D2
2	E1	2	B2
2	E1	2	D2
3	C1	NULL	NULL
NULL	NULL	4	C2

Sort merge join

Sort merge join is another join algorithm that requires sorting by the join key, so that the two DataFrames can be merged by a linear scan of both DataFrames.

X1	Y1
1	A1
2	B1
3	C1
1	D1
2	E1

X2	Y2
1	A2
2	B2
4	C2
2	D2

Sort

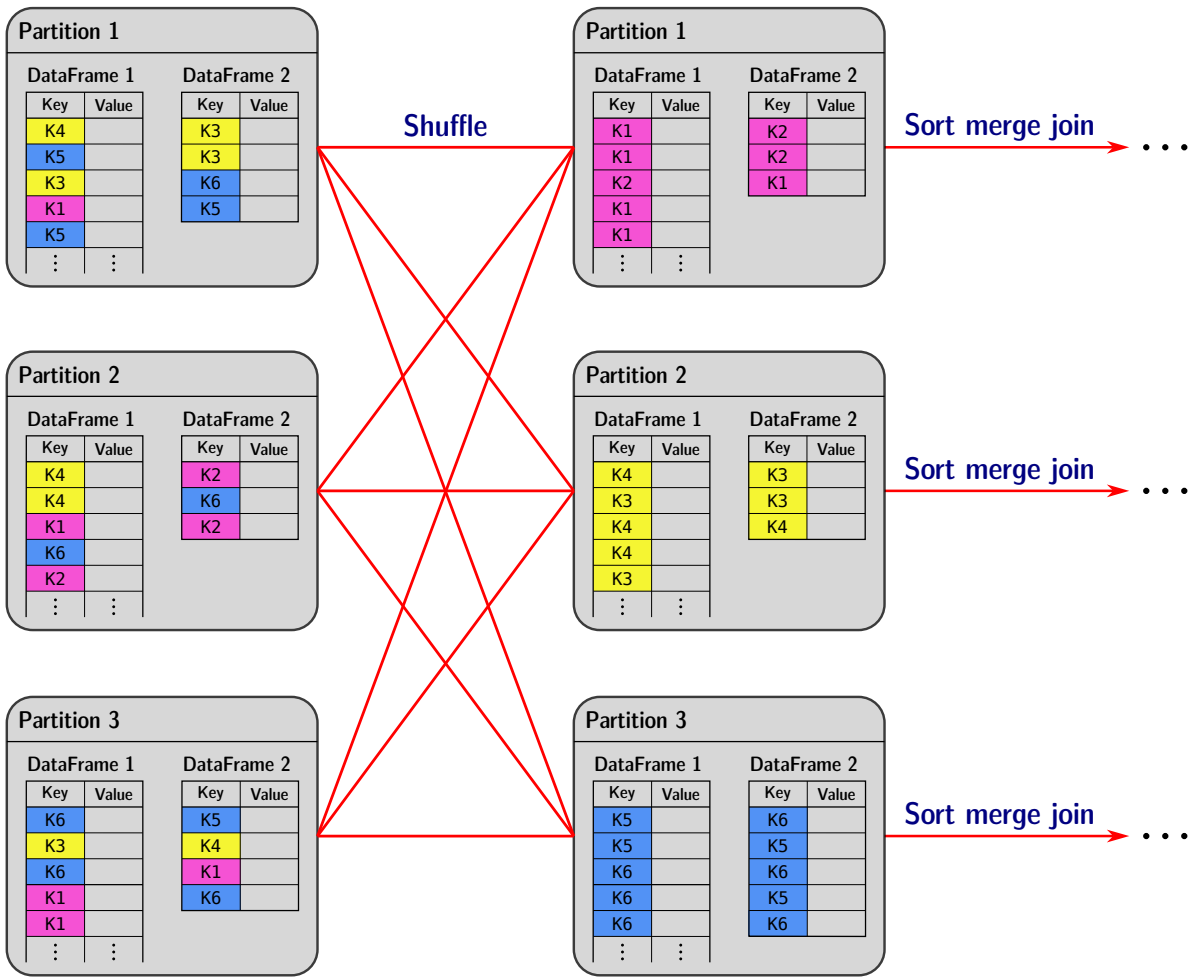
X1	Y1
1	A1
1	D1
2	B1
2	E1
3	C1

X2	Y2
1	A2
2	B2
2	D2
4	C2

X1	Y1	X2	Y2
1	A1	1	A2
1	D1	1	A2
2	B1	2	B2
2	B1	2	D2
2	E1	2	B2
2	E1	2	D2
3	C1	NULL	NULL
NULL	NULL	4	C2

Shuffle sort merge join

When the DataFrames are stored on multiple nodes, we can also perform a shuffle before the sort merge join like the shuffle hash join:



How Spark chooses the join strategy

By default, Spark chooses the join strategy according to the following rules:

- If one DataFrame is small enough to be broadcast, choose broadcast hash join.
- If broadcast hash join cannot be used and the join keys are sortable, pick shuffle sort merge join.
- Otherwise, pick shuffle hash join if one DataFrame is small enough to build the local hash map.
- Choose other methods if all the three methods are not applicable.[1]

Parameters can be tuned to change Spark's preference. Search for "Join Hints" for more details.

[1] Spark supports two more join algorithms: Cartesian join and broadcast nested loop join.

1 Sorting algorithms

2 Joins

3 Adaptive Query Execution (AQE)

4 Summary

What is Adaptive Query Execution (AQE)?

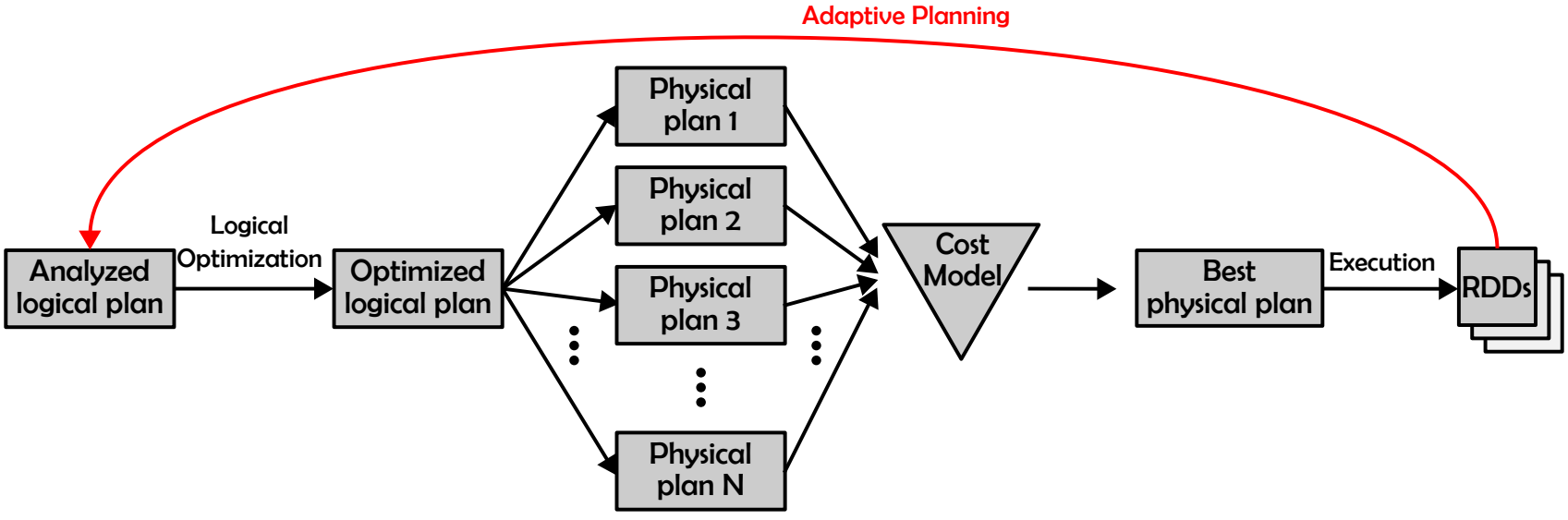
Adaptive Query Execution (AQE) is an optimization technique in Spark SQL that makes use of the runtime statistics to choose the most efficient query execution plan, which is enabled by default since Apache Spark 3.2.0. [2]

Major features of AQE:

- Coalescing post-shuffle partitions
- Converting sort merge join to other join algorithms
- Skew join optimization

[2] https://spark.apache.org/docs/latest/sql-performance-tuning.html

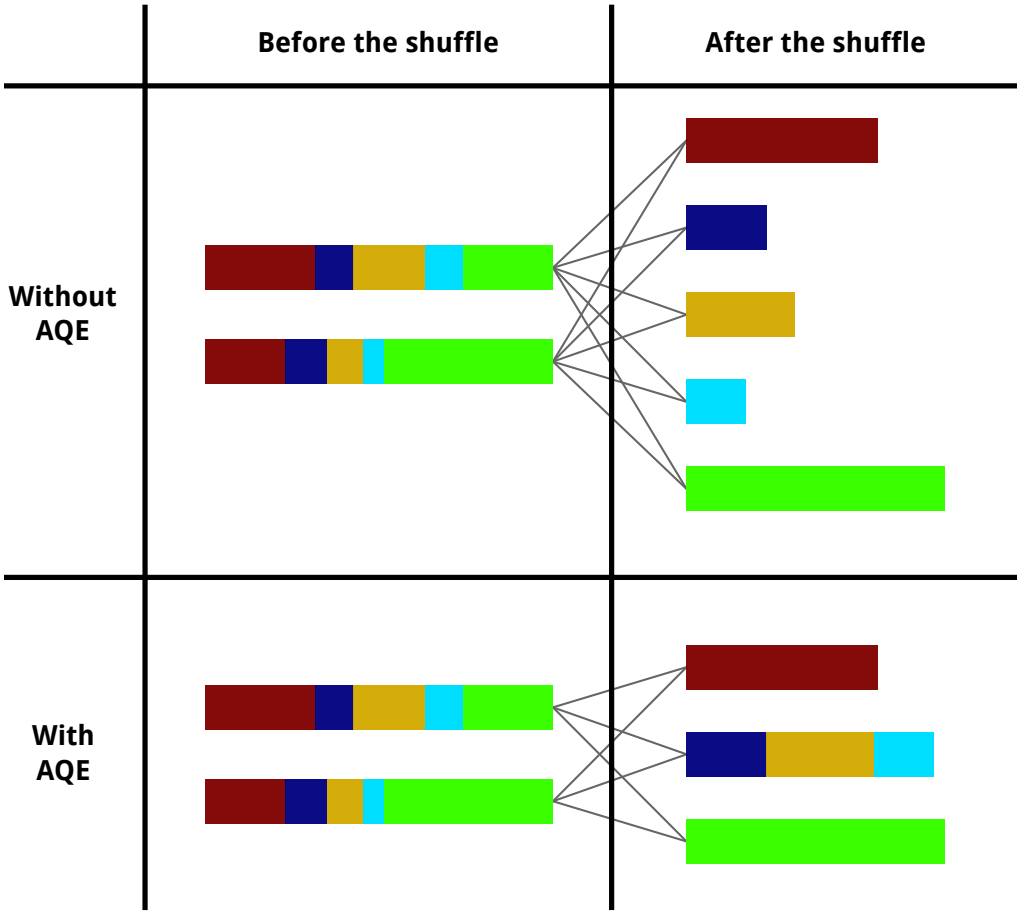
How AQE works



- When an action is performed, AQE kicks off all stages that do not depend on other stages.
- When one or more of these stages complete, AQE runs the optimizer according to the runtime statistics and updates the logical plan.
- The new logical plan is converted to the physical plan and carried out in following stages.
- The procedure repeats until all stages complete.

Coalescing post-shuffle partitions

In a shuffle, AQE coalesces contiguous small partitions to get better balance of data sizes across partitions.

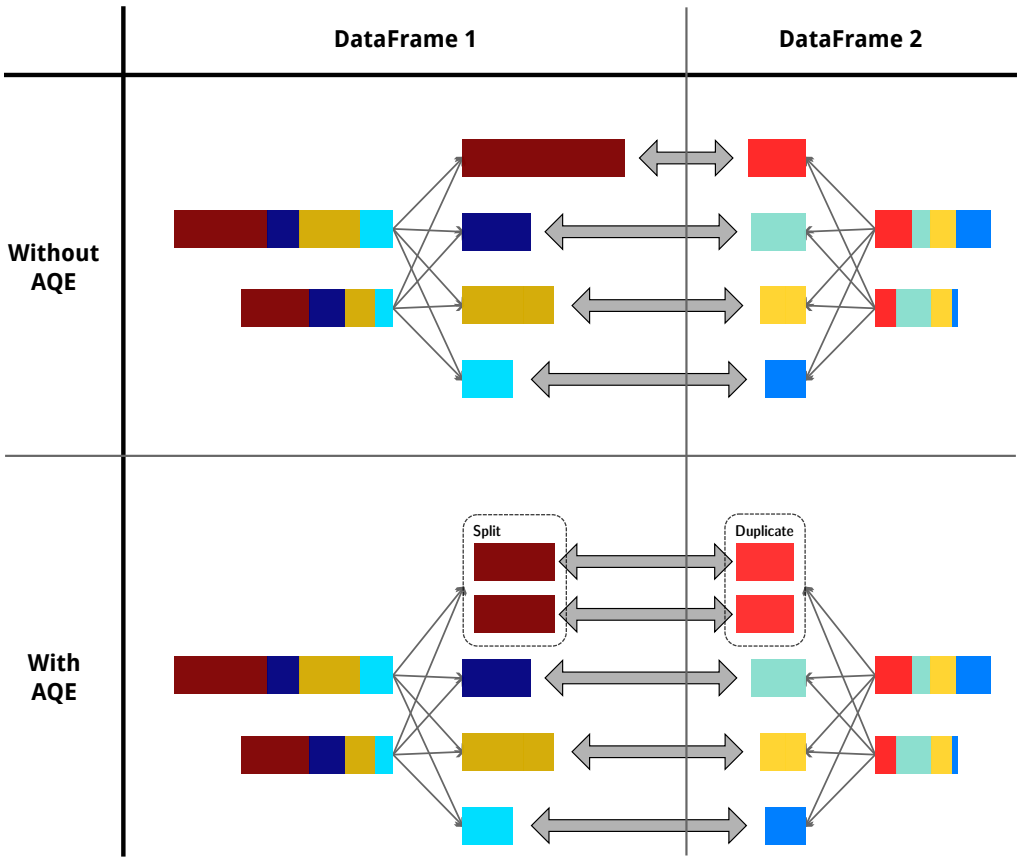


Converting sort merge join to other join algorithms

- Spark chooses the join alogrithm according to estimates of the sizes of the DataFrames.
- However, the estimation may go wrong when the join happens after a series of other operations.
- According to the runtime statistics, AQE may convert sort merge join to broadcast hash join if it finds one table is way smaller than the initial estimate.
- Similarly, AQE may also convert sort merge join to shuffle hash join if it finds that the hash table fits the memory.

Skew join optimization

Data skew can severely downgrade the performance of join queries. AQE handles skew in sort merge join by splitting (and replicating if needed) skewed tasks into roughly evenly sized tasks.



1 Sorting algorithms

2 Joins

3 Adaptive Query Execution (AQE)

4 Summary

Summary

- Sort in Spark
  - Partitioner: Partition by range
  - Sampler: Reservior sampling
  - Sorting algorithms: Timsort
- Sort in CUDA-X
  - External merge sort
  - The implementation of GPU sort using a priority queue
- Joins in Spark
  - Four types of joins: inner, left, right, full
  - Three join algorithms: broadcast hash, shuffle hash, shuffle sort merge
- Adaptive query execution
  - When AQE optimizes the plans
  - How AQE optimizes the plans